
Building Reliable Software with Python

A Practical Guide to Clean Code,
Data Wrangling, Web Services & APIs



Vinod Kumar Kayartaya

<https://vinod.co/>

Preface

Welcome to *Building Reliable Software with Python*. This book is designed to serve as a comprehensive, hands-on guide for developers, data scientists, and software engineers who want to master Python not just as a scripting tool, but as a robust platform for engineering production-grade software.

Python is often praised for its simplicity and readability. However, writing code that runs is very different from writing code that is clean, maintainable, and resilient under production stress. As software systems grow in complexity, developers must understand core architecture patterns, advanced object-oriented constructs, database design, exception safety, and performance constraints.

How This Book is Structured

This textbook is structured into 9 logical chapters:

1. **Chapter 1: Python History, Philosophy & Capabilities** – Introduces the philosophy behind Python and setting up virtual environments.
2. **Chapter 2: Sequence Types — Strings & Tuples** – Covers sequence slices, memory layouts, and immutability.
3. **Chapter 3: Mutable Sequences — Working with Lists** – Focuses on list manipulation, list comprehensions, and resource management.
4. **Chapter 4: Dictionaries & Exception Handling** – Details associative mapping structures and Exception-safe coding.
5. **Chapter 5: Functions, Scopes & Regular Expressions** – Explores scoping scopes, recursion, custom decorators, and regular expression text engines.
6. **Chapter 6: Object-Oriented Programming (OOP) in Python** – Examines cooperative multiple inheritance, Method Resolution Order (MRO), encapsulation, and dunder protocols.
7. **Chapter 7: Libraries, Debugging, Logging & Databases** – Solidifies standard tools, SQLite database interfaces, logging patterns, and pdb debug routines.
8. **Chapter 8: Data Science Foundations, Wrangling & Visualization** – Combines array vectorization with NumPy, Pandas data Wrangling/Pivot Tables, and plotting layouts

using Matplotlib, Seaborn, and interactive Plotly Express.

9. Chapter 9: Web Technologies — Micro-Frameworks & Web Scraping – Explores Flask routing, model template bindings, dynamically generated databases, forms, and web scraping crawler pipelines.

Each chapter follows a structured sequence: a detailed conceptual tutorial with copy-pasteable, runnable examples, followed by exactly six (or twelve for combined modules) scenario-based practice assignments classified by difficulty (Easy, Medium, and Difficult). These exercises are designed to simulate real-world software engineering tasks, encouraging hands-on implementation and problem-solving.

Target Audience

Whether you are a student transitioning to professional programming, a data scientist wanting to package models into web APIs, or a backend developer seeking to refine your object-oriented design patterns, this book will provide you with the practical skills required to build reliable, high-performance software.

Vinod Kumar Kayartaya <https://vinod.co/>

Table of Contents

Preface

Chapter 1: Python History, Philosophy & Capabilities

Section 1: Introduction, History & Philosophy

Section 2: Environment Setup & First Program

Section 3: Variables, Operators & Data Types

Section 4: Control Flow & Looping

Practice Assignments

Assignment 1: Leap Year Checker

Assignment 2: Fibonacci Sequence Generator

Assignment 3: Prime Number Checker

Assignment 4: Odd or Even Checker

Assignment 5: Basic Operator Calculator

Assignment 6: Sum of N Natural Numbers

Assignment 7: Multiplication Table Generator

Assignment 8: Score to Grade Converter

Chapter 2: Sequence Types — Strings & Tuples

Section 1: Strings (Immutable Text)

Section 2: Tuples (Immutable Sequences)

Practice Assignments

Assignment 1: Sentence Analysis (Character & Word Count)

Assignment 2: Reversed Uppercased String

Assignment 3: Email Domain Extractor

Assignment 4: Vowel & Consonant Frequency

Assignment 5: Custom Title Case Formatter

Assignment 6: Shift Cipher Encrypter

Assignment 7: Manual Substring Counter

Assignment 8: Name Anonymizer

Assignment 9: Longest Palindromic Substring
Assignment 10: Run-Length String Compression
Assignment 11: Group Anagrams
Assignment 12: Date Validator & Pretty Formatter

Chapter 3: Mutable Sequences — Working with Lists

Section 1: List Fundamentals & Accessing Elements

Section 2: List Comprehensions & Conversions

Section 3: Beginner Pitfalls & Common Mistakes

Practice Assignments

Assignment 1: The Wizard's Magic Bag
Assignment 2: Movie Night Playlist
Assignment 3: The Cargo Train Scanner
Assignment 4: Nightclub VIP Queue
Assignment 5: The Spy's Word Reverser
Assignment 6: Grading on a Curve
Assignment 7: Treasure Map Coordinate Filter
Assignment 8: De-duplicating Shopping Cart
Assignment 9: The Josephus Elimination Game
Assignment 10: Snake Game Board Renderer

Chapter 4: Dictionaries & Exception Handling

Section 1: Associative Mapping Structures (Dictionaries)

Section 2: Robust Error Isolation (Exception Handling)

Practical Examples (Interactive & Runnable)

Practice Assignments

Assignment 1: Inventory Tracker for Campus Bookstore
Assignment 2: Robust Phonebook Contact Registry
Assignment 3: Course Feedback Compiler & Sanitizer
Assignment 4: Atomic E-Commerce Order Processor
Assignment 5: Deep JSON/Configuration Key Traverser
Assignment 6: Atomic Transaction processing with Log Rollback

Chapter 5: Functions, Scopes & Regular Expressions

Section 1: Functional Abstraction & Scoping Rules

Section 2: Lambda Functions & Built-in Helpers

Section 3: Regular Expressions & Text Search Engines

Practical Examples (Interactive & Runnable)

Practice Assignments

Assignment 1: Campus Cafeteria Discount Calculator

Assignment 2: Academic Email Validator

Assignment 3: Corporate Directory Search & Scraper

Assignment 4: Dynamic Data Pipeline with Lambdas & Custom Sorting

Assignment 5: Stateful Ledger Scope Machine (LEGB Scopes & Closures)

Assignment 6: Server Log Analyzer & Traffic Classifier (Advanced RegEx)

Chapter 6: Object-Oriented Programming (OOP) in Python

Section 1: Object-Oriented Foundations (Classes & Objects)

Section 2: Advanced OOP (Inheritance, MRO & Polymorphism)

Section 3: Special Dunder Protocols & Operator Overloading

Practice Assignments

Assignment 1: Smart Home Appliance Control

Assignment 2: Vehicle Fleet Management

Assignment 3: E-Commerce Currency Converter

Assignment 4: Hospital Patient Register

Assignment 5: Multi-Channel Notification System (Multiple Inheritance & MRO)

Assignment 6: Custom Database Record Simulator

Chapter 7: Libraries, Debugging, Logging & Databases

Section 1: Python Standard Libraries

Section 2: Diagnostic Logging

Section 3: Interactive Debugging (pdb)

Section 4: Relational Databases (sqlite3)

Practical Examples (Interactive & Runnable)

Practice Assignments

Assignment 1: System Date Arithmetic Utility
Assignment 2: JSON Configuration Validator
Assignment 3: Corporate Event Logger
Assignment 4: Relational Employee Database CRUD Registry
Assignment 5: SQLite Contact Synchronizer with JSON Transaction Log
Assignment 6: File System Space Monitor & Database Auditing Daemon

Chapter 8: Data Science Foundations, Wrangling & Visualization

Section 1: NumPy Fundamentals & Vectorization

Section 2: SciPy Optimization & Statistical Computing

Section 3: Tabular Analysis Foundations (Pandas)

Practical Examples (Interactive & Runnable)

Section 4: Data Wrangling & Cleaning with Pandas

Section 5: Static & Interactive Data Visualization

Practical Examples (Interactive & Runnable)

Practice Assignments

Assignment 1: Product Order Vector Processor (NumPy Vectorization)
Assignment 2: Northwind Sales Filter (Pandas Filtering)
Assignment 3: Quantity Capping & Reshaping (NumPy Arrays)
Assignment 4: Shipping Freight Slicing (Pandas loc vs. iloc)
Assignment 5: Coordinates Alignment for Port Shipments (NumPy Linear Algebra)
Assignment 6: Shipping Cost Decay Optimizer (SciPy Optimization)
Assignment 7: Northwind Order Shipped Date Sanitizer
Assignment 8: Interactive Freight Cost Trend Visualizer
Assignment 9: Product Supplier Performance Reconciler
Assignment 10: Interactive Product Sales Bubble Plot
Assignment 11: Employee Monthly Sales Summary & Pivot Table
Assignment 12: Interactive Category Revenue by Country Dashboard

Chapter 9: Web Technologies — Micro-Frameworks & Web Scraping

Section 1: Web Architecture & Flask Micro-framework Basics

Section 2: Form Processing & Dynamic Web Databases

Practical Examples (Interactive & Runnable)

Section 3: Web Scraping & Crawler Pipelines

Practice Assignments

- Assignment 1: Academic Greeting Web App
 - Assignment 2: Dynamic Temperature Converter Web App
 - Assignment 3: Secure Administrator Gateway
 - Assignment 4: Academy Course Enrollment System
 - Assignment 5: Patient Ledger API & Query Module
 - Assignment 6: Inventory Management API with JSON Schema Validation
 - Assignment 7: Academic Quotes Collector (BeautifulSoup Scraper)
 - Assignment 8: SQLite-Powered Library Directory (Flask Web App)
 - Assignment 9: Academic Tables Extractor (Pandas & BeautifulSoup)
 - Assignment 10: Dynamic News Portal with Auto-Updater
 - Assignment 11: Recursive Crawler with Depth Limiter
 - Assignment 12: Scrapy Database Pipeline and Product Crawler
-

Chapter 1: Python History, Philosophy & Capabilities

Welcome to your first day of Python programming! Before we write any code, it is essential to understand where Python came from, why it was designed the way it was, and what makes it such a powerful tool in modern computing (especially in Artificial Intelligence and Data Science).

Section 1: Introduction, History & Philosophy

A Brief History of Python

Python was conceived in the **late 1980s** by **Guido van Rossum** at the *Centrum Wiskunde & Informatica* (CWI) in the Netherlands.

- Implementation of the language began in **December 1989** as a hobby project. Guido wanted a successor to the **ABC programming language** that could interface with the **Amoeba distributed operating system** and handle exceptions.
- **First Release (0.9.0)**: February 1991. It included classes, inheritance, exception handling, functions, and core data types like lists, dicts, and strings.
- **Python 2.0**: Released in October 2000. It introduced list comprehensions, garbage collection, and Unicode support.
- **Python 3.0**: Released in December 2008. It was a major, backward-incompatible release designed to fix structural design flaws in the language (such as fixing string representation to be Unicode by default and streamlining division).

NOTE:

*Guido van Rossum was known as Python's **Benevolent Dictator for Life (BDFL)** until he stepped down from the role in July 2018. The language is now governed by a*

five-member steering committee.

Why the name "Python"?

Contrary to popular belief, Python was not named after the snake. Guido van Rossum named the language after the British comedy group **Monty Python**, as he was reading published scripts from "*Monty Python's Flying Circus*" at the time and wanted a name that was short, unique, and slightly mysterious.

The Intent Behind Python (Design Philosophy)

Guido's goal was to design a language that was easy to read, write, and maintain. The core philosophy of Python is summarized in **The Zen of Python** (written by software engineer Tim Peters). You can read it in any Python console by typing:

```
import this
```

Key design tenets include:

- **Readability:** Code is read much more often than it is written. Python uses clean English-like keywords and relies on formatting indentation rather than braces or semicolons.
 - **Developer Time over CPU Time:** Computers are cheap, but developer time is expensive. Python focuses on rapid prototyping and clear syntax.
 - **Explicit over Implicit:** Code should not make magic assumptions.
 - **One Clear Way:** There should be one—and preferably only one—obvious way to solve a problem.
-

Python's Capabilities & Applications

Python is a **general-purpose, high-level, interpreted, dynamically typed** programming language. Its versatility enables it to power systems across diverse domains:

Key Core Strengths

1. **Multi-Paradigm Support:** You can write code using **Procedural, Object-Oriented (OOP)**, or **Functional** programming styles.
2. **Dynamically Typed:** Variable types are determined at runtime, allowing quick, flexible modifications.
3. **Batteries Included:** Python comes with a massive standard library for system tasks, mathematics, text parsing, file handling, and network requests.
4. **C/C++ Extensibility:** Python easily interfaces with compiled lower-level languages. This is crucial because high-performance scientific libraries (like NumPy, TensorFlow, and PyTorch) write their heavy mathematical logic in C/C++ for speed, but expose simple Python interfaces for ease of use.

Major Application Domains

- **Artificial Intelligence & Machine Learning:** Python is the undisputed industry standard for AI. Framework libraries like PyTorch, TensorFlow, Scikit-learn, and Keras are built for Python.
- **Data Science & Scientific Computing:** Powered by NumPy, Pandas, SciPy, and Matplotlib.
- **Web Development:** Web frameworks like **Django** (batteries-included) and **Flask** (micro-framework) allow for rapid development of web servers and APIs.
- **Automation & Scripting:** Often used by system administrators to automate repetitive tasks, parse files, and manage cloud infrastructures.
- **Web Scraping & APIs:** Libraries like BeautifulSoup, Requests, and Scrapy allow developers to harvest large-scale data off the internet.

Section 2: Environment Setup & First Program

Installing Python & Setting Up Your Workspace

To begin programming in Python, you need to install the Python interpreter and choose a development environment that suits your workflow.

Installing Python

1. Windows:

- Download the installer from the official website python.org/downloads.
- **IMPORTANT:** During installation, check the box that says "**Add Python to PATH**". If you skip this, your command line will not recognize the `python` command.

1. macOS:

- macOS usually comes with a system version of Python 2.x or 3.x. It is recommended to install the latest Python version using the official installer or via [Homebrew](#):

```
brew install python
```

1. Linux (Ubuntu/Debian):

- Install python via the package manager:

```
sudo apt update
sudo apt install python3 python3-pip
```

To verify your installation, open your Terminal or Command Prompt and type:

```
python3 --version    # Or 'python --version' on Windows
```

Introducing Python Development Environments (IDEs)

An Integrated Development Environment (IDE) or text editor is where you write and run your Python code. Here are the four most common choices:

1. IDLE (Integrated Development and Learning Environment)

- **What it is:** Python's built-in, default editor that comes bundled with standard installer packages.
- **Key Features:**
 - Features a simple Interactive Shell (Read-Eval-Print Loop - REPL) where you can type code and see results instantly.
 - Offers basic syntax highlighting and a simple text editor.
- **Best for:** Beginners writing their first scripts or trying out syntax snippets without installing third-party editors.

2. Visual Studio Code (VS Code)

- **What it is:** A free, open-source, lightweight code editor developed by Microsoft.
- **Key Features:**
 - Extensively customizable using extensions (install the **Python** and **Pylance** extensions).
 - Built-in terminal, source control (Git) integration, and highly flexible debugger.
 - Auto-formatting (via `black` or `ruff`) and syntax checking (linting) as you type.
- **Best for:** General-purpose developers, web developers, and system automation engineers who want a fast, extensible editor.

3. PyCharm

- **What it is:** A dedicated Python IDE developed by JetBrains. It comes in a free "Community Edition" and a paid "Professional Edition".
- **Key Features:**
 - Deep code intelligence: advanced autocomplete, automated code refactoring (renaming variables/methods across files), and quick-fix suggestions.
 - Built-in database tools, virtual environment manager, and Django/Flask support (in Professional).

- **Best for:** Large-scale commercial Python projects and developers who want a fully configured, out-of-the-box professional workspace.

4. Jupyter Notebook / JupyterLab

- **What it is:** An open-source web application that allows you to create documents containing live code, equations, visualizations, and narrative text.
 - **Key Features:**
 - Code is split into executable "cells" rather than run as a whole script.
 - Remembers variable states in memory between executions, allowing you to run cells out of order.
 - Displays graphs, tables, and HTML directly below the code cells.
 - **Best for:** Data Scientists, Machine Learning Engineers, and researchers who perform iterative data explorations and visualization.
-

Python Basic Syntax Guidelines

Before writing programs, you must familiarize yourself with Python's grammar rules. Python syntax is designed to be highly readable, which introduces a few unique rules:

1. Indentation is Mandatory

Unlike C, Java, or C++, which use curly braces `{ }` to define code blocks, Python uses **indentation** (whitespace at the beginning of a line).

- In Python, all statements inside a block (like a loop, function, or conditional) must be indented by the same number of spaces.
- The standard convention is **4 spaces** per indentation level. Do not mix tabs and spaces, as it leads to compilation errors.

2. Line Termination

Python statements are terminated by a **newline** (pressing Enter). Semicolons `;` at the end of a line are **not required** and are generally discouraged.

- If you have a very long statement that you want to split across multiple lines, you can use the backslash line continuation character `\`:

```
total_sum = 1 + 2 + 3 + \  
            4 + 5 + 6
```

3. Case Sensitivity

Python is strictly **case-sensitive**. This means variables named `age`, `Age`, and `AGE` are treated as three completely different, independent variables.

4. Comments

Comments are annotations written in the code to explain what it does. The Python interpreter completely ignores comments during execution.

- **Single-line Comments:** Start with a hash symbol `#`.

```
# This is a single-line comment  
x = 10 # This is an inline comment
```

- **Multi-line Comments / Docstrings:** Written using triple quotes `'''` or `"""`.

```
"""  
This is a multi-line comment  
or docstring, which is often used  
to document functions and classes.  
"""
```

The "Hello, World!" Program

The traditional entry point into learning any programming language is printing `"Hello, World!"` to the screen.

Code Implementation

Create a text file named `hello_world.py` and write the following single line:

```
print("Hello, World!")
```

Running the Program

You can run this program in two ways:

Option A: Running as a Script

Open your Terminal or Command Prompt, navigate to the directory where you saved `hello_world.py`, and run:

```
python3 hello_world.py
```

Output:

```
Hello, World!
```

Option B: Running in the Interactive REPL Shell

Open your Terminal, type `python3` (or `python` on Windows) to launch the interactive shell, and type the statement directly:

```
>>> print("Hello, World!")  
Hello, World!
```

To exit the interactive shell, type `exit()` and press Enter.

Anatomy of the Code

- `print()` : This is a built-in Python function that outputs text to the console.
 - `"Hello, World!"` : This is a literal string (a sequence of characters). It must be enclosed in double quotes `"..."` or single quotes `'...'` so Python knows it is text and not variable names.
-

Section 3: Variables, Operators & Data Types

Data Types in Python: Scalar vs. Collection Types

Data types determine what kind of value a variable can store and what operations can be performed on it. In Python, data types are broadly divided into two categories: **Scalar (Primitive) Types** and **Collection (Compound) Types**.

A. Scalar Data Types (Single-Value Types)

Scalar data types represent a single value. They are the most basic building blocks in Python.

Data Type	Keyword	Description	Example
Integer	<code>int</code>	Whole numbers, positive or negative, of arbitrary length.	<code>x = -45</code>
Complex	<code>complex</code>	Numbers containing a real and an imaginary part (written with a <code>j</code>).	<code>val = 2 + 3j</code>
Boolean	<code>bool</code>	Represents logical states. Can only be <code>True</code> or <code>False</code> .	<code>is_valid = True</code>
None Type	<code>NoneType</code>	A special constant (<code>None</code>) representing the absence of a value.	<code>data = None</code>

Code Examples for Scalar Types:

```
# Numeric checks
a = 10
b = 3.5
c = 1 + 2j

print(type(a)) #
print(type(b)) #
print(type(c)) #

# Boolean evaluations
is_greater = 10 > 5 # Evaluates to True
print(is_greater)   # True
print(type(is_greater)) #
```

B. Collection Data Types (Multi-Value / Compound Types)

Collection data types store multiple items inside a single variable reference. Python has four primary built-in collection types.

1. Lists (`list`)

- **Description:** Ordered, mutable (changeable) sequences of items. Allows duplicate elements.
- **Syntax:** Square brackets `[...]`
- **Example:** `fruits = ["apple", "banana", 10, True]`

2. Tuples (`tuple`)

- **Description:** Ordered, **immutable** (cannot be modified after creation) sequences of items. Allows duplicate elements.
- **Syntax:** Parentheses `(...)`
- **Example:** `coordinates = (12.97, 77.59)`

3. Dictionaries (`dict`)

- **Description:** Unordered, mutable mappings of key-value pairs. Keys must be unique and immutable.
- **Syntax:** Curly braces with colons `{key: value}`
- **Example:** `student = {"name": "Arham", "age": 24}`

4. Sets (`set`)

- **Description:** Unordered, mutable collections of **unique** elements. Does not allow duplicates.
- **Syntax:** Curly braces `{...}`
- **Example:** `unique_ids = {101, 102, 103, 101} # Automatically filters duplicate
101`

Code Examples for Collection Types:

```
# List vs. Tuple mutability demo
my_list = [1, 2, 3]
my_list[0] = 99 # Valid! my_list becomes [99, 2, 3]

my_tuple = (1, 2, 3)
# my_tuple[0] = 99 # TypeError: 'tuple' object does not support item assignment

# Dictionary lookup
phone_book = {"Police": 100, "Ambulance": 102}
print(phone_book["Police"]) # 100
```

Creating & Using Variables in Python

In Python, a **variable** is a named reference (or label) pointing to an object stored in the computer's memory.

1. Variables are References

When you write `x = 10`, Python does the following:

1. Creates an integer object in memory containing the value `10`.
2. Binds the name `x` to point to that object.
3. If you later reassign `x = "hello"`, Python creates a string object `"hello"`, redirects `x` to point to it, and the old integer `10` is eventually cleaned up by Python's **garbage collector** if nothing else points to it.

```
x = 5
y = x # y now points to the same object as x
print(id(x) == id(y)) # True (they share the same memory location)
```

2. Variable Naming Rules

When naming variables, you must follow these rules:

- Variable names must start with a **letter** or an **underscore** (`_`). They cannot start with a digit.
- They can contain letters, numbers, and underscores (`a-z`, `A-Z`, `0-9`, `_`).
- They cannot contain spaces, punctuation marks, or mathematical symbols.
- They cannot be one of Python's **reserved keywords** (e.g., `if`, `else`, `for`, `while`, `def`, `class`, `import`, `return`, `True`, `False`, `None`).
- Follow standard Python styling conventions (**PEP 8**): Use `snake_case` for variable and function names (e.g., `user_age`, `total_price`).

3. Multiple Assignments

Python allows you to bind multiple variables in a single line:

```
# Bind multiple variables to the same value
x = y = z = 100

# Parallel assignment (unpacking)
name, age, is_student = "Alice", 21, True
```

Operators in Python

Operators are special symbols used to perform computations on variables and values.

A. Arithmetic Operators

Used to perform standard mathematical operations:

Operator	Name	Description	Example
<code>+</code>	Addition	Adds two values.	<code>5 + 3</code> $\rightarrow$ <code>8</code>
<code>*</code>	Multiplication	Multiplies two values.	<code>5 * 3</code> $\rightarrow$ <code>15</code>
<code>//</code>	Floor Division	Divides and discards the decimal fraction (truncates down).	<code>5 // 2</code> $\rightarrow$ <code>2</code>
<code>%</code>	Modulo	Returns the division remainder.	<code>5 % 2</code> $\rightarrow$ <code>1</code>
<code>**</code>	Exponentiation	Raises base to the power of exponent.	<code>2 ** 3</code> $\rightarrow$ <code>8</code>

The Difference between `/` and `//`

```
print(10 / 3)    # 3.3333333333333335 (float division)
print(10 // 3)   # 3 (truncates the decimal part, returns int)
print(-10 // 3)  # -4 (rounds down towards negative infinity)
```

B. Comparison (Relational) Operators

Used to compare two values. They always return a Boolean: `True` or `False`.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>True</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>True</code>
<code><</code>	Less than	<code>3 < 5</code>	<code>True</code>
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	Less than or equal to	<code>3 <= 5</code>	<code>True</code>

C. Logical Operators

Used to combine conditional statements:

- **and** : Returns `True` if **both** statements are true (e.g., `5 > 3 and 10 > 2` is `True`).
- **or** : Returns `True` if **at least one** statement is true (e.g., `5 > 10 or 10 > 2` is `True`).
- **not** : Reverses the logical state (e.g., `not (5 > 3)` is `False`).

Short-circuit Evaluation:

Logical operators in Python use short-circuiting:

- In `A and B` , if `A` is `False` , Python does not evaluate `B` because the overall result is guaranteed to be `False` .
- In `A or B` , if `A` is `True` , Python does not evaluate `B` because the overall result is guaranteed to be `True` .

D. Assignment Operators

Used to assign values to variables, often combined with arithmetic operations (shorthand operators):

```
x = 10    # Standard assignment
x += 5    # Equivalent to x = x + 5 (x becomes 15)
x -= 2    # Equivalent to x = x - 2 (x becomes 13)
x *= 2    # Equivalent to x = x * 2 (x becomes 26)
x /= 2    # Equivalent to x = x / 2 (x becomes 13.0)
```

Basic Input/Output (I/O) Operations in Python

A program interacts with users by taking data in (Input) and showing results back (Output). In Python, this is primarily managed by the built-in functions `print()` and `input()`.

A. Output Operations: `print()`

The `print()` function writes data to the standard output (usually the terminal screen).

1. Printing Multiple Values

You can print multiple variables or values in a single call by separating them with commas. By default, Python separates them with a space.

```
name = "Alice"
age = 21
print("Name:", name, "Age:", age) # Output: Name: Alice Age: 21
```

2. Custom Separators (`sep=`)

You can override the default space separator between items using the `sep` parameter.


```
print("24", "08", "2026", sep="-") # Output: 24-08-2026
```

3. Custom Line Endings (`end=`)

By default, the `print()` function appends a newline character (`\n`) at the end of the print statement. You can change this using the `end` parameter.

```
print("Hello", end=" ")  
print("World") # Output: Hello World (on the same line)
```

4. Formatting Output Strings

There are three main ways to inject variables into output strings:

- **Method 1: String Concatenation** (Legacy / Tedious)
- Requires manually casting variables to strings.

```
print("Age: " + str(age))
```

- **Method 2: `.format()` method** (Legacy)

```
print("Name: {}, Age: {}".format(name, age))
```

- **Method 3: F-strings (Formatted String Literals)** (Modern / Recommended)
- Prefix the string with an `f` or `F`, and write variables directly inside curly braces `{}`. It is faster, cleaner, and allows evaluating expressions.

```
print(f"Name: {name}, Age: {age}")  
print(f"Double of age is: {age * 2}")
```

B. Input Operations: `input()`

The `input()` function pauses program execution and waits for the user to type text on the keyboard and press Enter.

IMPORTANT:

The `input()` function ALWAYS returns the user's input as a string (`str`). If you need numeric values, you must convert (cast) them explicitly using functions like `int()` or `float()` .

Handling String Input:

```
user_name = input("Enter your username: ")
print(f"Hello, {user_name}!")
```

Handling Numeric Input (Casting):

```
# Convert to integer
qty = int(input("Enter quantity: "))
price = float(input("Enter unit price: "))

total_cost = qty * price
print(f"Total Cost: ${total_cost:.2f}") # ':.2f' limits output to 2 decimal pla
```

What happens if you forget to cast?

If you try to perform arithmetic operations directly on string inputs, Python will perform string concatenation (for `+`) or raise a `TypeError` (for other operators like `-`, `*`, `/`):

```
x = input("Enter first number: ") # User enters: 5
y = input("Enter second number: ") # User enters: 3
print(x + y) # Output: "53" (concatenates strings rather than adding numbers!)
```

Section 4: Control Flow & Looping

Flow of Control: Conditional Statements

By default, Python executes statements sequentially from top to bottom. Conditional statements allow you to diverge this execution path by running certain blocks of code only if specific conditions are met.

In Python, conditional flow is controlled by the keywords `if`, `elif` (short for else-if), and `else`.

A. Core Rules of Conditional Statements

1. **The Colon (:):** Every conditional statement header (`if`, `elif`, `else`) must end with a colon.
 2. **Indentation:** The code block to be executed if a condition is met must be indented (standard 4 spaces). The end of the block is marked by returning to the outer indentation level.
 3. **Condition Expression:** Python evaluates the expression after `if` or `elif` as a Boolean (`True` or `False`).
-

B. The `if` Statement

Runs a block of code only if the condition evaluates to `True`.

```
temperature = 35

if temperature > 30:
    print("It is a hot day!") # Runs only if temperature > 30
print("Drive safely.")      # Always runs (outside the if block)
```

C. The `if-else` Statement

Provides an alternative execution block when the condition is `False` .

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are too young to vote.")
```

D. The `if-elif-else` Chain

Used to check multiple mutually-exclusive conditions in sequence. Python checks the conditions from top to bottom and executes **only the first block** whose condition is `True` . All subsequent blocks are skipped.

```
score = int(input("Enter your exam score (0-100): "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

E. Nested `if-else` Statements

You can place conditional structures inside other conditional blocks to resolve complex, dependent criteria.

```
has_license = True
age = 20

if age >= 18:
    print("Age verified.")
    if has_license:
        print("You are allowed to rent a car.")
    else:
        print("You need a valid license to rent a car.")
else:
    print("You are too young to drive.")
```

F. Truthy and Falsy Values in Python

In Python, values of non-Boolean data types can be implicitly evaluated in conditional tests.

- **Falsy Values:** Evaluate to `False` in conditions:
 - `None`
 - `False`
 - `0` (integer zero)
 - `0.0` (float zero)
 - `""` (empty string)
 - `[]` (empty list), `()` (empty tuple), `{}` (empty dictionary/set)
- **Truthy Values:** Any value not on the Falsy list evaluates to `True`.

```
# checking for empty lists or strings pythonically
name = input("Enter name: ")
if name: # Evaluates to True if name is not an empty string
    print(f"Hi {name}!")
else:
    print("You didn't enter a name!")
```

Looping Structures in Python

Loops are used to repeatedly execute a block of code. Python supports two main loop types:

`while` loops and `for` loops.

A. The `while` Loop

A `while` loop repeatedly executes a block of code as long as a specified condition remains `True`.

```
count = 1
while count <= 5:
    print(f"Count is: {count}")
    count += 1 # IMPORTANT: Update condition variable to prevent an infinite lo
```

Infinite Loops

If the loop condition never evaluates to `False`, the loop runs forever, freezing your program.

```
# Warning: Infinite Loop!
# count = 1
# while count <= 5:
#     print(count) # Missing 'count += 1', count stays 1 forever
```

Press `Ctrl + C` in your terminal to force-terminate an infinite loop.

B. The `for` Loop

A `for` loop is used to iterate over a sequence (such as a string, list, tuple, set, dictionary, or a numeric range).

1. Iterating over a string

```
word = "Python"
for letter in word:
    print(letter)  # Prints each character on a new line
```

2. The `range()` Function

To run a loop a specific number of times, combine the `for` loop with the built-in `range()` function.

- `range(stop)` : Runs from `0` up to `stop - 1` (stop is exclusive).

```
for i in range(3):
    print(i)  # Prints: 0, 1, 2
```

- `range(start, stop)` : Runs from `start` up to `stop - 1`.

```
for i in range(2, 6):
    print(i)  # Prints: 2, 3, 4, 5
```

- `range(start, stop, step)` : Runs from `start` to `stop - 1`, incrementing by `step` each time.

```
for i in range(1, 10, 2):
    print(i)  # Prints odd numbers: 1, 3, 5, 7, 9
```

C. Loop Control Statements: `break` and `continue`

You can alter the standard execution of a loop using `break` and `continue`.

- `break` : Terminates the loop immediately.

```
# Search for value 7
for num in range(1, 10):
    if num == 7:
        print("Found 7! Stopping search.")
        break
    print(f"Checking {num}...")
```

- **continue** : Skips the rest of the current iteration and jumps directly to the next cycle.

```
# Print numbers 1-5 except 3
for num in range(1, 6):
    if num == 3:
        continue # Skip the print line below for 3
    print(num)
```

D. The Unique **else** Clause in Loops

In Python, loops can have an optional **else** block.

- **Rule:** The code in the **else** block runs **only if the loop finishes successfully without encountering a **break** statement.**
- **Use Case:** Ideal for search operations to run "not found" fallback code.

```
# Search for even numbers in a list
numbers = [1, 3, 5, 7]

for num in numbers:
    if num % 2 == 0:
        print(f"Found even number: {num}")
        break
else:
    # Runs only if the loop finished without hitting 'break'
    print("No even numbers found in the list.")
```


E. Nested Loops

A loop written inside the body of another loop.

```
# Print coordinates grid
for x in range(1, 3):
    for y in range(1, 4):
        print(f"({x}, {y})", end=" ")
    print() # Line break after inner loop completes
(1, 1) (1, 2) (1, 3)
(2, 1) (2, 2) (2, 3)
```

Loop Control Structures (`break` , `continue` , and `pass`)

While writing loops, you often need to alter the flow of iteration based on external conditions. Python provides three core control keywords: `break` , `continue` , and `pass` .

A. The `break` Statement

The `break` statement immediately terminates the current loop execution. Program control jumps directly to the first statement outside the loop block.

Flow Diagram Analogy:

```
[ Start Loop ] -> [ Condition True? ] -> [ Code Block ] -> [ break encountered? ]
                                     |                               | No
                                     v                               v
                               [ Exit Loop ]                     [ Next Iteration ]
```

Practical Example:

A simple console menu that loops indefinitely until the user chooses to exit:

```
while True:
    choice = input("Enter 'q' to quit, any other key to continue: ")
    if choice.lower() == 'q':
        print("Exiting loop...")
        break # Immediately exits the while loop
    print("Running process...")
print("Program continues here.")
```

B. The `continue` Statement

The `continue` statement skips the remaining code statements inside the loop body for the **current iteration only**, and immediately jumps to the next cycle of the loop (re-evaluates the loop condition).

Practical Example:

Printing only odd numbers from a list:

```
numbers = [1, 2, 3, 4, 5, 6]

for num in numbers:
    if num % 2 == 0:
        continue # Skips print(num) for even numbers and goes to next loop iter
    print(f"Odd number: {num}")
```

Output:

```
Odd number: 1
Odd number: 3
Odd number: 5
```

C. The `pass` Statement

The `pass` statement is a **null operation**—nothing happens when it executes.

- **Why do we need it?:** Python relies on indentation blocks. If you write a loop, function, or class block with no body, Python will crash with an `IndentationError`. The `pass` statement serves as a syntactic placeholder.

Practical Example:

```
# 1. Placeholder in a loop to write logic later
for i in range(100):
    pass # Keeps the loop valid without raising IndentationError

# 2. Placeholder in a conditional branch
if score > 90:
    pass # TODO: Add bonus marks logic
else:
    print("No change.")

# 3. Placeholder for empty function shells
def fetch_api_data():
    pass # Skeleton definition
```

D. Side-by-Side Comparison

Feature	<code>break</code>	<code>continue</code>	<code>pass</code>
Action	Terminates the loop structure immediately.	Skips current loop cycle and starts the next.	Does nothing; acts as a syntactic placeholder.
Loop Exit?	Yes	No	No
Line Skip?	Yes (all remaining lines and iterations)	Yes (remaining lines of current cycle only)	No (all lines continue executing normally)
Syntax Role	Behavioral control.	Behavioral control.	Syntactic placeholder only.

Practice Assignments

Objective

Apply variables, operators, loops, and conditional statements to solve introductory algorithmic challenges.

Assignment 1: Leap Year Checker

Write a program that takes a year as input from the user and checks whether it is a leap year or not.

- **Leap Year Criteria:** A year is a leap year if it is divisible by 4, except for century years (ending in 00), which must also be divisible by 400.
- **Sample Input:** `2024`
- **Sample Output:** `2024 is a Leap Year.`

Assignment 2: Fibonacci Sequence Generator

Write a Python script to print the first N terms of the Fibonacci sequence, where N is provided by the user.

- **Fibonacci sequence:** 0, 1, 1, 2, 3, 5, 8, 13, 21, $\dots$
 - **Sample Input:** `N = 6`
 - **Sample Output:** `0, 1, 1, 2, 3, 5`
-

Assignment 3: Prime Number Checker

Write a program that checks whether a positive integer entered by the user is a prime number.

- **Logic:** A prime number is a number greater than 1 that has no positive divisors other than 1 and itself.
 - **Sample Input:** `17`
 - **Sample Output:** `17 is a prime number.`
-

Assignment 4: Odd or Even Checker

Write a program that prompts the user for an integer and prints whether it is even or odd.

- **Sample Input:** `7`
 - **Sample Output:** `7 is an Odd number.`
-

Assignment 5: Basic Operator Calculator

Create a program that takes two numbers and a math operator (+, -, *, /) from the user, performs the corresponding calculation, and prints the result.

- **Sample Input:** num1=15 , num2=3 , operator='/'
 - **Sample Output:** Result: 5.0
-

Assignment 6: Sum of N Natural Numbers

Write a script that accepts a positive integer \$N\$ from the user and calculates the sum of all natural numbers up to \$N\$.

- **Formula:** $\sum_{i=1}^N i = \frac{N(N+1)}{2}$
 - **Sample Input:** N = 10
 - **Sample Output:** Sum: 55
-

Assignment 7: Multiplication Table Generator

Write a program that takes an integer from the user and prints its multiplication table from 1 to 10.

- **Sample Input:** 5
- **Sample Output:**

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Assignment 8: Score to Grade Converter

Write a script that takes a numeric test score from the user (0 to 100) and displays a corresponding letter grade based on the following scale:

- 90-100: A
 - 80-89: B
 - 70-79: C
 - 60-69: D
 - Below 60: F
-

Chapter 2: Sequence Types — Strings & Tuples

Welcome to Chapter 2! Today we focus on Python's primary immutable sequence structures: **Strings** (sequences of characters) and **Tuples** (sequences of arbitrary objects). We will explore how they store values, how to access and slice them, and how their immutable nature affects memory management.

Section 1: Strings (Immutable Text)

1.1 What is a String?

In Python, a **String** is an ordered sequence of Unicode characters representing textual data.

- Strings are **immutable**. Once created, their contents in memory cannot be altered. Any operation that appears to modify a string actually creates a brand-new string object in memory.
-

1.2 Different Ways to Create Strings in Python

Python provides multiple ways to declare and initialize strings, offering flexibility depending on the content of the text:

A. Single Quotes (`'...'`)

The most basic way to define a string.


```
message = 'Hello, Python!'
```

B. Double Quotes (`"..."`)

Works exactly like single quotes. However, double quotes are useful when your string contains a single quote/apostrophe, as it avoids the need to write escape characters (`\\`).

```
# No escaping needed for the apostrophe
quote = "Python is Guido's creation."

# If single quotes were used, escaping is required:
# quote = 'Python is Guido\\'s creation.'
```

C. Triple Quotes (`'''...'''` or `"""..."""`)

Triple quotes are used for defining **multiline strings** or text containing both single and double quotes.

```
multiline_text = """This is a string
that spans across multiple
different lines in Python."""
```

*Note: Triple quotes are also used for writing **docstrings** (documentation comments) at the beginning of functions, classes, and modules.*

D. Using the `str()` Constructor (Type Casting)

You can convert other data types (integers, floats, lists, booleans) into their string representations using the built-in `str()` function.

```
age = 25
age_string = str(age)    # Converted to "25"
pi_string = str(3.14)    # Converted to "3.14"
```

1.3 Understanding the `str` Class

In Python, every string we create is an instance (an object) of the built-in class `str`.

```
s = "acts"
print(type(s))  # Output:
```

Core Characteristics of the `str` Class:

1. Immutability in Memory

When you perform operations on a string object, Python leaves the original string completely untouched in memory. Instead, it computes and registers a new string object elsewhere in memory.

```
original = "Python"
print(id(original))  # E.g., 4381982704

# Modifying the string
modified = original + " 3"
print(id(modified))  # E.g., 4381983584 (a completely new address!)
print(original)      # Still prints "Python"
```

2. The `__str__()` Method

When you invoke the `str(object)` constructor, Python internally looks up and executes that object's `__str__()` special (dunder) method. This method defines how the object should represent itself as a readable text string.

- For example, printing a list object internally uses the list's `__str__()` method to format it inside brackets `[...]`.

3. Inspecting the Class

You can see all methods and attributes exposed by the `str` class in your console using the `dir()` function:

```
print(dir(str)) # Displays all string helper methods
```

And to see full documentation on how to use any method:

```
help(str.split) # Displays usage info for split()
```

1.4 Accessing Characters in Strings (Indexing)

Since strings are ordered sequences, every character in a string occupies a specific numerical position called an **index**. Python allows you to retrieve individual characters using square brackets `[]` enclosing the index number.

A. Positive Indexing (Zero-Based)

Python uses **zero-based indexing**, meaning the first character of the string starts at index `0`, the second at index `1`, and the last character is at index `len(string) - 1`.

B. Negative Indexing (Backward Counting)

Python also supports **negative indexing** to access elements from right to left.

- The last character of the string is at index `-1`.
- The second-to-last character is at index `-2`.
- The first character is at index `-len(string)`.

C. Visual Representation of String Indexing

Let's look at how the string `"PYTHON"` is indexed:

Character:	P	Y	T	H	O	N
	---	---	---	---	---	---
Positive Idx:	0	1	2	3	4	5
Negative Idx:	-6	-5	-4	-3	-2	-1

Code Examples:

```
text = "PYTHON"

# Positive Indexing
print(text[0]) # Output: 'P' (First character)
print(text[2]) # Output: 'T' (Third character)
print(text[5]) # Output: 'N' (Last character)

# Negative Indexing
print(text[-1]) # Output: 'N' (Last character)
print(text[-2]) # Output: 'O' (Second-to-last character)
print(text[-6]) # Output: 'P' (First character)
```

D. Pitfall: The `IndexError`

If you attempt to access an index that is outside the range of the string, Python will raise an `IndexError`.

```
text = "PYTHON" # len(text) is 6
# print(text[6]) # IndexError: string index out of range
# print(text[-7]) # IndexError: string index out of range
```

Always ensure that your target index is between `-len(string)` and `len(string) - 1`.

1.5 Basic String Operations (Concatenation & Repetition)

Python provides simple operators (`+` and `*`) to combine and multiply text strings.

A. String Concatenation (`+`)

Concatenation means gluing two or more strings together end-to-end. You do this in Python using the plus (`+`) operator.

```
first_name = "Guido"
last_name = "van Rossum"

# Concatenate with a space in between
full_name = first_name + " " + last_name
print(full_name)  # Output: Guido van Rossum
```

Implicit Concatenation

If you place two string **literals** adjacent to each other, Python automatically concatenates them even without the `+` operator.

```
message = "Hello " "World"
print(message)  # Output: Hello World
```

Note: This only works with literal strings, not with variables.

Pitfall: TypeError on Non-String Concatenation

You cannot concatenate a string with a non-string data type (like an integer or a float) directly. You must cast the non-string to a string first.

```
age = 35
# print("Age: " + age) # TypeError: can only concatenate str (not "int") to str

# Fix by casting
print("Age: " + str(age)) # Output: Age: 35
```

B. String Repetition (*)

You can repeat a string a specified number of times using the multiplication (*) operator. The multiplier **must be an integer**.

```
prefix = "la "
chorus = prefix * 3
print(chorus) # Output: la la la

# Creating a divider line
divider = "-" * 30
print(divider) # Output: -----
```

Code Examples:

```
# Combining Concatenation and Repetition
laugh = "Ha"
fun = laugh * 3 + "!"
print(fun) # Output: HaHaHa!
```

Note: Multiplying a string by 0 or a negative integer returns an empty string "" .

1.6 String Formatting

String formatting allows you to insert dynamic variables or expressions into static text strings. In Python, there are three main methods of formatting:

A. C-Style `%` Formatting (Legacy)

The oldest method, borrowing syntax from the C language's `printf` function. It uses format specifiers (like `%s` for string, `%d` for integer, `%f` for float) as placeholders.

```
name = "Rajan"
age = 24
result = "Name: %s, Age: %d" % (name, age)
print(result)  # Output: Name: Rajan, Age: 24
```

Note: This method is legacy and generally discouraged in modern Python because it gets hard to read when handling many variables.

B. The `str.format()` Method (Python 2.6+)

Uses curly braces `{}` as placeholders. You supply variables inside the `.format()` call.

```
name = "Rajan"
age = 24

# Positional formatting
print("Name: {}, Age: {}".format(name, age))

# Positional indexing
print("Age: {1}, Name: {0}".format(name, age))  # Swaps order

# Named keyword placeholders
print("Name: {n}, Age: {a}".format(n="Kishori", a=22))
```

C. F-Strings (Formatted String Literals - Python 3.6+)

The modern, fastest, and most readable string formatting technique. You prefix the string literal with an `f` or `F` and write variable names or expressions directly inside the `{}` braces.

```
name = "Esha"
age = 23
print(f"Name: {name}, Age: {age}") # Output: Name: Esha, Age: 23
```

D. Advanced F-String Features & "Hacks"

The `{}` syntax inside f-strings is incredibly powerful and offers several built-in format specifiers and formatting hacks:

1. Self-Documenting Debugging Syntax (`{variable=}`) (Python 3.8+)

If you append an equal sign `=` to a variable or expression inside `{}`, Python prints both the literal expression text and its evaluated value. This is highly useful for debugging and logging.

```
name = "Vinod Kumar"
city = "Bangalore"

# Traditional print debugging
print(f"name={name}, city={city}") # Output: name=Vinod Kumar, city=Bangalore

# Using the '=' debugging hack
print(f"{name=}, {city=}") # Output: name='Vinod Kumar', city='Bangalore'
```

2. Alignment and Padding (`:<`, `:>`, `:^`)

You can control the width, alignment, and fill character of the text output using format specifiers following a colon `:`:

- `:<width`: Left-align within a fixed width (default for strings).
- `:>width`: Right-align within a fixed width (default for numbers).
- `:^width`: Center-align within a fixed width.

- Provide a character before the alignment symbol to act as a custom fill character.

```
city = "Bangalore"

print(f"[{city:<15}]") # Left-aligned: [Bangalore    ]
print(f"[{city:>15}]") # Right-aligned: [      Bangalore]
print(f"[{city:^15}]") # Center-aligned: [  Bangalore  ]

# Using a custom padding fill character (e.g. '*')
print(f"{city:*^17}") # Center star-padded: *****Bangalore*****
```

3. Number Conversions: Binary, Octal, Hex, and Percents

You can convert integers or floats inline into other representations using special formats:

- `:b` : Binary representation.
- `:o` : Octal representation.
- `:x` : Hexadecimal representation.
- `:%` : Percentage representation (multiplies by 100 and formats as %).

```
num = 42
print(f"Binary of {num}: {num:b}") # Output: 101010
print(f"Hex of {num}: {num:x}")    # Output: 2a

ratio = 0.275
print(f"Percentage: {ratio:.1%}") # Output: 27.5%
```

4. Inline Datetime Formatting

Instead of importing datetime and calling `.strftime()` to get pretty strings, you can format date/time objects directly inside f-strings:

```
import datetime
today = datetime.date(2026, 8, 26)
print(f"Date: {today:%B %d, %Y}") # Output: Date: August 26, 2026
```

5. Dictionary Key Lookup and Quote Nesting (Python 3.12+ updates)

How Python handles quotes inside f-string expressions depends on the Python version you are running:

- **Python 3.12 and newer (PEP 701):** Quote reuse is **fully permitted**. You can use the same quotes inside the `{}` placeholders as the outer string without causing errors.

```
profile = {"name": "Vinod", "city": "Bangalore"}  
# Valid in Python 3.12+  
print(f"City: {profile['city']}") # Output: City: Bangalore
```

- **Python 3.11 and older:** Reusing the same quotes causes a `SyntaxError` because the interpreter misinterprets the inner quotes as the closing bound of the f-string. You must alternate single and double quotes.

```
# Required for Python 3.11 and older (and good for backward compatibility)  
print(f"City: {profile['city']}") # Output: City: Bangalore
```

1.7 Built-in String Methods

The `str` class provides a set of built-in methods to perform manipulations on strings. Here are some of the most commonly used methods, using data related to the name **Vinod Kumar Kayartaya**, email **vinod@vinod.co**, and city **Bangalore**:

1. Case Conversions: `.upper()`, `.lower()`, `.title()`

- `.upper()`: Converts all characters to uppercase.
- `.lower()`: Converts all characters to lowercase.
- `.title()`: Capitalizes the first letter of every word.

```
name = "Vinod Kumar Kayartaya"

print(name.upper()) # Output: VINOD KUMAR KAYARTAYA
print(name.lower()) # Output: vinod kumar kayartaya
print("vinod kumar".title()) # Output: Vinod Kumar
```

2. Stripping Whitespace: `.strip()`, `.lstrip()`, `.rstrip()`

Removes leading and trailing spaces, tabs, or newlines.

- `.strip()` : Removes whitespace from both ends.
- `.lstrip()` : Removes from left side only.
- `.rstrip()` : Removes from right side only.

```
email = "   vinod@vinod.co   "

print(f"[{email}]") # Output: [   vinod@vinod.co   ]
print(f"[{email.strip()}]") # Output: [vinod@vinod.co]
```

3. Splitting and Joining: `.split()`, `.join()`

- `.split(separator)` : Splits a string into a list of substrings based on the separator (defaults to spaces).
- `string.join(iterable)` : Concatenates a list of strings using the primary string as a glue separator.

```
name = "Vinod Kumar Kayartaya"
# Split the string by spaces into a list
name_parts = name.split()
print(name_parts) # Output: ['Vinod', 'Kumar', 'Kayartaya']

# Join the list parts back using a dash '-'
joined_name = "-".join(name_parts)
print(joined_name) # Output: Vinod-Kumar-Kayartaya
```

4. Search and Index: `.find()` , `.index()`

Used to search for a substring within a string.

- `.find(sub)` : Returns the lowest start index where substring is found. Returns `-1` if not found.
- `.index(sub)` : Same as `.find()` , but raises a `ValueError` if the substring is not found.

```
city = "Bangalore"

print(city.find("galore")) # Output: 3 (Index of 'g')
print(city.find("Acts"))   # Output: -1 (Not found)
# print(city.index("Acts")) # Raises ValueError: substring not found
```

5. Prefix/Suffix Checks: `.startswith()` , `.endswith()`

Returns a Boolean indicating if a string starts or ends with a target pattern.

```
email = "vinod@vinod.co"

print(email.startswith("vinod")) # Output: True
print(email.endswith(".co"))     # Output: True
print(email.endswith(".com"))    # Output: False
```

6. Substring Replacement: `.replace()`

Replaces all occurrences of a target substring with a new substring.

```
city = "Bangalore"

# Replace 'B' with 'M' (Wordplay: Bangalore -> Mangalore)
new_city = city.replace("B", "M")
print(new_city)  # Output: Mangalore
```

Section 2: Tuples (Immutable Sequences)

A **Tuple** is a built-in Python sequence type that is **ordered** and **immutable**. Tuples can store multiple items of different data types (heterogeneous data) inside a single variable.

2.1 Defining and Accessing Tuples

1. Defining Tuples

Tuples are written as a list of values separated by commas, usually enclosed in parentheses `()`. Note that in Python, parentheses are technically optional when defining tuples, but they are highly recommended for code readability.

```
# A tuple containing integers
numbers = (1, 2, 3)

# Defined without parentheses (Tuple Packing)
shorthand_tuple = "Vinod", "Bangalore", 560001
print(type(shorthand_tuple)) # Output:

# A tuple containing heterogeneous (mixed) data types
profile = ("Vinod", 25, "Bangalore", True)

# Nested tuples
nested_tuple = ((1, 2), ("a", "b"))
```

Rule: The Single-Item Tuple Comma

If you want to create a tuple that contains only one element, you **must include a trailing comma**. Without the comma, Python treats the parentheses as mathematical parentheses and infers the scalar type of the inner element.

```
not_a_tuple = ("Bangalore") # Python infers this as a string
print(type(not_a_tuple))    # Output:

actual_tuple = ("Bangalore",) # Trailing comma marks it as a tuple
print(type(actual_tuple))    # Output:
```

2. Accessing Elements

Like strings, tuples support zero-based positive indexing, negative indexing, and slicing using square brackets `[]`.

```
city_coords = ("Bangalore", 12.97, 77.59)

# Positive indexing
print(city_coords[0])    # Output: 'Bangalore' (First element)

# Negative indexing
print(city_coords[-1])   # Output: 77.59 (Last element)

# Slicing tuples
sub_tuple = city_coords[1:3]
print(sub_tuple)         # Output: (12.97, 77.59)
```

2.2 Operations and Immutability

1. Operations on Tuples

Since tuples are sequences, they support basic concatenation (+) and repetition (*) operations. Because tuples are immutable, these operations do not modify the original tuples; they return new ones.

```
t1 = (1, 2)
t2 = (3, 4)

# Concatenation
t3 = t1 + t2
print(t3)    # Output: (1, 2, 3, 4)

# Repetition
t4 = t1 * 3
print(t4)    # Output: (1, 2, 1, 2, 1, 2)
```

2. Understanding Immutability

Once a tuple is created in memory, its elements cannot be reassigned, added, or deleted. Attempting to do so raises a `TypeError`.

```
user_info = ("vinod@vinod.co", "Bangalore")

# Attempting to reassign an element
# user_info[1] = "Mangalore" # TypeError: 'tuple' object does not support item
```

The Exception: Mutable Objects inside an Immutable Tuple

Immutability applies only to the **references** held by the tuple, not the values inside mutable referents. If a tuple contains a mutable object (like a list), you cannot replace the list object with another object, but you **can** modify the elements inside that list!

```
# A tuple containing an integer and a mutable list
mixed_tuple = (10, [20, 30])

# This is NOT allowed (modifying the tuple reference at index 1)
# mixed_tuple[1] = [40, 50] # Raises TypeError

# This IS allowed (modifying the contents of the mutable list inside the tuple)
mixed_tuple[1][0] = 99
print(mixed_tuple) # Output: (10, [99, 30])
```

2.3 Tuple Packing and Unpacking

Tuple packing and unpacking are powerful features in Python that allow you to bundle values together and separate them into individual variables efficiently.

1. Tuple Packing

When we assign multiple values to a single variable name separated by commas, Python "packs" those values into a single tuple.

```
# Packing values
address = ("vinod@vinod.co", "Bangalore", 560001)
```


2. Tuple Unpacking

Unpacking extracts the values from a tuple and assigns them to individual variables. The number of variables on the left side of the assignment **must match** the number of elements in the tuple.

```
# Unpacking values
email, city, pin = address
print(email)  # Output: vinod@vinod.co
print(city)   # Output: Bangalore
```

Extended Unpacking with the Star (*) Operator

If the number of variables on the left does not match the number of elements in the tuple, you can collect multiple values into a list using the `*` operator.

```
numbers = (1, 2, 3, 4, 5)

# Collect all middle values into a list
first, *middle, last = numbers
print(first)    # Output: 1
print(middle)   # Output: [2, 3, 4] (List of remaining values)
print(last)     # Output: 5
```

Swapping Variables

Tuple unpacking makes swapping variable values clean and readable without requiring a temporary variable:

```
a = "Vinod"
b = "Bangalore"

# Swap values
a, b = b, a
print(a)  # Output: Bangalore
print(b)  # Output: Vinod
```

Practice Assignments

Objective

Apply string indexing, slicing, concatenation, repetition, formatting, and built-in string methods to solve problems.

Part A: Easy Complexity (3 Exercises)

Assignment 1: Sentence Analysis (Character & Word Count)

Write a Python program that prompts the user to enter a sentence. The program must count and display:

1. The total number of characters (including spaces and punctuation).
2. The total number of words.

- **Sample Input:** "Learning Python is fun!"

- **Sample Output:**

```
Total Characters: 23
Total Words: 4
```

Assignment 2: Reversed Uppercased String

Write a program that takes a string input from the user, reverses the string, converts the entire reversed string to uppercase, and prints the result.

- **Sample Input:** "Bangalore"

- **Sample Output:** "EROLAGNAB"

Assignment 3: Email Domain Extractor

Write a program that prompts the user to enter an email address string. Extract the domain name (the part after the @) and print it. If the string is not a valid email (does not contain exactly one @), print "Invalid Email".

- **Sample Input:** "vinod@vinod.co"
 - **Sample Output:** "vinod.co"
 - **Sample Input:** "vinod.co"
 - **Sample Output:** "Invalid Email"
-

Part B: Medium Complexity (5 Exercises)

Assignment 4: Vowel & Consonant Frequency

Write a program that prompts the user to enter a string and counts:

1. The individual frequency of each vowel (a, e, i, o, u), case-insensitively.
2. The total count of all consonants.

- **Sample Input:** "Vinod Kumar Kayartaya"
- **Sample Output:**

```
Vowel Frequencies:  
a: 4  
e: 0  
i: 1  
o: 1  
u: 1  
Total Consonants: 12
```

Assignment 5: Custom Title Case Formatter

Write a program that accepts a string input from the user and outputs it in Title Case (capitalizing the first letter of each word and lowercasing the remaining letters). **Do not use Python's built-in .title() method.**

- **Sample Input:** "WELCOME TO BANGALORE CITY"
- **Sample Output:** "Welcome To Bangalore City"

Assignment 6: Shift Cipher Encrypter

Write a program that prompts the user for a text string and a shift integer, and encrypts the text using a Caesar cipher. It should shift each alphabetical character in the string by the specified shift number down the alphabet. Maintain uppercase and lowercase characters, and leave spaces or punctuation marks completely unchanged.

- **Sample Input:** (User inputs string "Vinod" and shift 3)
- **Sample Output:** "Ylqrg"

Assignment 7: Manual Substring Counter

Write a program that prompts the user to enter a main text string and a substring. Count how many times the substring appears in the main string **without using Python's built-in `.count()` method**.

- **Sample Input:** (User inputs main string "banana" and substring "an")
- **Sample Output:** 2

Assignment 8: Name Anonymizer

Write a program that prompts the user to enter a full name (first name, middle name, last name) and anonymizes it. The output should print the initials of the first and middle names followed by the full last name. If the name consists of only a single word, print it as-is.

- **Sample Input:** "Vinod Kumar Kayartaya"
- **Sample Output:** "V. K. Kayartaya"
- **Sample Input:** "Bangalore"
- **Sample Output:** "Bangalore"

Part C: Difficult Complexity (2 Exercises)

Assignment 9: Longest Palindromic Substring

Write a program that prompts the user to enter a text string and finds the longest substring within it that reads the same forward and backward. If there are multiple palindromic substrings of the same maximum length, print any one of them.

- **Sample Input:** `"babad"`
- **Sample Output:** `"bab"` (or `"aba"`)
- **Sample Input:** `"cbbd"`
- **Sample Output:** `"bb"`

Assignment 10: Run-Length String Compression

Write a program that prompts the user to enter a text string and compresses it using run-length encoding (listing character counts next to each repeated character). If the compressed string is not smaller in size than the original string, print the original string.

- **Sample Input:** `"aabcccccaaa"`
 - **Sample Output:** `"a2b1c5a3"`
 - **Sample Input:** `"abcd"`
 - **Sample Output:** `"abcd"` (since `"a1b1c1d1"` is longer than `"abcd"`)
-

Part D: Challenge Complexity (2 Exercises)

Assignment 11: Group Anagrams

Write a program that starts with a list of strings defined at the top of your script (e.g., `words = ["eat", "tea", "tan", "ate", "nat", "bat"]`) and groups the anagrams (words formed by rearranging letters) together. Print the final grouped list of lists.

- **Hardcoded Input:** `words = ["eat", "tea", "tan", "ate", "nat", "bat"]`
- **Sample Output:** `[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`

Assignment 12: Date Validator & Pretty Formatter

Write a program that prompts the user to enter a date string in the format `"DD/MM/YYYY"` .

WARNING:

Do not use any built-in date/time library functions (such as the `datetime` or `time` modules) to format or validate the dates. You must parse and split the string manually, and use a custom tuple of month names for the pretty output if needed.

Your program must:

1. Verify if the date is valid. To be valid:
 - The month must be between `1` and `12` inclusive.
 - The day must be valid for that specific month (e.g., April, June, September, November have 30 days; others have 31 days).
 - For February, the day must be at most `29` in a leap year (divisible by 4, except for centuries not divisible by 400) and at most `28` in standard years.
 1. If the date is valid, use a tuple of month names (`"January", "February", ...`) to format and print the date in a long-form readable layout: `"MonthName DD, YYYY"` .
 2. If the date is invalid, print `"Invalid Date"` .
 - **Sample Input:** `"26/08/2026"`
 - **Sample Output:** `"August 26, 2026"`
 - **Sample Input:** `"29/02/2026"` (2026 is not a leap year)
 - **Sample Output:** `"Invalid Date"`
 - **Sample Input:** `"31/04/2026"` (April only has 30 days)
 - **Sample Output:** `"Invalid Date"`
-

Chapter 3: Mutable Sequences —

Working with Lists

Welcome to Chapter 3! Today, we transition from immutable sequences (Strings and Tuples) to **Lists**, which are Python's primary **mutable** sequence type. Lists are incredibly versatile: they can grow or shrink dynamically, hold heterogeneous data types, and be modified directly in memory without creating new objects.

Section 1: List Fundamentals & Accessing Elements

1.1 What is a List?

A list is an ordered, indexed collection of items. In Python, lists are:

- **Mutable:** You can add, remove, or modify elements in-place.
- **Heterogeneous:** A single list can contain elements of different data types (e.g., integers, strings, other lists, booleans).
- **Dynamic:** Python handles resizing automatically.

NOTE:

Under the Hood (CPython Implementation):

*In the standard CPython interpreter, lists are **not** implemented as linked lists. Instead, they are implemented as **variable-length dynamic arrays of object references (pointers)**.*

** **Access Speed:** This contiguous array structure allows for very fast $O(1)$ constant-time lookup/modification of any element by index.*

** **Memory Pre-allocation:** To avoid resizing the array on every `.append()`, Python overallocates capacity. As a result, appending elements has an **amortized** time complexity of $O(1)$.*

* **Insertion/Deletion Costs:** Inserting or deleting elements from the beginning or middle of the list requires shifting all subsequent elements, yielding a time complexity of $O(n)$.

Syntax:

Lists are defined by enclosing comma-separated values inside square brackets `[...]`.

```
# An empty list
empty_list = []

# List of integers
numbers = [10, 20, 30, 40]

# List of mixed data types
mixed_list = ["Alice", 42, 3.14, True, None]

# Nested list (representing a 2D grid/matrix)
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
```

1.2 Accessing Elements (Indexing & Slicing)

Like Strings and Tuples, Lists are zero-indexed and support slicing.

1. Indexing


```
fruits = ["apple", "banana", "cherry", "date"]

# Positive Indexing (Left-to-Right)
print(fruits[0])    # Output: apple
print(fruits[2])    # Output: cherry

# Negative Indexing (Right-to-Left)
print(fruits[-1])   # Output: date (Last element)
print(fruits[-3])   # Output: banana
```

2. Slicing

Slicing extracts a sub-list using the syntax `list[start:stop:step]` (stop index is exclusive).

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:6])    # Output: [2, 3, 4, 5] (index 2 to 5)
print(numbers[:4])     # Output: [0, 1, 2, 3] (start to index 3)
print(numbers[5:])     # Output: [5, 6, 7, 8, 9] (index 5 to end)
print(numbers[::2])    # Output: [0, 2, 4, 6, 8] (every second element)
print(numbers[::-1])   # Output: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (reverses list)
```

1.3 Memory Behavior: Aliasing vs. Copying (Crucial!)

Because lists are mutable, you must understand how Python manages variables pointing to them in memory.

1. Aliasing (Sharing References)

When you assign one list variable to another, Python does **not** create a copy of the list. Instead, both variables point to the **same object in memory**.

```
list1 = [1, 2, 3]
list2 = list1 # list2 is now an alias for list1

list2.append(99)
print(list1) # Output: [1, 2, 3, 99] (list1 changed because list2 is list1!)
print(id(list1) == id(list2)) # Output: True
```

2. Shallow Copy (Outer-level Copy)

A **Shallow Copy** creates a new list container, but copies references to the items inside. If your list contains nested mutable objects (like nested lists), the copy and the original will still share the same nested sub-lists!

- **Methods:** Use `.copy()`, slice notation `[:]`, or the `list()` constructor.

```
# Shallow copy with simple values (works fine)
simple1 = [1, 2, 3]
simple2 = simple1.copy()
simple2.append(99)
print(simple1) # Output: [1, 2, 3] (Original is unaffected)

# Shallow copy with nested lists (shares reference to inner lists)
nested1 = [[1, 2], [3, 4]]
nested2 = nested1.copy()

# Modify the nested sub-list in the copy
nested2[0][0] = 99
print(nested1) # Output: [[99, 2], [3, 4]] (Original was modified!)
print(id(nested1[0]) == id(nested2[0])) # Output: True (Inner lists share the s
```

3. Deep Copy (Recursive Copy)

A **Deep Copy** recursively copies all objects inside the list, creating entirely new, independent copies of all nested mutable elements.

- **Method:** Use Python's built-in `copy` module and call `copy.deepcopy()` .

```
import copy

nested1 = [[1, 2], [3, 4]]
nested2 = copy.deepcopy(nested1) # Recursively copies nested lists

# Modify the nested sub-list in the deep copy
nested2[0][0] = 99
print(nested1) # Output: [[1, 2], [3, 4]] (Original is completely safe and unaf
print(nested2) # Output: [[99, 2], [3, 4]]
print(id(nested1[0]) == id(nested2[0])) # Output: False (Separate memory alloca
```

List Modification & Methods

Since lists are mutable, we can add, modify, or remove elements in-place.

2.1 Modifying Elements by Index

```
items = ["phone", "laptop", "tablet"]
items[1] = "desktop"
print(items) # Output: ['phone', 'desktop', 'tablet']
```

2.2 Adding Elements

- `.append(item)` : Adds an item to the end of the list.
- `.insert(index, item)` : Inserts an item at a specific index, shifting subsequent items to the right.
- `.extend(iterable)` : Appends all items of another iterable (like a list) to the end.

```
shopping = ["milk", "bread"]

# Append
shopping.append("eggs")
print(shopping)  # Output: ['milk', 'bread', 'eggs']

# Insert
shopping.insert(1, "butter")
print(shopping)  # Output: ['milk', 'butter', 'bread', 'eggs']

# Extend
snacks = ["chips", "cookies"]
shopping.extend(snacks)
print(shopping)  # Output: ['milk', 'butter', 'bread', 'eggs', 'chips', 'cookies']
```

2.3 Removing Elements

- `.remove(item)` : Removes the first occurrence of `item` from the list. Raises a `ValueError` if the item is not found.
- `.pop(index)` : Removes and returns the item at `index` . If no index is provided, it removes and returns the **last** item.
- `del list[index]` : Deletes the element at the specified index or slice range.
- `.clear()` : Removes all elements, leaving the list empty.

```
tasks = ["code", "test", "deploy", "test"]

# Remove first occurrence
tasks.remove("test")
print(tasks)  # Output: ['code', 'deploy', 'test']

# Pop last element
popped_item = tasks.pop()
print(f"Popped: {popped_item}")  # Output: Popped: test
print(tasks)  # Output: ['code', 'deploy']

# Pop by index
first_task = tasks.pop(0)
print(f"Popped index 0: {first_task}")  # Output: Popped index 0: code
print(tasks)  # Output: ['deploy']

# Del statement
numbers = [10, 20, 30, 40]
del numbers[1:3]  # Deletes indices 1 and 2
print(numbers)  # Output: [10, 40]
```

2.4 Searching and Sorting Operations

- `.index(item)` : Returns the index of the first occurrence of `item`. Raises `ValueError` if not present.
- `.count(item)` : Returns the number of times `item` appears in the list.
- `.sort()` : Sorts the list in-place (ascending order).
- `.reverse()` : Reverses the elements of the list in-place.

```
grades = [90, 75, 88, 75, 95]

print(grades.count(75)) # Output: 2
print(grades.index(88)) # Output: 2

# Sort in-place (ascending)
grades.sort()
print(grades) # Output: [75, 75, 88, 90, 95]

# Sort in-place (descending)
grades.sort(reverse=True)
print(grades) # Output: [95, 90, 88, 75, 75]

# Reverse in-place
grades.reverse()
print(grades) # Output: [75, 75, 88, 90, 95]
```

List Operators & Helpers

3.1 Common List Operators

- **Concatenation (+)**: Joins two lists to form a **new** list.
- **Repetition (*)**: Repeats the list elements a specified number of times, returning a **new** list.
- **Membership (in / not in)**: Checks if an item exists inside a list, returning a Boolean.
- **In-Place Concatenation (+=)**: Appends the elements of another list to the existing list in-place (equivalent to `.extend()`).
- **In-Place Repetition (*=)**: Multiplies the elements of the list in-place.

Memory Comparison: Standard vs. In-Place Operators

Because lists are mutable, there is a major difference in memory handling between standard operators and their in-place shorthands:

```
# 1. Standard Concatenation vs. In-Place
lst = [1, 2]
print(id(lst))          # E.g., 4390192832

# Standard Concatenation (creates a NEW list object)
lst = lst + [3, 4]
print(id(lst))          # E.g., 4390195584 (Different ID - new list created!)

# In-Place Concatenation (modifies existing list in-place)
lst += [5, 6]
print(id(lst))          # E.g., 4390195584 (Same ID - modified in-place!)
```

```
# 2. Basic Operator Usage Examples
group1 = [1, 2]
group2 = [3, 4]

# Plus operator
combined = group1 + group2
print(combined)  # Output: [1, 2, 3, 4]

# Multiply operator
repeated = group1 * 3
print(repeated)  # Output: [1, 2, 1, 2, 1, 2]

# In-place repetition
group1 *= 2
print(group1)    # Output: [1, 2, 1, 2]

# Membership tests
colors = ["red", "green", "blue"]
print("red" in colors)      # Output: True
print("yellow" not in colors) # Output: True
```

Section 2: List Comprehensions & Conversions

List comprehensions provide a clean, concise syntax for creating a new list by executing an operation on each element of an existing sequence.

4.1 Basic Syntax

The syntax for list comprehensions is written inside square brackets. Keywords are highlighted in `<span style="color: #d73a49">red</span>`, and the optional filtering clause is enclosed in `[ ]`:

```
new_list = [expression <b><span style="color: #d73a49">for</span></b> item <b><s
```

- **expression** : The output value or operation to perform on each item (e.g., `x ** 2`, `x.upper()`).
- **item** : The variable representing the current element from the iterable (e.g., `x`, `num`, `word`).
- **iterable** : The sequence or collection being looped over (e.g., `range()`, `list`, `string`).
- **[if condition]** : An **optional** filter. The item is only processed if this condition evaluates to `True` .

4.2 Comparison: Standard For Loop vs. List Comprehension

Let's create a list of squares of even numbers from 1 to 5.

Traditional Way:

```
squares = []
for x in range(1, 6):
    if x % 2 == 0:
        squares.append(x ** 2)
print(squares) # Output: [4, 16]
```

List Comprehension Way:


```
squares = [x ** 2 for x in range(1, 6) if x % 2 == 0]
print(squares) # Output: [4, 16]
```

4.3 Practical Use Cases of List Comprehensions

List comprehensions are not just syntactic sugar; they are widely used in Python for clean and efficient data processing. Here are the most common practical use cases:

1. Data Type Conversion (Type Casting)

Often, inputs read from a file or user terminal are received as strings. List comprehensions make it easy to parse them into numerical types.

```
string_numbers = ["10", "20", "30", "40"]
integers = [int(num) for num in string_numbers]
print(integers) # Output: [10, 20, 30, 40]
```

2. Text Cleaning & Normalization

You can clean lists of user strings (e.g., removing whitespace and converting to lowercase) in a single line.

```
raw_cities = ["  Bangalore  ", " MANGALORE", "chennai  ", "Delhi"]
clean_cities = [city.strip().title() for city in raw_cities]
print(clean_cities) # Output: ['Bangalore', 'Mangalore', 'Chennai', 'Delhi']
```

3. Filtering Data

Extracting specific items from a list that match a logical condition.

```
emails = ["vinod@vinod.co", "kishori@acts.in", "student@gmail.com", "admin@vinod.co"]

# Keep only emails belonging to the 'vinod.co' domain
corporate_emails = [email for email in emails if email.endswith("@vinod.co")]
print(corporate_emails) # Output: ['vinod@vinod.co', 'admin@vinod.co']
```

4. Conditional Transformations (If-Else Expressions)

If you want to transform elements *and* include a fallback value when the condition is false, you can write the `if-else` statement **before** the `for` loop.

- **Syntax:** `[expr_if_true if condition else expr_if_false for item in iterable]`

```
scores = [45, 88, 30, 92, 50]
# Classify scores as "Pass" (>= 50) or "Fail" (< 50)
results = ["Pass" if score >= 50 else "Fail" for score in scores]
print(results) # Output: ['Fail', 'Pass', 'Fail', 'Pass', 'Pass']
```

5. Flattening a 2D List (Nested Loops)

You can flatten a multi-dimensional array (list of lists) into a flat 1D list using nested loop syntax inside the comprehension.

- **Syntax:** `[item for sublist in matrix for item in sublist]` (loops are written in the order they would be nested traditionally).

```
matrix = [[1, 2], [3, 4], [5, 6]]
flat_list = [num for row in matrix for num in row]
print(flat_list) # Output: [1, 2, 3, 4, 5, 6]
```

Converting between Lists and Strings

Converting data between text strings and list collections is one of the most common scripting tasks.

5.1 Splitting Strings to Lists: `.split()`

The string method `.split(separator)` splits a single string into a list of strings based on the specified separator pattern. If no separator is provided, it splits by any whitespace.

```
csv_data = "apple,banana,cherry"
fruits_list = csv_data.split(",")
print(fruits_list) # Output: ['apple', 'banana', 'cherry']

sentence = "Python is awesome"
words = sentence.split() # Splits by spaces
print(words) # Output: ['Python', 'is', 'awesome']
```

5.2 Joining List items to Strings: `.join()`

The string method `separator.join(list)` joins a list of strings into a single string, inserting the separator string in between elements.

```
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence) # Output: Python is awesome

items = ["milk", "eggs", "bread"]
comma_separated = ", ".join(items)
print(comma_separated) # Output: milk, eggs, bread
```

Note: `.join()` only works if all elements inside the list are strings. If you have integers, cast them to strings first.

Section 3: Beginner Pitfalls & Common Mistakes

1. The `IndexError`

Trying to access or modify an index that does not exist in the list.

```
names = ["Alice", "Bob"]
# print(names[2]) # IndexError: list index out of range
```

Tip: Always use `len(list)` to verify boundaries.

2. Modifying a List while Iterating Over It

Modifying a list (adding or removing items) while looping over it using a `for` loop causes indices to shift, leading to skipped elements or logic errors.

```
# Dangerous Example (Avoid this):
nums = [1, 2, 3, 4]
for num in nums:
    if num % 2 == 0:
        nums.remove(num) # Modifying inside iteration!
```

Fix: Iterate over a copy of the list instead:

```
for num in nums.copy():
    if num % 2 == 0:
        nums.remove(num)
```

Practice Assignments

Objective

Practice list declarations, indexing, slicing, modification methods, list operators, list comprehensions, and nested lists through interactive, scenario-based problems.

IMPORTANT:

Functions (`def`) have not been taught yet.

Write all assignments as procedural scripts. Prompt the user for input using `input()` , perform list processing, and print results using `print()` .

Part A: Easy Complexity (3 Exercises)

Assignment 1: The Wizard's Magic Bag

Scenario: A wizard has a magic bag containing a sequence of items: `["staff", "potion", "spellbook"]` . When the wizard steps through a magic portal, two things happen:

1. A new item enters the bag (prompts the user to input the item name to append to the end).
2. The oldest item in the bag (at index 0) is dissolved and ejected.

Write a program to simulate this portal transition and print the final bag contents.

- **Sample Input:** (User inputs `"amulet"`)
- **Sample Output:**

```
Portal transition activated!
Ejected oldest item: staff
Current items in the magic bag: ['potion', 'spellbook', 'amulet']
```

Assignment 2: Movie Night Playlist

Scenario: You are organizing a movie marathon. You start with a playlist: `["Inception", "The Matrix", "Interstellar"]` . Prompt the user to enter the name of a movie they want to add.

- If the movie is already in the list, print `"Already added!"` and do not insert it.
- If it is not in the list, append it to the end of the list.

Finally, sort the movie list alphabetically and print the updated playlist.

- **Sample Input:** "Interstellar"
- **Sample Output:**

```
Already added!
Alphabetical Playlist: ['Inception', 'Interstellar', 'The Matrix']
```

- **Sample Input:** "Avatar"
- **Sample Output:**

```
Added Avatar!
Alphabetical Playlist: ['Avatar', 'Inception', 'Interstellar', 'The Matrix']
```

Assignment 3: The Cargo Train Scanner

Scenario: A train has wagons carrying different resources: ["coal", "iron", "gold", "coal", "timber", "coal"]. The train conductor wants to inspect the cargo. Write a program that prompts the user to enter a resource type (e.g., "coal" or "gold").

- Print the total number of wagons carrying that resource (using `.count()`).
- If the resource is on the train, print the index of the very first wagon carrying it (using `.index()`). If it is not found, print "Resource not found on train!".
- **Sample Input:** "coal"
- **Sample Output:**

```
Number of coal wagons: 3
First coal wagon is at index: 0
```

- **Sample Input:** "oil"
- **Sample Output:** "Resource not found on train!"

Part B: Medium Complexity (5 Exercises)

Assignment 4: Nightclub VIP Queue

Scenario: A nightclub bouncer maintains a list of VIP guests who are allowed inside: `["Guido", "Esha", "Rajan", "Kishori"]`. As guests arrive at the door, the bouncer prompts the user to enter their name.

- If the guest is on the VIP list, move them from their current position in the queue and insert them at the front of the queue (index 0).
- If the guest is not on the VIP list, print "Access denied. Not on the VIP list." and do not modify the list.

Run this program in a loop. The loop should stop when the user types `"exit"`. Print the updated queue state after each guest arrives.

- **Sample Walkthrough:**

```
Current VIP queue: ['Guido', 'Esha', 'Rajan', 'Kishori']
Enter guest name: Rajan
Rajan moved to the front!
Current VIP queue: ['Rajan', 'Guido', 'Esha', 'Kishori']

Enter guest name: Vinod
Access denied. Not on the VIP list.
Current VIP queue: ['Rajan', 'Guido', 'Esha', 'Kishori']

Enter guest name: exit
```

Assignment 5: The Spy's Word Reverser

Scenario: A secret agent wants to send an encrypted message. The encryption rule is simple: **reverse every word in the sentence, but keep the order of words unchanged**. Write a program that prompts the user for a sentence, splits it, uses a list comprehension to reverse the letters of each word, and joins them back together.

- **Sample Input:** `"Meet me at midnight"`
- **Sample Output:** `"teeM em ta thgindim"`

Assignment 6: Grading on a Curve

Scenario: A professor wants to adjust exam grades. Prompt the user to enter a list of space-separated test scores. Convert them to a list of integers. Using a **single list comprehension with conditionals**, apply the following curve rules:

- If a score is below 50, add 10 points.
- If a score is 50 or higher, add 5 points.
- The maximum possible score is capped at 100 (e.g., a score of 98 becomes 100, not 103).

Print the original and the curved grades.

- **Sample Input:** "45 88 30 98 50"
- **Sample Output:**

```
Original: [45, 88, 30, 98, 50]
Curved: [55, 93, 40, 100, 55]
```

Assignment 7: Treasure Map Coordinate Filter

Scenario: You have a list of coordinate pairs representing suspected treasure locations on a map: `coords = [[12, 5], [-3, 14], [8, -2], [15, 9], [-5, -6]]`. However, the treasure can only exist in the first quadrant of the map (where **both** the X coordinate and Y coordinate are strictly greater than zero (i.e., `x > 0` and `y > 0`)). Write a program that uses a list comprehension to filter the list and print only the valid coordinates.

- **Hardcoded Input:** `coords = [[12, 5], [-3, 14], [8, -2], [15, 9], [-5, -6]]`
- **Sample Output:** `[[12, 5], [15, 9]]`

Assignment 8: De-duplicating Shopping Cart

Scenario: An online shopping cart has duplicate items due to double-clicks: `["apple", "banana", "apple", "orange", "banana", "banana"]`. Write a program that processes the list and removes all duplicate items, **but keeps the first occurrence of each item in its original order**. Print the cleaned cart.

- **Hardcoded Input:** `cart = ["apple", "banana", "apple", "orange", "banana", "banana"]`

- **Sample Output:** `['apple', 'banana', 'orange']`
-

Part C: Difficult Complexity (2 Exercises)

Assignment 9: The Josephus Elimination Game

Scenario: A group of N soldiers (numbered 1 to N) stand in a circle. Starting from the first soldier, every K -th soldier is eliminated from the circle. The count continues with the next remaining soldier, moving clockwise. This process repeats until only one soldier remains. Write a program that prompts the user to enter N (number of soldiers) and K (elimination interval). Simulate the game using a list and print the order of eliminations and the final survivor.

- **Sample Input:** `N = 5 , K = 2`
- **Sample Output:**

```
Soldier circle initialized: [1, 2, 3, 4, 5]
Eliminated soldier: 2 (Remaining: [1, 3, 4, 5])
Eliminated soldier: 4 (Remaining: [1, 3, 5])
Eliminated soldier: 1 (Remaining: [3, 5])
Eliminated soldier: 5 (Remaining: [3])
The sole survivor is: 3
```

Assignment 10: Snake Game Board Renderer

Scenario: Render a simple 2D text game board. Write a program that performs the following steps in sequence:

1. Creates a 5×5 grid filled with dots `"."` represented as a nested list.
2. Places a food item `"F"` at grid position `[2, 3]`.
3. Prompts the user to enter coordinate inputs: a `row` and a `col` (integers between 0 and 4) for the snake's head.
4. Places the snake's head `"S"` at the user-supplied coordinate `[row, col]`, overwriting the character at that position.

5. If the user-supplied coordinates are exactly `[2, 3]` , print the message `"Yum! The snake ate the food!"` (the snake `"S"` will occupy index `[2, 3]` on the printed board, overwriting the `"F"`).

6. Prints the grid neatly line-by-line (each row's elements separated by spaces).

- **Sample Input:** (User inputs Row `0` and Column `3`)

- **Sample Output:**

```
. . . S .  
. . . . .  
. . . F .  
. . . . .  
. . . . .
```

- **Sample Input:** (User inputs Row `2` and Column `3`)

- **Sample Output:**

```
. . . . .  
. . . . .  
. . . S .  
. . . . .  
. . . . .  
Yum! The snake ate the food!
```

Chapter 4: Dictionaries & Exception Handling

Welcome to Chapter 4! Today we cover two essential pillars of robust Python programming:

1. **Dictionaries:** Python's native implementation of associative arrays or hash maps.
2. **Exception Handling:** The mechanism to handle runtime errors gracefully, keeping programs running under unexpected conditions.

Section 1: Associative Mapping Structures (Dictionaries)

1. Introduction to Dictionaries

A **dictionary** in Python is an unordered collection (insertion-ordered starting from Python 3.7) of items. Each item is stored as a **key-value pair**.

- **Key:** Must be unique and **hashable** (immutable types such as strings, numbers, or tuples containing only immutable elements).
- **Value:** Can be of any arbitrary Python data type (lists, dictionaries, integers, custom objects, etc.) and does not need to be unique.

Dictionaries are optimized for retrieving data. Under the hood, Python uses a hash table structure, allowing lookup, insertion, and deletion operations in average $O(1)$ time complexity.

2. Defining Dictionaries

There are multiple ways to define a dictionary:

```

# 1. Empty dictionary
empty_dict_1 = {}
empty_dict_2 = dict()

# 2. Dictionary literal with data
student = {
    "name": "Arham",
    "age": 21,
    "course": "PGCP-AI",
    "grades": [85, 90, 88]
}

# 3. Using the dict() constructor with keyword arguments
employee = dict(name="Lisa", id=1042, department="R&D")

# 4. Using dict() with list of tuples (key-value pairs)
colors = dict([("red", "#FF0000"), ("green", "#00FF00"), ("blue", "#0000FF")])

print("Student:", student)
print("Employee:", employee)
print("Colors:", colors)

```

3. Accessing Items

You can access values using their corresponding keys. Python offers two primary methods:

A. Bracket Notation (`dict[key]`)

Directly look up a key. If the key does not exist, Python raises a `KeyError`.

```

profile = {"username": "vinod_k", "role": "admin"}

# Valid access
print(profile["username"]) # Output: vinod_k

# Invalid access (raises KeyError)
try:
    print(profile["email"])
except KeyError as e:
    print(f"KeyError caught: Key {e} does not exist.")

```

B. The Safe `.get()` Method

Returns the value if the key exists; otherwise, returns `None` or a specified default value. It **never** raises a `KeyError`.

```
profile = {"username": "vinod_k", "role": "admin"}

# Safe retrieval
email = profile.get("email")
print("Email:", email) # Output: Email: None

# Safe retrieval with custom default
email_with_default = profile.get("email", "no-email@example.com")
print("Email (with default):", email_with_default) # Output: no-email@example.c
```

C. Retrieving Views (`.keys()` , `.values()` , and `.items()`)

These methods return dynamic view objects that reflect dictionary changes in real time.

```
inventory = {"apples": 10, "bananas": 24}

# View of keys
keys_view = inventory.keys()
print("Keys:", list(keys_view)) # Output: ['apples', 'bananas']

# View of values
values_view = inventory.values()
print("Values:", list(values_view)) # Output: [10, 24]

# View of key-value tuples
items_view = inventory.items()
print("Items:", list(items_view)) # Output: [('apples', 10), ('bananas', 24)]
```

4. Modifying and Adding Items

Dictionaries are mutable. You can add new key-value pairs or update existing ones.

```

car = {"brand": "Tesla", "model": "Model 3"}

# Adding a new key-value pair
car["year"] = 2023

# Modifying an existing value
car["model"] = "Model S"

# Using the .update() method to add/modify multiple items at once
car.update({"color": "red", "year": 2024})

print("Updated Car:", car)
# Output: {'brand': 'Tesla', 'model': 'Model S', 'year': 2024, 'color': 'red'}

```

5. Deleting Items

Python provides several ways to delete entries:

```

stats = {"HP": 100, "MP": 50, "Speed": 75, "Defense": 60}

# 1. del keyword: Removes key-value pair. Raises KeyError if key doesn't exist.
del stats["Defense"]

# 2. .pop(): Removes key and returns its value. Returns default if key is not found.
mp_value = stats.pop("MP")
print(f"Popped MP value: {mp_value}")

speed_fallback = stats.pop("Stamina", 0) # Stamina is not in stats, returns 0
print(f"Popped Stamina (fallback): {speed_fallback}")

# 3. .popitem(): Removes and returns the last inserted key-value pair as a tuple
last_item = stats.popitem()
print(f"Popped last item: {last_item}") # Output: ('Speed', 75)

# 4. .clear(): Wipes the entire dictionary, making it empty.
stats.clear()
print("Cleared stats:", stats) # Output: {}

```

6. Dictionary Comprehensions

Similar to list comprehensions, dictionary comprehensions provide a concise way to construct dictionaries from iterables.

Syntax:

```
{key_expression: value_expression for item in iterable if condition}
```

Example:

```
# Create a dictionary of squares for even numbers from 1 to 10
squares = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print("Even Squares:", squares)
# Output: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

# Inverting a dictionary (assuming unique values)
original = {"a": 1, "b": 2, "c": 3}
inverted = {value: key for key, value in original.items()}
print("Inverted:", inverted)
# Output: {1: 'a', 2: 'b', 3: 'c'}
```

7. Iterating Through Dictionaries

You can loop through a dictionary in different ways:

```
user_roles = {"alice": "manager", "bob": "developer", "charlie": "tester"}

print("--- Iterating over Keys (Default) ---")
for name in user_roles:
    print(name)

print("\n--- Iterating over Values ---")
for role in user_roles.values():
    print(role)

print("\n--- Iterating over Key-Value Pairs ---")
for name, role in user_roles.items():
    print(f"User: {name} | Role: {role}")
```

Section 2: Robust Error Isolation (Exception Handling)

1. Understanding Exceptions

An **exception** is an error that occurs during the execution of a program (runtime). When Python encounters an error it cannot handle, it creates (or "raises") an exception object. If unhandled, the program terminates abruptly (crashes).

Common built-in exceptions include:

- `ZeroDivisionError` : Raised when dividing a number by zero.
- `ValueError` : Raised when a function receives an argument of correct type but inappropriate value (e.g., trying to convert `"abc"` to an integer).
- `KeyError` : Raised when a dictionary key is not found.
- `IndexError` : Raised when a sequence subscript is out of range.
- `TypeError` : Raised when an operation is applied to an object of inappropriate type.
- `FileNotFoundError` : Raised when a file or directory is requested but does not exist.

2. The `try-except` Block

To prevent crashes, wrap error-prone code inside a `try` block, and handle potential errors inside one or more `except` blocks.

```
try:
    number = int(input("Enter an integer: "))
    result = 100 / number
    print(f"Result: {result}")
except ValueError:
    print("Error: That was not a valid integer!")
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Catching Multiple Exceptions in a Single Block

You can group multiple exceptions into a tuple if they share the same handling logic:


```
try:
    # Potentially problematic operations
    data = [10, 20]
    val = data[5] / 0
except (IndexError, ZeroDivisionError) as e:
    print(f"An index or arithmetic error occurred: {e}")
```

Catching All Exceptions (Generic Catch)

Use a generic `except Exception as e` to catch all standard errors. Avoid using a bare `except:` as it catches system-exiting signals (`SystemExit` , `KeyboardInterrupt`), which makes stopping your program with `Ctrl+C` difficult.

```
try:
    x = 1 / 0
except Exception as e:
    print(f"Something went wrong: {e}")
```

3. The `else` Clause

The `else` block runs **only if no exceptions were raised** in the `try` block. It is useful for separating the code that might cause exceptions from code that should execute only upon successful completion.

```
try:
    file_content = "105"
    number = int(file_content)
except ValueError:
    print("Could not parse file content as an integer.")
else:
    # Executes only if no ValueError occurred
    print(f"Successfully parsed number: {number}")
    double_val = number * 2
    print(f"Double: {double_val}")
```

4. The `finally` Clause (Cleanup)

The `finally` block **always executes**, regardless of whether an exception was raised, caught, or completely unhandled. It is primarily used to release external resources (like files, database connections, or network sockets).

```
try:
    print("Opening transaction log...")
    # Simulate a crash inside the try block
    result = 1 / 0
except ZeroDivisionError:
    print("Handling division by zero...")
finally:
    # This block executes no matter what
    print("Closing transaction log safely. Done.")
```

Output:

```
Opening transaction log...
Handling division by zero...
Closing transaction log safely. Done.
```

5. Raising Exceptions (`raise`)

You can manually trigger an exception using the `raise` keyword. This is useful for enforcing business rules or validating function arguments.

```
def set_percentage(value):
    if value < 0 or value > 100:
        raise ValueError("Percentage must be between 0 and 100 inclusive.")
    print(f"Percentage set to: {value}%")

try:
    set_percentage(150)
except ValueError as e:
    print(f"Validation failed: {e}")
```

6. Custom Exceptions

You can define custom exceptions to represent errors specific to your application domain. To do this, inherit from the built-in `Exception` class.

```
# Define a custom exception class
class InsufficientFundsError(Exception):
    """Raised when an account withdrawal exceeds the available balance."""
    def __init__(self, balance, amount):
        super().__init__(f"Attempted to withdraw ${amount} with a balance of ${b
        self.balance = balance
        self.amount = amount

# Usage
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    current_balance = 50
    new_balance = withdraw(current_balance, 75)
except InsufficientFundsError as e:
    print(f"Transaction Rejected: {e}")
```

7. Behavior of `return` in `try-except-finally`

A common conceptual pitfall: **What happens if a function executes `return` statements inside both the `try` (or `except`) block AND the `finally` block?**

Rule: The `finally` block's `return` statement will override any prior `return` statements or active exceptions in the `try` or `except` blocks.

```
def check_return_behavior():
    try:
        print("Inside try block")
        return "Return from try"
    except Exception:
        return "Return from except"
    finally:
        print("Inside finally block")
        return "Return from finally" # This overrides the try block's return

result = check_return_behavior()
print("Result of function call:", result)
```

Output:

```
Inside try block
Inside finally block
Result of function call: Return from finally
```

WARNING:

Putting `return` statements inside `finally` blocks is generally discouraged because it can suppress unhandled exceptions silently, making debugging difficult.

Practical Examples (Interactive & Runnable)

Example 1: Document Word Frequency Counter

A complete program that processes text to count words, utilizing string methods, dictionary operations, and sorting.

```

def count_word_frequencies(paragraph):
    # Dictionary to hold the word counts
    word_counts = {}

    # Preprocessing: remove punctuation, convert to lowercase, and split
    cleaned_text = ""
    for char in paragraph.lower():
        if char.isalnum() or char.isspace():
            cleaned_text += char
        else:
            cleaned_text += " " # Replace punctuation with spaces

    words = cleaned_text.split()

    # Counting frequencies
    for word in words:
        # Using get() to safely handle initial counting
        word_counts[word] = word_counts.get(word, 0) + 1

    return word_counts

# Run Example
sample_text = "Python is amazing! Python is fast, and Python is easy to learn."
frequencies = count_word_frequencies(sample_text)

# Sort dictionary by value (frequencies) in descending order
sorted_frequencies = dict(sorted(frequencies.items(), key=lambda item: item[1],

print("Word Frequencies:")
for word, count in sorted_frequencies.items():
    print(f" - {word}: {count}")

```

Example 2: Robust Numeric Input Reader

An interactive loop that guarantees retrieval of a valid number from user terminal input.

```
def read_valid_integer(prompt, min_val=0, max_val=100):
    while True:
        try:
            user_input = input(prompt)
            # Try to convert input to integer
            value = int(user_input)

            # Business rule validation
            if value < min_val or value > max_val:
                raise ValueError(f"Value must be between {min_val} and {max_val}")

        except ValueError as err:
            # Catches both non-numeric text and values outside range
            print(f"Invalid input: {err}. Please try again.\n")
        else:
            # Executes only if conversion and validation succeeded
            print("Input successfully accepted!")
            return value
```

Practice Assignments

Objective

Model key-value storage relations, implement robust custom exceptions, and design atomic transaction processing using try-except-finally blocks.

Easy Assignments

Assignment 1: Inventory Tracker for Campus Bookstore

Scenario

The Campus Bookstore needs a backend helper module to manage books and their quantities. The inventory is stored in a Python dictionary where keys are book titles (strings) and values are

quantities in stock (non-negative integers).

Problem Description

Write a function `manage_bookstore_inventory(inventory, action, book_title, quantity=0)` that handles inventory operations safely.

1. The `action` parameter can be one of three options: `"add"`, `"sell"`, or `"lookup"`.

2. Add Action (`"add"`):

- Add the specified `quantity` to the existing stock of `book_title`.
- If the book is not in the inventory dictionary, add it as a new key with `quantity` as the value.

1. Sell Action (`"sell"`):

- Decrease the stock of `book_title` by the specified `quantity`.
- If the book is not found in the inventory, print a message: `Error: Book '<book_title>' not found in inventory.` and make no changes. (Do not let the program crash with a `KeyError`).
- If the requested quantity to sell exceeds the stock available, print: `Error: Insufficient stock for '<book_title>'. Available: <current_stock>.` and make no changes.
- If the stock reaches exactly `0` after a successful sale, remove the book key from the inventory entirely.

1. Lookup Action (`"lookup"`):

- Look up the stock quantity of `book_title` and return it.
- Use safe dictionary retrieval; if the book does not exist, return `0` without throwing a `KeyError`.

The function must return the updated/current inventory dictionary.

Example Walkthrough

```

## Initial Inventory
inventory = {"Python Basics": 10, "Learning AI": 5}

## 1. Add Stock
inventory = manage_bookstore_inventory(inventory, "add", "Python Basics", 5)
## Result: {"Python Basics": 15, "Learning AI": 5}

## 2. Sell Stock Safely (Missing Book)
inventory = manage_bookstore_inventory(inventory, "sell", "Data Science 101", 1)
## Console output: Error: Book 'Data Science 101' not found in inventory.

## 3. Sell Stock (Insufficient)
inventory = manage_bookstore_inventory(inventory, "sell", "Learning AI", 10)
## Console output: Error: Insufficient stock for 'Learning AI'. Available: 5.

## 4. Sell Stock (Exactly Zero Stock)
inventory = manage_bookstore_inventory(inventory, "sell", "Learning AI", 5)
## Result: {"Python Basics": 15}

```

Assignment 2: Robust Phonebook Contact Registry

Scenario

You are writing a Command-Line Interface (CLI) contact registry that maps user names to their phone numbers. The program needs to validate user inputs robustly to prevent corrupted formatting or empty values from breaking the registry database.

Problem Description

1. Define a custom exception class named `InvalidPhoneNumberError` that inherits from `Exception`.
2. Write a function `register_contact(phonebook, name, phone_input)` :
 - `phonebook` is a dictionary mapping contact names (strings) to their phone numbers (strings).
 - Validate the `name` parameter: it must be a non-empty string consisting only of alphabetic characters and spaces. If invalid, raise a standard `ValueError` with the message: "Contact

name must be a non-empty alphabetic string."

- Validate the `phone_input` parameter: it must consist only of digits. Check this by attempting to convert it to an integer using `int()` .
- If the conversion fails (raises a `ValueError`), catch that exception and raise your custom `InvalidPhoneNumberError` with the message: "Phone number must contain digits only."
- If validations pass, store `phone_input` **as a string** in the `phonebook` under the key `name` (preserving any leading zeros).
- Return the updated `phonebook` dictionary.

Example Walkthrough

```
contacts = {}

## 1. Valid Input
contacts = register_contact(contacts, "Alice", "0987654321")
## Result: {"Alice": "0987654321"}

## 2. Invalid Phone Number (Raises InvalidPhoneNumberError)
try:
    contacts = register_contact(contacts, "Bob", "123-456-789")
except InvalidPhoneNumberError as e:
    print(e) # Output: Phone number must contain digits only.

## 3. Invalid Name (Raises ValueError)
try:
    contacts = register_contact(contacts, "Bob123", "9876543210")
except ValueError as e:
    print(e) # Output: Contact name must be a non-empty alphabetic string.
```

Medium Assignments

Assignment 3: Course Feedback Compiler & Sanitizer

Scenario

Student feedback records contain ratings from 1 to 5 stars. Due to raw data entry issues, the feedback database has some course entries with list values that are empty, or lists containing invalid elements (such as string annotations like `"Excellent"` or `None` values).

Problem Description

Write a function `compile_feedback(ratings_dict)` that processes course feedback:

- The parameter `ratings_dict` is a dictionary where keys are course names (strings) and values are lists of ratings (which should be numeric but may contain invalid types).
- The function must return a dictionary mapping each course name to its average rating, rounded to **2 decimal places**.
- Implement the following error handling criteria:
 1. For each rating inside a course's list, attempt to convert it to a `float`. If a rating cannot be converted (throws a `ValueError` or `TypeError`), catch the exception, print a warning: `"Warning: Invalid rating value '<val>' in course '<course>' skipped."`, and continue processing the rest of the list.
 2. If a course has no valid ratings (the list is empty or contains no convertible numbers), computing the average will trigger a division-by-zero error. Catch `ZeroDivisionError`, print a warning: `"Warning: No valid ratings found for course '<course>'. Rating set to 0.0."`, and assign the course an average rating of `0.0`.

Sample Input

```
feedback_data = {
    "Python Programming": [5, 4, "4", "Great", 5],
    "Machine Learning": [],
    "Deep Learning": ["Good", "Average", None]
}
```

Expected Output

Console Warnings Printed:

```
Warning: Invalid rating value 'Great' in course 'Python Programming' skipped.
Warning: No valid ratings found for course 'Machine Learning'. Rating set to 0.0
Warning: Invalid rating value 'Good' in course 'Deep Learning' skipped.
Warning: Invalid rating value 'Average' in course 'Deep Learning' skipped.
Warning: Invalid rating value 'None' in course 'Deep Learning' skipped.
Warning: No valid ratings found for course 'Deep Learning'. Rating set to 0.0.
```

Returned Dictionary:

```
{
    "Python Programming": 4.5,
    "Machine Learning": 0.0,
    "Deep Learning": 0.0
}
```

Assignment 4: Atomic E-Commerce Order Processor

Scenario

You are building an ordering subsystem for an online store. Orders containing multiple products must be processed **atomically**: either the entire order completes successfully, or the entire transaction fails. If one item in the order is out of stock or is unrecognized, no stock should be deducted for any other item (rollback).

Problem Description

1. Define two custom exceptions:

- `ProductNotFoundError` (raised when a product ID is not present in the catalog).
- `OutOfStockError` (raised when the customer's ordered quantity exceeds the available stock).

1. Write a function `process_order(catalog, order)` :

- `catalog` is a dictionary containing product database records. Format:

```
catalog = {
    "P01": {"price": 100.0, "stock": 5},
    "P02": {"price": 50.0, "stock": 2}
}
```

- `order` is a dictionary containing product IDs (keys) and quantities ordered (values).
Format: `{"P01": 2, "P02": 1}`.
- **Validation Phase:** Before modifying any inventory levels:
 - Check if all ordered keys exist in the catalog. If a product ID does not exist, raise `ProductNotFoundError` with message: `"Product '<product_id>' not found in store catalog."`
 - Check if the catalog contains sufficient stock for each item ordered. If the ordered quantity exceeds available stock, raise `OutOfStockError` with message: `"Product '<product_id>' is out of stock. Requested: <requested_qty>, Available: <available_stock>."`
- **Execution Phase:** If (and only if) all products pass validation:
 - Deduct the ordered quantities from the stock numbers in the catalog dictionary.
 - Calculate and return the total cost of the order (float).
 - If an exception was raised during validation, the catalog must remain completely unchanged.

Example Walkthrough

```

catalog = {
    "P01": {"price": 10.0, "stock": 5},
    "P02": {"price": 20.0, "stock": 10}
}

## 1. Successful Order
total = process_order(catalog, {"P01": 2, "P02": 1})
## Returns: 40.0
## Catalog stock changes to: P01 stock = 3, P02 stock = 9

## 2. Failed Order (Triggers Rollback)
## Current Catalog: {"P01": {"price": 10.0, "stock": 3}, "P02": {"price": 20.0,
try:
    total = process_order(catalog, {"P01": 2, "P02": 15})
except OutOfStockError as e:
    print(e) # Output: Product 'P02' is out of stock. Requested: 15, Available:

## Verify Catalog Stock: P01 must remain at 3 (NOT decreased to 1).
print(catalog["P01"]["stock"]) # Output: 3

```

Difficult Assignments

Assignment 5: Deep JSON/Configuration Key Traverser

Scenario

Configuration files loaded from JSON databases consist of nested dictionary hierarchies. Checking key existence at every level using nested conditions (`if key in dictionary`) leads to complex and verbose code. You need to write a clean traverser utility that navigates nested dictionaries using exceptions.

Problem Description

Write a function `traverse_nested_config(config_dict, path_str, default=None)`:

- `config_dict` is a nested dictionary configuration tree.
- `path_str` is a string specifying the configuration path using dot notation (e.g., `"server.database.port"`).

- The function should split the `path_str` on `.` characters and traverse down `config_dict`.
- **Implementation Constraint:** You **must** attempt to traverse keys directly. Do not use key-existence checks (like `if key in dict`) or class-checks (like `if isinstance(sub_dict, dict)`). Instead, handle the lookup path directly inside a `try` block and catch the following exceptions to return the `default` value:
 - Catch `KeyError` if any key in the path does not exist.
 - Catch `TypeError` or `AttributeError` if you try to index a primitive, non-dictionary value (e.g., trying to access a key like `"port"` on a configuration value that resolved to a string or number).
 - If `path_str` is empty or `config_dict` is not a valid dictionary, return the `default` value.

Test Data & Test Cases

```
config = {
    "server": {
        "host": "127.0.0.1",
        "port": 8080,
        "ssl": {
            "enabled": True,
            "cert_path": "/etc/ssl/certs"
        }
    },
    "database": "postgresql://localhost:5432"
}

## Test Case 1: Valid Path
print(traverse_nested_config(config, "server.ssl.cert_path"))
## Output: /etc/ssl/certs

## Test Case 2: Missing Key (Triggers KeyError)
print(traverse_nested_config(config, "server.database.username", "guest"))
## Output: guest

## Test Case 3: Indexing Non-Dictionary value (Triggers TypeError)
## Here config["database"] is a string, which cannot be indexed with "host"
print(traverse_nested_config(config, "database.host", "localhost"))
## Output: localhost
```

Assignment 6: Atomic Transaction processing with Log Rollback

Scenario

A bank updates user balances in a database dictionary based on transaction files. To ensure accounting consistency, if *any* single transaction in a batch contains an error (such as a negative transfer amount, an unrecognized account number, or an overdraft), the *entire batch* must fail, all accounts must be restored to their initial states, and a rollback action must be logged to a text file.

Problem Description

1. Define three custom exception classes inheriting from `Exception` :

- `AccountNotFoundError` (raised when an account ID is missing from the registry).
- `OverdraftError` (raised when a withdrawal amount exceeds the account balance).
- `InvalidTransactionError` (raised when the transaction type is unrecognized or if transaction amounts are non-positive).

1. Write a function `process_transaction_batch(accounts, batch_list, log_path)` :

- `accounts` is a dictionary where keys are account numbers (strings) and values are balances (floats), e.g., `{"ACC01": 500.0, "ACC02": 200.0}`.
- `batch_list` is a list of dictionaries representing transactions, e.g.:

```
[
    {"acc": "ACC01", "type": "deposit", "amt": 150.0},
    {"acc": "ACC02", "type": "withdraw", "amt": 50.0}
]
```

- `log_path` is a string referencing the path of the transaction log file.
- **Atomicity Requirements:**
- Create a deep copy of the `accounts` dictionary before starting any transaction modifications to act as a restore point (backup).
- Iterate through `batch_list` and apply the changes to `accounts` :

- If the transaction `"acc"` does not exist in `accounts`, raise `AccountNotFoundError` with message: `"Account '<acc>' not found."`
- If transaction `"type"` is not `"deposit"` or `"withdraw"`, raise `InvalidTransactionError` with message: `"Invalid transaction type '<type>'."`
- If transaction `"amt"` is less than or equal to `0`, raise `InvalidTransactionError` with message: `"Transaction amount must be positive."`
- If transaction `"type"` is `"withdraw"` and the account balance is less than `"amt"`, raise `OverdraftError` with message: `"Insufficient funds. Account <acc> has balance <bal>, requested <amt>."`

- **Exception Handling & Rollback:**

- If any exception is raised during the processing of the list, catch the exception:
- Restore the `accounts` dictionary to the exact state saved in your backup.
- Open the file at `log_path` (create it if it doesn't exist, append to it if it does) and write the following entry:

```
[ROLLBACK] Batch aborted: <Exception Class Name> - <Exception Message>\n
```

- Re-raise the caught exception so that the calling program knows the transaction batch failed.
- If all transactions in the batch are executed successfully:
- Open the file at `log_path` and write:

```
[SUCCESS] Batch completed. <number_of_transactions> transaction(s) processed.\n
```

- Return the updated `accounts` dictionary.
- **Constraint:** Ensure all file operations are safely cleaned up. Use context managers (`with open(...)`) or `try...finally` to write to the log file.

Example Walkthrough


```

accounts = {"ACC01": 100.0, "ACC02": 50.0}
log_file = "transactions.log"

## Batch 1: Valid transactions
batch_1 = [
    {"acc": "ACC01", "type": "withdraw", "amt": 30.0},
    {"acc": "ACC02", "type": "deposit", "amt": 20.0}
]
accounts = process_transaction_batch(accounts, batch_1, log_file)
## Result: accounts changes to {"ACC01": 70.0, "ACC02": 70.0}
## transactions.log writes: "[SUCCESS] Batch completed. 2 transaction(s) process

## Batch 2: Invalid transaction (triggers rollback)
batch_2 = [
    {"acc": "ACC01", "type": "deposit", "amt": 50.0},
    {"acc": "ACC02", "type": "withdraw", "amt": 200.0} # Overdraft!
]
try:
    accounts = process_transaction_batch(accounts, batch_2, log_file)
except OverdraftError as e:
    print(f"Caught: {e}")

## Verify Rollback: ACC01 must remain 70.0, NOT updated to 120.0.
print(accounts) # Output: {"ACC01": 70.0, "ACC02": 70.0}
## transactions.log writes: "[ROLLBACK] Batch aborted: OverdraftError - Insuffic

```

Chapter 5: Functions, Scopes & Regular Expressions

Welcome to Chapter 5! Today we will explore:

1. **Functions and Abstraction:** Organizing and modularizing code.
 2. **Scoping Rules:** How variable lookups work under the LEGB rule.
 3. **Anonymous (Lambda) Functions:** Creating light, one-line functions.
 4. **Built-in Helpers:** Inspecting and manipulating data with standard functions.
 5. **Regular Expressions (RegEx):** Pattern matching and string manipulation.
-

Section 1: Functional Abstraction & Scoping Rules

1. Defining and Calling Functions

A **function** is a reusable block of organized code used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

```
# Defining a simple function
def greet_student(name):
    """Docstring explaining the function's purpose: greet a student."""
    return f"Welcome, {name}, to Software Academy!"

# Calling the function
message = greet_student("Arham")
print(message) # Output: Welcome, Arham, to Software Academy!
```

2. Argument Passing Mechanics

Python offers extremely flexible ways to pass arguments to functions.

A. Positional and Keyword Arguments

- **Positional Arguments:** Assigned based on their position/order in the call.
- **Keyword Arguments:** Assigned by specifying parameter names during the call, allowing you to pass them in any order.

```
def describe_pet(animal_type, pet_name):  
    print(f"My {animal_type}'s name is {pet_name}.")  
  
# Positional call  
describe_pet("Hamster", "Harry") # Output: My Hamster's name is Harry.  
  
# Keyword call (order doesn't matter)  
describe_pet(pet_name="Bruno", animal_type="Dog") # Output: My Dog's name is Br
```

B. Default Parameter Values

Parameters can have default values. If a value is not supplied during execution, the default is used.

```
def make_coffee(size, flavor="Regular"):  
    print(f"Serving a {size} cup of {flavor} coffee.")  
  
make_coffee("Large") # Output: Serving a Large cup of Regular coffee  
make_coffee("Medium", "Vanilla") # Output: Serving a Medium cup of Vanilla coffe
```

IMPORTANT:

*Non-default parameters must always be declared **before** default parameters in the function definition.* `def func(a=10, b):` is syntax error.

C. Arbitrary Arguments: `*args` and `**kwargs`

- `*args` : Collects extra positional arguments as a **tuple**.
- `**kwargs` : Collects extra keyword arguments as a **dictionary**.

```
def report_achievements(student_name, *subjects, **details):
    print(f"Student: {student_name}")
    print(f"Enrolled in: {subjects}")
    print(f"Metadata:")
    for key, val in details.items():
        print(f" - {key}: {val}")

report_achievements("Lisa", "Python", "AI Basics", batch="August 2026", id="A104")
# Output:
# Student: Lisa
# Enrolled in: ('Python', 'AI Basics')
# Metadata:
#   - batch: August 2026
#   - id: A104
```

D. Keyword-Only and Positional-Only Arguments

Introduced in modern Python:

- `/` : Denotes parameters to its left must be **positional-only**.
- `*` : Denotes parameters to its right must be **keyword-only**.

```
def strict_function(pos_only, /, standard, *, kw_only):
    print(pos_only, standard, kw_only)

# Valid call
strict_function(10, "hello", kw_only="world")

# Invalid calls (will raise TypeError)
# strict_function(pos_only=10, standard="hello", kw_only="world")
# strict_function(10, "hello", "world")
```

Scoping Rules (LEGB Rule)

Python looks up variables in a specific order: **Local** $\rightarrow$ **Enclosing** $\rightarrow$ **Global** $\rightarrow$ **Built-in**.

```

+-----+
| BUILT-IN (e.g., print, len, range) |
| +-----+ |
| | GLOBAL (Module level variables) | | | | | |
| | +-----+ | |
| | | ENCLOSING (Outer nested functions) | | |
| | | +-----+ | | |
| | | | LOCAL (Current function body) | | | |
| | | +-----+ | | |
| | +-----+ | |
| +-----+ |
+-----+

```

1. Variables and Boundaries

- **Local:** Variables created inside the executing function.
- **Enclosing:** Variables inside outer scopes of nested functions.
- **Global:** Variables declared at the top-level of a module.
- **Built-in:** Names preloaded by Python (like `print()`, `ValueError`).

2. The `global` Keyword

To modify a variable defined at the module-level from inside a function, declare it as `global`.

```

count = 10 # Global variable

def increment_global():
    global count
    count += 1
    print("Inside function:", count)

increment_global() # Output: Inside function: 11
print("Global scope:", count) # Output: Global scope: 11

```

3. The `nonlocal` Keyword

In nested functions, to modify a variable in the immediate outer (enclosing) scope, declare it as `nonlocal` .

```
def outer_counter():
    step = 0 # Enclosing scope variable

    def inner():
        nonlocal step
        step += 1
        return step

    return inner

counter = outer_counter()
print(counter()) # Output: 1
print(counter()) # Output: 2
```

Section 2: Lambda Functions & Built-in Helpers

A **lambda function** is a small, anonymous function that can have any number of arguments but only a **single expression**.

Syntax:

```
lambda arguments: expression
```

```
# Simple addition lambda
add = lambda x, y: x + y
print(add(5, 7)) # Output: 12

# Commonly used with map, filter, and sorted:
numbers = [1, 2, 3, 4, 5, 6]

# 1. Filter: extract even values
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Evens:", evens) # Output: [2, 4, 6]

# 2. Map: square the list
squares = list(map(lambda x: x**2, numbers))
print("Squares:", squares) # Output: [1, 4, 9, 16, 25, 36]

# 3. Sorted: sorting tuples by second value
points = [(1, 9), (5, 2), (3, 7)]
points_sorted = sorted(points, key=lambda point: point[1])
print("Sorted Points:", points_sorted) # Output: [(5, 2), (3, 7), (1, 9)]
```

Built-in Helper Functions

Python has useful built-in inspection helpers:

- `type(obj)` : Returns the type of `obj` .
- `id(obj)` : Returns the memory identity of `obj` .
- `dir(obj)` : Lists valid attributes/methods available on `obj` .
- `enumerate(iterable)` : Returns an iterator yielding tuple pairs: `(index, item)` .
- `zip(*iterables)` : Aggregates elements from multiple iterables into tuples.

```
# Enumeration demo
names = ["Alice", "Bob"]
for idx, name in enumerate(names, start=1):
    print(f"{idx}: {name}")

# Zip demo
scores = [85, 92]
zipped = dict(zip(names, scores))
print("Zipped Dict:", zipped) # Output: {'Alice': 85, 'Bob': 92}
```

Section 3: Regular Expressions & Text Search Engines

Regular expressions are patterns used to match and extract character combinations in strings. In Python, use the `re` module.

1. Key Meta-characters

- `\d` : Matches any decimal digit (equivalent to `[0-9]`).
- `\w` : Matches alphanumeric characters and underscores (`[a-zA-Z0-9_]`).
- `\s` : Matches whitespace characters (spaces, tabs, newlines).
- `+` : Matches 1 or more repetitions of the preceding pattern.
- `*` : Matches 0 or more repetitions of the preceding pattern.
- `?` : Matches 0 or 1 repetition of the preceding pattern.
- `^` / `$` : Matches the start / end of a string.
- `.` : Matches any character except a newline.

2. Core `re` Module Functions

A. Finding Matches: `re.search()` vs `re.match()`

- `re.match()` : Checks for a match **only at the beginning** of the string.
- `re.search()` : Scans the **entire string** for a match.


```
import re

text = "Software Academy Bangalore"

# Match checks only the beginning
match_res = re.match(r"acts", text)
print("Match found:", match_res) # Output: None

# Search checks the entire string
search_res = re.search(r"acts", text)
print("Search found:", search_res.group()) # Output: acts
```

B. Getting Multiple Matches: `re.findall()` & `re.finditer()`

- `re.findall(pattern, string)` : Returns all non-overlapping matches as a list of strings.
- `re.finditer(pattern, string)` : Returns an iterator yielding match objects.

```
numbers_text = "Today is 28th, temperature is 26 degrees, speed limit is 60."
digits = re.findall(r"\d+", numbers_text)
print("Digits:", digits) # Output: ['28', '26', '60']
```

C. Substituting Patterns: `re.sub()`

Replaces occurrences of a pattern with a replacement string.

```
raw_log = "Secret code: 456-789. System OK."
# Mask numeric codes
masked_log = re.sub(r"\d+", "XXX", raw_log)
print("Masked:", masked_log) # Output: Secret code: XXX-XXX. System OK.
```

3. Capture Groups and Patterns

By surrounding parts of your regex with parentheses `()`, you define **capture groups** to extract specific subsets of matches.

```
email = "info_office@domain.com"
pattern = r"^([a-z0-9._]+)@([a-z0-9.-]+)$"

match = re.search(pattern, email)
if match:
    # group(0) returns the entire matching string
    print("Full Email:", match.group(0))
    # group(1) returns the first capture group
    print("Username:", match.group(1)) # Output: info_office
    # group(2) returns the second capture group
    print("Domain:", match.group(2))   # Output: domain.com
```

Practical Examples (Interactive & Runnable)

Example 1: Robust Password Quality Assurer

Uses a RegEx query to check password specifications.

```

import re

def is_strong_password(password):
    # Rule 1: Length >= 8
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    # Rule 2: At least one uppercase letter
    if not re.search(r"[A-Z]", password):
        return False, "Password must contain at least one uppercase letter."

    # Rule 3: At least one lowercase letter
    if not re.search(r"[a-z]", password):
        return False, "Password must contain at least one lowercase letter."

    # Rule 4: At least one digit
    if not re.search(r"\d", password):
        return False, "Password must contain at least one digit."

    # Rule 5: At least one special symbol
    if not re.search(r"[@#$%&+=!]", password):
        return False, "Password must contain at least one special character (@#$%&+=!)"

    return True, "Strong password!"

# Run tests
test_pass = "P@ssw0rd2026"
valid, feedback = is_strong_password(test_pass)
print(f"Password '{test_pass}' check: {feedback}")
# Output: Password 'P@ssw0rd2026' check: Strong password!

```

Example 2: Closure-Based Rate Limiter (Stateful Closure)

Demonstrates scopes, closures, and the `nonlocal` keyword to throttle events.

```

import time

def create_rate_limiter(max_calls, interval_seconds):
    """Creates a throttling closure state machine."""
    call_timestamps = []

    def attempt_execution(task_name):
        nonlocal call_timestamps
        current_time = time.time()

        # Keep only timestamps within the current interval window
        call_timestamps = [t for t in call_timestamps if current_time - t < interval_seconds]

        if len(call_timestamps) < max_calls:
            call_timestamps.append(current_time)
            print(f"[SUCCESS] Running task: {task_name}. Calls in window: {len(call_timestamps)}")
            return True
        else:
            print(f"[BLOCKED] Rate limit exceeded for {task_name}. Try again later.")
            return False

    return attempt_execution

# Run Example
limiter = create_rate_limiter(max_calls=2, interval_seconds=3)
limiter("Download File 1") # Success
limiter("Download File 2") # Success
limiter("Download File 3") # Blocked

```

Practice Assignments

Objective

Design modular functions with varied parameters, enforce boundary check parameters using keyword-only scopes, implement stateful closures using `nonlocal` variables, and extract structured text properties using compiled RegEx capture groups.

Easy Assignments

Assignment 1: Campus Cafeteria Discount Calculator

Scenario

The Campus Cafeteria needs a modular pricing function to calculate student bills. The cafeteria offers main combo meals, optional side-dishes, standard tax rates, promotional discounts, and delivery charges.

Problem Description

Write a function named `calculate_cafeteria_bill(base_price, *items, tax_rate=0.05, discount=0.0, delivery_fee=0.0)` that calculates the final bill.

1. `base_price` (float): The cost of the main combo meal.
2. `*items` (floats): A variable-length positional argument list representing prices of additional side items.
3. `tax_rate` (float): The tax percentage (default `0.05` for 5% tax). This **must** be a keyword-only parameter.
4. `discount` (float): A percentage value (e.g., `10.0` represents a 10% discount, default `0.0`) applied directly to the subtotal before taxes.
5. `delivery_fee` (float): A flat shipping surcharge added to the final total after taxes (default `0.0`).

Calculation Rules:

1. Sum the `base_price` and all side item prices (`*items`) to compute the raw subtotal.
2. Deduct the discount from the raw subtotal to compute the discounted subtotal:

$$\text{Discounted Subtotal} = \text{Raw Subtotal} \times \left(1 - \frac{\text{discount}}{100}\right)$$

1. Compute the tax value by multiplying the discounted subtotal by `tax_rate`.
2. Add the tax and `delivery_fee` to the discounted subtotal to get the final bill.
3. Return the final total rounded to **2 decimal places**.

Example Walkthrough

```
## 1. Standard meal, no sides, default tax, no discount, no delivery
total1 = calculate_cafeteria_bill(100.0)
## Subtotal = 100.0
## Tax = 100.0 * 0.05 = 5.0
## Return: 105.00

## 2. Meal with sides, custom tax rate, 10% discount, flat delivery fee
total2 = calculate_cafeteria_bill(100.0, 20.0, 30.0, tax_rate=0.08, discount=10.0)
## Raw Subtotal = 100.0 + (20.0 + 30.0) = 150.0
## Discounted Subtotal = 150.0 * (1 - 10/100) = 135.0
## Tax = 135.0 * 0.08 = 10.8
## Final Total = 135.0 + 10.8 + 15.0 = 160.8
## Return: 160.80
```

Assignment 2: Academic Email Validator

Scenario

The Academy academic portal needs to validate user registration submissions so that only valid academic emails ending in `.edu` or `.res.in` are registered.

Problem Description

Write a function `validate_academic_email(email)` that checks if a string is a valid academic email address using a regular expression.

1. The email must satisfy the following syntax rules:

- **Username:** Must consist only of lowercase letters, numbers, dots, and underscores (`a-z` , `0-9` , `.` , `_`). It must contain at least one character.
- **Separator:** Must contain exactly one `@` symbol.
- **Domain:** Must consist of lowercase letters, numbers, dots, and hyphens (`a-z` , `0-9` , `.` , `-`).
- **Suffix:** The domain must end with either `.edu` or `.res.in` (and nothing else).

1. The regular expression must perform an exact match of the entire string (use boundary markers `^` and `$`).

2. The function must return `True` if the email matches all criteria, and `False` otherwise.

Example Walkthrough

```
print(validate_academic_email("arham.khan@domain.res.in")) # Output: True
print(validate_academic_email("lisa_stud12@mit.edu"))      # Output: True
print(validate_academic_email("vinod@gmail.com"))          # Output: False (inva
print(validate_academic_email("ALICE@college.edu"))        # Output: False (cont
print(validate_academic_email("bob@mit.edu.com"))           # Output: False (does
```

Medium Assignments

Assignment 3: Corporate Directory Search & Scraper

Scenario

You are writing a parser to extract formatted employee phone records from unstructured text files. Employee phone numbers are formatted in multiple ways across the directory.

Problem Description

Write a function `scrape_directory_phones(directory_text)` that extracts phone records from text and returns a structured list of dictionaries.

1. The function must detect phone numbers matching any of the following three formats:

- `AAA-PPP-LLLL` (e.g., `123-456-7890`)
- `(AAA) PPP-LLLL` (e.g., `(123) 456-7890`)
- `AAAPPPLLLL` (10 consecutive digits, e.g., `1234567890`)

where `AAA` represents the area code (3 digits), `PPP` represents the prefix (3 digits), and `LLLL` represents the line number (4 digits).

1. Design a single compiled RegEx pattern to parse all three formats using **capture groups**.
2. For each match found in `directory_text` , build a dictionary with the following keys:

- `"area_code"` : String containing the extracted 3 area code digits.
- `"prefix"` : String containing the extracted 3 prefix digits.
- `"line_number"` : String containing the extracted 4 line number digits.
- `"formatted"` : A normalized phone string in the format `"(AAA) PPP-LLLL"`.

1. Return a list of these dictionaries. If no phone numbers are found, return an empty list.

Sample Input

```
directory = "Contact HR at 123-456-7890 or the helpdesk at (987) 654-3210. Direc
```

Expected Output

```
[
  {"area_code": "123", "prefix": "456", "line_number": "7890", "formatted": "(123) 456-7890"},
  {"area_code": "987", "prefix": "654", "line_number": "3210", "formatted": "(987) 654-3210"},
  {"area_code": "555", "prefix": "888", "line_number": "1234", "formatted": "(555) 888-1234"}
]
```

Assignment 4: Dynamic Data Pipeline with Lambdas & Custom Sorting

Scenario

An AI classification pipeline processes raw data inputs. Each raw input is a tuple of string annotations describing a product name, its price, and rating. The pipeline needs to clean, filter, and sort these records.

Problem Description

Write a function `process_dataset(dataset)` that processes a dataset using built-in higher-order functions (`map`, `filter`) and `lambda` expressions:

- `dataset` is a list of tuples containing string records. Example:


```
[("Laptop", "Price: 1200", "Rating: 4.8"), ("Phone", "Price: 800", "Rating:
```

- Your pipeline must execute the following sequential steps:
 1. **Parsing:** From the incoming raw tuples, extract the product name (string), numeric price (float), and rating (float). (You can use string splitting or RegEx to isolate the numeric values).
 2. **Filtering:** Use `filter()` with a **lambda** function to keep only items with a parsed price less than or equal to `1000.0`.
 3. **Mapping:** Use `map()` with a **lambda** function to transform the filtered entries into dictionaries of the following structure:

```
{"product": <name>, "price": <float_price>, "score": <float_rating>}
```

1. **Sorting:** Sort the resulting list of dictionaries in **descending order of their score** using `sorted()` with a **lambda** key selector. If two items have the same score, their relative order does not matter.
- The function should return the sorted list of dictionaries.

Sample Input

```
data_input = [  
    ("Laptop", "Price: 1200", "Rating: 4.8"),  
    ("Phone", "Price: 800", "Rating: 4.5"),  
    ("Mouse", "Price: 25", "Rating: 4.7"),  
    ("Charger", "Price: 15", "Rating: 4.2")  
]
```

Expected Output

```
[  
    {"product": "Mouse", "price": 25.0, "score": 4.7},  
    {"product": "Phone", "price": 800.0, "score": 4.5},  
    {"product": "Charger", "price": 15.0, "score": 4.2}  
]
```

(Note: "Laptop" is excluded since its price of 1200 exceeds 1000.0).

Difficult Assignments

Assignment 5: Stateful Ledger Scope Machine (LEGB Scopes & Closures)

Scenario

You are developing a stateful balance ledger tracker that tracks account state history. To satisfy strict architecture requirements, you must manage this state without defining any classes (`class` keyword is prohibited). Instead, you must use closures, nested functions, and Python scoping variables.

Problem Description

1. Define a global variable `AUDIT_TRANSACTION_COUNT = 0` at the top level of your script.
2. Implement a function `create_bank_account(owner_name, initial_balance)` that returns a dictionary of actions.
 - Inside `create_bank_account`, initialize local variables `balance` (float, set to `initial_balance`) and `history` (list of strings, initially containing `["Account created with 1000.0"]` or similar initial message).
 - Define three nested functions inside `create_bank_account`:
 - `deposit(amount)`:
 - Adds `amount` to the local `balance` variable.
 - Appends the string `"deposit <amount>"` to the local `history` list.
 - Increments the global `AUDIT_TRANSACTION_COUNT` by `1` using the `global` keyword.
 - `withdraw(amount)`:
 - Checks if the current local `balance` is sufficient (balance $\geq$ amount).
 - If yes, deducts `amount` from `balance`, appends the string `"withdraw <amount>"` to `history`, and increments global `AUDIT_TRANSACTION_COUNT` by `1`.
 - If the balance is insufficient, raises a standard `ValueError` with message `"Insufficient balance"`.

- `get_statement()` :
- Returns a tuple containing `(owner_name, current_balance, history_list_copy)` . (Note: Make sure `history_list_copy` is a copy of the history list to prevent direct external modification).
- Return a dictionary containing key-value mappings to these inner functions:

```
return {  
    "deposit": deposit,  
    "withdraw": withdraw,  
    "statement": get_statement  
}
```

1. **Constraint:** You must utilize the `nonlocal` keyword to modify the variables `balance` and `history` inside the nested functions.

Example Walkthrough

```

## Initial State
print(AUDIT_TRANSACTION_COUNT) # Output: 0

## Create account
acc = create_bank_account("Arham", 1000.0)

## Deposit
acc["deposit"](200.0)

## Withdraw
acc["withdraw"](150.0)

## Overdraft attempt (should raise ValueError)
try:
    acc["withdraw"](2000.0)
except ValueError as e:
    print(e) # Output: Insufficient balance

## Get statement
owner, bal, txn_history = acc["statement]()
print(owner) # Output: Arham
print(bal) # Output: 1050.0
print(txn_history) # Output: ['Account created with 1000.0', 'deposit 200.0', 'w

## Verify global log count
print(AUDIT_TRANSACTION_COUNT) # Output: 2

```

Assignment 6: Server Log Analyzer & Traffic Classifier (Advanced RegEx)

Scenario

An automated server monitor analyzes web traffic logs to detect security issues. The monitor extracts HTTP details from log strings and filters out requests originating from local network IP addresses.

Problem Description

Write a function `analyze_server_logs(logs_text)` that parses web logs:

- `logs_text` is a multi-line string containing server logs. Each log line matches this exact format:

```
"<IP> - - [<timestamp>] \"<HTTP_METHOD> <URL> <HTTP_VERSION>\" <STATUS_CODE>
<BYTES>" Example: "192.168.1.5 - - [28/Aug/2026:10:00:00] \"GET /index.html
HTTP/1.1\" 200 1024"
```

- The function must perform the following:

1. Compile a single regular expression using **named capture groups** to extract:

- `ip` : The source IP address.
- `time` : The timestamp value inside the brackets `[]`.
- `method` : The HTTP method (GET, POST, PUT, DELETE).
- `resource` : The URL route value (e.g., `/index.html`).
- `status` : The integer status code.
- `bytes` : The integer bytes sent.

1. Parse the input `logs_text` line-by-line using your regex. If a line does not match the format, print a warning: `"Warning: Could not parse line: '<line>'. Skipping."` and continue to the next line.

2. Local IP Address Filtering:

- Check the extracted IP address. If the IP starts with `"192.168."` or `"10."`, it is classified as a local network request.
- Filter out and ignore local network requests; do not include them in the final list.

1. For external request logs, compile a dictionary containing:

```
{"ip": ip, "time": time, "method": method, "resource": resource, "status":
status_code, "bytes": bytes_sent} (Note: status and bytes must be stored as integers).
```

1. Return a list of these dictionaries.

Sample Input

```
log_data = ""192.168.1.5 - - [28/Aug/2026:10:00:00] "GET /index.html HTTP/1.1"
8.8.8.8 - - [28/Aug/2026:10:10:00] "GET /api/v1/users HTTP/1.1" 200 4096
Corrupted log entry here
10.0.0.12 - - [28/Aug/2026:10:15:00] "POST /submit_data HTTP/1.1" 403 512
172.16.0.4 - - [28/Aug/2026:10:20:00] "POST /login HTTP/1.1" 401 256""
```

Expected Output

Console Warnings Printed:

```
Warning: Could not parse line: 'Corrupted log entry here'. Skipping.
```

Returned List:

```
[
  {
    "ip": "8.8.8.8",
    "time": "28/Aug/2026:10:10:00",
    "method": "GET",
    "resource": "/api/v1/users",
    "status": 200,
    "bytes": 4096
  },
  {
    "ip": "172.16.0.4",
    "time": "28/Aug/2026:10:20:00",
    "method": "POST",
    "resource": "/login",
    "status": 401,
    "bytes": 256
  }
]
```

(Note: IP 192.168.1.5 and 10.0.0.12 are skipped because they are local).

Chapter 6: Object-Oriented Programming (OOP) in Python

Welcome to Chapter 6! Today we explore **Object-Oriented Programming (OOP)**, a programming paradigm that structures code using classes and objects. We will cover:

1. **Core Concepts:** Classes, Objects, Instantiation, and the `self` parameter.
 2. **Attributes & Scopes:** Instance vs. Class variables, and references.
 3. **OOP Decorators:** `@classmethod`, `@staticmethod`, and `@property`.
 4. **Inheritance & MRO:** Single/Multiple inheritance, and cooperative super calls.
 5. **Polymorphism:** Method overriding and overloading behavior.
 6. **Encapsulation:** Private/Protected naming conventions and name mangling.
 7. **Special Dunder Methods:** Representation (`__str__`, `__repr__`), Operator Overloading (`__add__`, `__eq__`), and Custom Iterators (`__iter__`, `__next__`).
-

Section 1: Object-Oriented Foundations (Classes & Objects)

Classes, Objects, and Instantiation

- **Class:** A user-defined blueprint or template for creating objects.
- **Object:** An instance of a class containing real values and executable behaviors.
- **Instantiation:** The process of allocating memory and initializing a new object.

```

class Student:
    # Constructor/Initializer method
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age    # Instance attribute

    # Instance method
    def display_details(self):
        return f"Student: {self.name}, Age: {self.age}"

# Instantiation
student_1 = Student("Arham", 21)
print(student_1.display_details()) # Output: Student: Arham, Age: 21

```

The `self` Parameter

In Python, `self` represents the specific instance of the class that is currently invoking the method.

- You must include `self` as the first parameter in all instance methods.
- When you call the method as `obj.method()`, Python automatically passes the object reference as the first argument (`self`).

Instance Variables vs. Class Variables

- **Instance Variables:** Defined inside methods (usually `__init__`) prefixed with `self.`. They belong to a specific object instance.
- **Class Variables:** Defined directly inside the class body but outside any methods. They are shared across all instances of the class.


```

class PythonStudent:
    course = "PGCP-AI" # Class Variable (shared by all)

    def __init__(self, name):
        self.name = name # Instance Variable (unique to each)

s1 = PythonStudent("Arham")
s2 = PythonStudent("Lisa")

print(s1.name, "| Course:", s1.course) # Arham | Course: PGCP-AI
print(s2.name, "| Course:", s2.course) # Lisa | Course: PGCP-AI

```

OOP Decorators (@classmethod, @staticmethod, @property)

Python provides built-in decorators to modify class method behavior.

Class Methods (@classmethod)

- Receives the class (`cls`) as the first parameter instead of `self`.
- Can modify class state that applies to all instances.
- Often used to define "factory methods" (alternative constructors).

```

class DateConverter:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, date_str):
        # Parses "YYYY-MM-DD" and creates a new object
        parts = list(map(int, date_str.split("-")))
        return cls(parts[0], parts[1], parts[2])

# Use the factory classmethod to instantiate
date_obj = DateConverter.from_string("2026-08-28")
print(date_obj.year) # Output: 2026

```

Static Methods (@staticmethod)

- Does not receive `self` or `cls` parameters.
- Behaves exactly like a standard function, but resides inside the class namespace.
- Used for helper or utility functions that don't need to access or modify class/instance state.

```
class MathUtility:
    @staticmethod
    def is_even(num):
        return num % 2 == 0

print(MathUtility.is_even(10))  # Output: True
```

Properties (`@property`)

- Converts a method call into a read-only attribute getter.
- Combined with `.setter` and `.deleter` decorators, properties allow you to enforce validations on attribute updates.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    @property
    def balance(self):
        """Getter property."""
        return self.__balance

    @balance.setter
    def balance(self, new_val):
        """Setter property with validation."""
        if new_val < 0:
            raise ValueError("Balance cannot be negative.")
        self.__balance = new_val

acc = Account(100.0)
print(acc.balance)  # Output: 100.0 (called without parenthesis)
acc.balance = 150.0 # Invokes the setter
# acc.balance = -50.0 # Raises ValueError
```

Section 2: Advanced OOP (Inheritance, MRO & Polymorphism)

Single Inheritance & super() Calls

A child class inherits attributes and methods from a single parent class. Use `super()` to invoke parent methods.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name) # Initialize parent class attributes
        self.emp_id = emp_id
```

Multiple Inheritance & cooperative MRO

A class can inherit from multiple parent classes.

- **Method Resolution Order (MRO):** The order in which Python searches for a method or attribute in a class hierarchy.
- You can inspect this order using the `.__mro__` attribute or `.mro()` method.

```
class A:
    def process(self):
        print("Process A")

class B(A):
    def process(self):
        print("Process B")
        super().process()

class C(A):
    def process(self):
        print("Process C")
        super().process()

class D(B, C):
    def process(self):
        print("Process D")
        super().process()

d = D()
d.process()
# Output order shows the cooperative MRO resolution:
# Process D -> Process B -> Process C -> Process A

print(D.__mro__)
# Output: (, , , , )
```

Method Overriding & Overloading

Polymorphism allows different classes to define methods with the same name.

Method Overriding

A subclass provides a specific implementation of a method that is already defined by its parent class.

```

class Animal:
    def make_sound(self):
        return "Generic Sound"

class Dog(Animal):
    def make_sound(self):
        return "Woof"

class Cat(Animal):
    def make_sound(self):
        return "Meow"

animals = [Dog(), Cat()]
for animal in animals:
    print(animal.make_sound()) # Woof, then Meow

```

Method Overloading

Unlike Java or C++, Python does not support standard method overloading (defining multiple methods with the same name but different signatures). In Python, the last method definition overrides all previous ones.

To implement overloading behavior, use default parameters or variable arguments (`*args`):

```

class Calculator:
    def add(self, a, b, c=None):
        if c is not None:
            return a + b + c
        return a + b

calc = Calculator()
print(calc.add(2, 3)) # Output: 5
print(calc.add(2, 3, 5)) # Output: 10

```

Encapsulation & Private Data Mangling

Encapsulation restricts direct access to some of an object's components.

- **Public:** Accessible from anywhere (default). E.g., `self.name`.
- **Protected:** A convention indicating the variable should not be accessed outside the class. Prefixed with a single underscore. E.g., `self._name`.
- **Private:** Restricts direct access. Prefixed with double underscores. E.g., `self.__name`.
- **Name Mangling:** Python replaces double-underscore variable names under the hood with `_ClassName__variable_name` to prevent external access.

```
class SecureDevice:
    def __init__(self, key):
        self.__secret_key = key # Private attribute

device = SecureDevice("12345")

# Trying to access directly raises AttributeError
try:
    print(device.__secret_key)
except AttributeError:
    print("Cannot access private variable __secret_key")

# Accessing via name mangling (Discouraged, but possible)
print("Mangling access:", device._SecureDevice__secret_key) # Output: 12345
```

Section 3: Special Dunder Protocols & Operator Overloading

Special methods are prefixed and suffixed with double underscores (`__`). They allow objects to integrate with Python built-in behaviors.

String Representation (`__str__` , `__repr__`)

- `__str__`: Returns a user-friendly string representation of the object (called by `print()` or `str()`).
- `__repr__`: Returns an unambiguous, developer-friendly string representation (called by `repr()` or in interactive shells).

```

class Coordinates:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __repr__(self):
        return f"Coordinates(x={self.x}, y={self.y})"

pt = Coordinates(3, 4)
print(str(pt))    # Output: (3, 4)
print(repr(pt))   # Output: Coordinates(x=3, y=4)

```

Operator Overloading (`__add__` , `__eq__`)

You can define custom behavior for mathematical and comparison operators:

```

class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        """Overloads the + operator."""
        if not isinstance(other, Money):
            raise TypeError("Can only add Money objects.")
        return Money(self.amount + other.amount)

    def __eq__(self, other):
        """Overloads the == operator."""
        if not isinstance(other, Money):
            return False
        return self.amount == other.amount

m1 = Money(10)
m2 = Money(20)
m3 = m1 + m2
print(m3.amount)    # Output: 30
print(m1 == Money(10)) # Output: True

```

Custom Iterators (`__iter__` , `__next__`)

An object can be made iterable by implementing the iterator protocol:

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

for num in Countdown(3):
    print(num)
# Output:
# 3
# 2
# 1
```

Practice Assignments

Objective

Apply inheritance, encapsulation, polymorphism, and decorators to structure class-based designs. Implement multiple inheritance, properties, operator overloading, and custom iterators.

Easy Assignments

Assignment 1: Smart Home Appliance Control

Scenario

You are designing a control model for a smart home thermostat. The target temperature must be guarded against invalid bounds (e.g., set too high or too low, causing damage or excessive energy usage).

Problem Description

Create a class named `SmartThermostat` that implements the following specifications:

1. Class-level Constant Variables:

- `MIN_TEMP = 10.0` (float)
- `MAX_TEMP = 35.0` (float)

1. Constructor (`__init__`):

- Accepts parameters: `appliance_name` (string) and `initial_temp` (float).
- Sets a private attribute `__appliance_name` (assigned from `appliance_name`).
- Sets a private attribute `__target_temp` (float). Call the setter property inside the constructor or perform checks to ensure that if the `initial_temp` is out of the `[MIN_TEMP, MAX_TEMP]` bounds, it defaults to `22.0` .

1. Properties:

- `target_temp` (read-write property):
- **Getter:** Returns the value of `__target_temp` .
- **Setter:** Checks if the new temperature is within the range `[MIN_TEMP, MAX_TEMP]` inclusive. If valid, updates `__target_temp` . If invalid, raises a `ValueError` with message: `"Temperature must be between 10.0 and 35.0 degrees."`
- `appliance_name` (read-only property):
- **Getter:** Returns `__appliance_name` .
- (No setter defined, making it read-only after creation).

Example Walkthrough

```

thermostat = SmartThermostat("Living Room AC", 24.0)
print(thermostat.appliance_name) # Output: Living Room AC
print(thermostat.target_temp)    # Output: 24.0

thermostat.target_temp = 28.0    # Updates successfully
print(thermostat.target_temp)    # Output: 28.0

try:
    thermostat.target_temp = 5.0 # Out of range!
except ValueError as e:
    print(e) # Output: Temperature must be between 10.0 and 35.0 degrees.

```

Assignment 2: Vehicle Fleet Management

Scenario

A delivery company manages different vehicle classes. They want to calculate travel range based on fuel capacity, and adjust range for trucks depending on cargo load.

Problem Description

1. Create a base class named `Vehicle` with:

- **Constructor (`__init__`):** Accepts `make` (string), `model` (string), and `fuel_capacity` (float, in liters).
- **Method `calculate_range(fuel_efficiency)`:** Calculates and returns the vehicle's range (in kilometers) by multiplying the `fuel_capacity` by the `fuel_efficiency` (km per liter).
- **Method `get_description()`:** Returns a formatted string: `"Vehicle: <make> <model>"`.

1. Create a subclass named `DeliveryTruck` that inherits from `Vehicle`:

- **Constructor (`__init__`):** Accepts `make` (string), `model` (string), `fuel_capacity` (float), and `cargo_load` (float, in metric tons). Uses `super().__init__()` to initialize base vehicle parameters.
- **Method `calculate_range(fuel_efficiency)`:** Overrides the base method. Heavy loads reduce efficiency. Reduce the range calculation by **10% for every metric ton** of

`cargo_load` currently carried.

$$\\text{Adjusted Range} = \\text{Base Range} \\times (1.0 - 0.1 \\times \\text{cargo_load})$$

- **Method `get_description()`** : Overrides the base method. Returns a formatted string:

```
"Truck: <make> <model> carrying <cargo_load> tons".
```

Example Walkthrough

```
truck = DeliveryTruck("Volvo", "FH16", 300.0, cargo_load=2.0)

## Base range calculations without load adjustment would be 300 * 5 = 1500 km.
## 2.0 tons load reduces range by 20% (10% * 2) -> 1500 * 0.8 = 1200 km.
print(truck.calculate_range(5.0)) # Output: 1200.0
print(truck.get_description())    # Output: Truck: Volvo FH16 carrying 2.0 tons
```

Medium Assignments

Assignment 3: E-Commerce Currency Converter

Scenario

An online store represents transaction totals as structured objects containing currency labels. To prevent logic errors, the system must not add different currencies directly and should print descriptive labels.

Problem Description

Create a class named `PriceAmount` with the following requirements:

1. **Constructor (`__init__`)**: Accepts `value` (float) and `currency` (string). Standardize the `currency` string value by converting it to uppercase.
2. **Dunder Methods for String Representation**:
 - `__str__` : Returns a string formatted as "`<currency> <value>`" with the value rounded to **2 decimal places** (e.g., "`USD 19.99`").

- `__repr__`: Returns a detailed programmer representation: `"PriceAmount (value=<value>, currency='<currency>')"` (value rounded to 2 decimal places).

1. Operator Overloading:

- `__add__(self, other)`:
- Check if `other` is an instance of `PriceAmount` and has the **same** currency value.
- If the currency values do not match, raise a `ValueError` with the message:

```
"Cannot add price amounts with different currencies: '<currency1>' and '<currency2>'."
```

- If valid, return a **new** `PriceAmount` instance with the summed value and the same currency.
- `__eq__(self, other)`:
- Returns `True` if `other` is an instance of `PriceAmount`, has the same currency, and the values are identical. Otherwise, returns `False`.

Example Walkthrough

```
p1 = PriceAmount(19.99, "usd")
p2 = PriceAmount(10.01, "USD")
p3 = PriceAmount(15.00, "EUR")

print(str(p1))      # Output: USD 19.99
print(repr(p1))     # Output: PriceAmount(value=19.99, currency='USD')

total = p1 + p2
print(str(total))   # Output: USD 30.00

print(p1 == PriceAmount(19.99, "USD")) # Output: True

try:
    bad_addition = p1 + p3
except ValueError as e:
    print(e) # Output: Cannot add price amounts with different currencies: 'USD'
```

Assignment 4: Hospital Patient Register

Scenario

A hospital patient ledger automatically tracks patient counts and assigns sequentially numbered keys. It also validates input dates to prevent registration crashes.

Problem Description

Create a class named `Patient` that satisfies the following:

1. Class-level Variables:

- `_patient_counter` (integer, initialized to `0`): Tracks the total count of patient instances created.

1. Static Method `validate_dob_format(dob_str)` :

- Uses a Regular Expression pattern to check if the date of birth matches the format `"YYYY-MM-DD"` exactly (4 digits, a hyphen, 2 digits, a hyphen, 2 digits).
- Returns `True` if correct, and `False` otherwise.

1. Constructor (`__init__`):

- Accepts parameters: `name` (string) and `dob` (string, representation of date of birth).
- First, calls `Patient.validate_dob_format(dob)`. If it returns `False`, raise a `ValueError` with the message: `"Invalid date of birth format: '<dob>'. Expected YYYY-MM-DD."`
- If validation passes, increments the class variable `_patient_counter` by `1`.
- Assigns a unique `patient_id` as a string: `"PAT-"` followed by the value of `1000 + _patient_counter` (e.g., `"PAT-1001"`, `"PAT-1002"`).
- Stores `name` and `dob` as instance variables.

1. Class Method `get_total_patients()` :

- Returns the value of `_patient_counter`.

Example Walkthrough

```
## 1. Valid Registration
p1 = Patient("Arham Khan", "1999-05-15")
print(p1.patient_id) # Output: PAT-1001

## 2. Invalid DOB registration (throws ValueError)
try:
    p2 = Patient("Lisa", "12/08/1998")
except ValueError as e:
    print(e) # Output: Invalid date of birth format: '12/08/1998'. Expected YYYY-MM-DD
print(Patient.get_total_patients()) # Output: 1
```

Difficult Assignments

Assignment 5: Multi-Channel Notification System (Multiple Inheritance & MRO)

Scenario

An automated incident response engine sends server health alert broadcasts. Depending on incident severity, it sends notifications via Email, SMS, or both using cooperative multiple inheritance.

Problem Description

Implement a cooperative multiple inheritance structure using the following class designs:

1. Base Class `Notifier` :

- Constructor (`__init__`): Accepts `sender_id` (string).
- Method `send(message)` : Returns a list containing the log: `["[Notifier <sender_id>] general broadcast: <message>"]` .

1. Subclass `EmailNotifier` (inherits from `Notifier`):

- Constructor (`__init__`): Accepts `email_server` (string) along with any other keyword parameters. It must forward parameters to the next class in the hierarchy using `super().__init__()` or direct calls.

- Method `send(message)` : Calls `super().send(message)` to get the log list, prepends the string `"[Email via <email_server>] sending: <message>"` to the list, and returns it.

1. Subclass `SMSNotifier` (inherits from `Notifier`):

- Constructor (`__init__`): Accepts `sms_gateway` (string) along with any other keyword parameters. It must forward parameters to the next class in the MRO.
- Method `send(message)` : Calls `super().send(message)` to get the log list, prepends the string `"[SMS via <sms_gateway>] sending: <message>"` to the list, and returns it.

1. Subclass `HybridAlertChannel` (inherits from BOTH `EmailNotifier` and `SMSNotifier` in that order):

- Constructor (`__init__`): Accepts `sender_id` (string), `email_server` (string), and `sms_gateway` (string). Passes all values cooperatively through `super().__init__()`.
- Method `send(message)` : Calls `super().send(message)` to get the consolidated log list. Prepends `"[HYBRID ALERT] Initiating dual channels..."` to the list and returns it.

1. Requirements:

- The hierarchy must support **cooperative initialization and cooperative method dispatch**. Calling `super().__init__()` or `super().send()` must pass details down the entire MRO path without skipping parent classes or duplicating calls.
- Print the Method Resolution Order (`.__mro__` or `.mro()`) of `HybridAlertChannel` to verify the lookup path.

Example Walkthrough

```

alert = HybridAlertChannel(sender_id="SYS-ADMIN", email_server="smtp.domain.com")
logs = alert.send("Disk space 95%")

for log in logs:
    print(log)

```

Expected Console Output Logs:

```
[HYBRID ALERT] Initiating dual channels...
[Email via smtp.domain.com] sending: Disk space 95%
[SMS via gw.domain.com] sending: Disk space 95%
[Notifier SYS-ADMIN] general broadcast: Disk space 95%
```

Assignment 6: Custom Database Record Simulator

Scenario

You are developing a custom result set class to represent database queries. To make it behave like a native Python list, the object must support indexes, search lookups by name strings, iteration using custom iterator classes, and return database size.

Problem Description

1. Create a class named `DatabaseRecord` :

- **Constructor (`__init__`):** Accepts `record_id` (int) and `data` (dictionary).
- **Dunder Methods:** Implement `__repr__` and `__str__` returning `"Record(id=<record_id>, data=<data>)"` .

1. Create a custom iterator class named `ResultSetIterator` :

- **Constructor (`__init__`):** Accepts `records_list` (list of `DatabaseRecord` instances). Initialize an index counter to `0` .
- **Dunder Methods:**
 - `__iter__(self)` : Returns `self` .
 - `__next__(self)` : Yields the next `DatabaseRecord` in the list. If no records remain, raises `StopIteration` .

1. Create a class named `DatabaseResultSet` :

- **Constructor (`__init__`):** Accepts `records_list` (list of `DatabaseRecord` objects).
- **Dunder Methods:**
 - `__len__(self)` : Returns the count of records in the result set.

- `__iter__(self)` : Returns a new `ResultSetIterator` object initialized with `records_list`.
- `__getitem__(self, key)` :
- If `key` is an **integer**, return the `DatabaseRecord` at that index. If the index is out of bounds, let it raise a standard `IndexError`.
- If `key` is a **string**, search the database records. Return the first `DatabaseRecord` object whose `data["name"]` matches the key string.
- If the name string is not found, raise a custom exception named `RecordNotFoundError` (which you must define) with the message: `"Record with name '<name>' not found in database."`

Example Walkthrough

```

## Setup records
r1 = DatabaseRecord(101, {"name": "Alice", "role": "Admin"})
r2 = DatabaseRecord(102, {"name": "Bob", "role": "User"})

results = DatabaseResultSet([r1, r2])

## 1. Length
print(len(results)) # Output: 2

## 2. Integer Indexing
print(results[0].data["role"]) # Output: Admin

## 3. String lookup
record = results["Bob"]
print(record.record_id) # Output: 102

## 4. Iteration
for rec in results:
    print(rec.record_id)
## Output:
## 101
## 102

## 5. Missing key lookup
try:
    missing = results["Charlie"]
except RecordNotFoundError as e:
    print(e) # Output: Record with name 'Charlie' not found in database.

```

Chapter 7: Libraries, Debugging, Logging & Databases

Welcome to Chapter 7! Today we cover utility components required for production Python development:

1. **Python Standard Libraries:** Working with `os`, `sys`, `pathlib`, `datetime`, and `json`.
2. **Diagnostic Logging:** Configuring logging levels, handlers, and formats to record program telemetry.
3. **Interactive Debugging:** Using the Python Debugger (`pdb`) to trace variable states and execution lines.
4. **Relational Databases & DB-API:** Querying databases using `sqlite3` and utilizing parameterized inputs to prevent SQL injections.

Section 1: Python Standard Libraries

Python is famous for its "batteries included" philosophy, providing a rich set of built-in modules.

1. Modern Paths with `pathlib`

The `pathlib` module offers an object-oriented approach to interacting with filesystem paths across different operating systems.

```

from pathlib import Path

# Create a path object representing the current working directory
current_dir = Path(".")

# Joins paths using the slash operator (/)
target_file = current_dir / "Day_07" / "Assignment.md"

# Checking properties
print("Exists:", target_file.exists())
print("Is File:", target_file.is_file())
print("Filename:", target_file.name)
print("Parent directory:", target_file.parent)

```

2. Dates and Times with `datetime`

The `datetime` module handles date arithmetic and formatting.

```

from datetime import datetime, timedelta

# Current time
now = datetime.now()
print("Now:", now.isoformat())

# Creating specific dates
target_date = datetime(2026, 8, 28)

# Date calculations using timedelta
two_weeks_later = target_date + timedelta(days=14)
print("Two Weeks Later:", two_weeks_later.strftime("%d-%b-%Y")) # Output: 11-Sep

# Parsing strings to datetime (strptime)
parsed_date = datetime.strptime("2026-08-28", "%Y-%m-%d")
print("Parsed Date Month:", parsed_date.month)

```

3. Serialization with `json`

The `json` module translates Python dictionaries and lists to JSON strings (serialization) and back (deserialization).

```
import json

config = {"host": "localhost", "port": 8080, "debug": True}

# Serialize: Python dict -> JSON string
json_str = json.dumps(config, indent=4)
print(json_str)

# Deserialize: JSON string -> Python dict
parsed_config = json.loads(json_str)
print("Parsed port:", parsed_config["port"])
```

Section 2: Diagnostic Logging

Instead of using `print()` statements—which output to standard console channels and are hard to filter—production applications use Python's built-in `logging` module.

1. Logging Levels

Log levels classify events based on severity:

- `DEBUG` : Detailed diagnostic information (primarily for troubleshooting).
- `INFO` : Confirmation that things are working as expected.
- `WARNING` : Indication that something unexpected happened (default threshold).
- `ERROR` : A serious issue that prevented a specific function from executing.
- `CRITICAL` : A severe error indicating the program itself may be unable to continue.

2. Configuring Handlers and Formatters

You can configure log messages to output to the console, write to files, or be formatted with metadata (timestamps, file names, line numbers).

```

import logging

# Create a custom logger
logger = logging.getLogger("Main_App")
logger.setLevel(logging.DEBUG) # Set the lowest threshold level to capture

# Create Handlers
console_handler = logging.StreamHandler()
file_handler = logging.FileHandler("app.log")

# Set thresholds for individual handlers
console_handler.setLevel(logging.INFO)
file_handler.setLevel(logging.WARNING)

# Create Formatters and link them to handlers
formatter = logging.Formatter("[% (asctime)s] [% (name)s] [% (levelname)s] - % (message)s")
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

# Add Handlers to the Logger
logger.addHandler(console_handler)
logger.addHandler(file_handler)

# Logging statements
logger.info("Application starting up...") # Console only
logger.warning("Configuration file is missing.") # Both Console & File
logger.error("Failed to connect to database.") # Both Console & File

```

Section 3: Interactive Debugging (`pdb`)

The **Python Debugger (`pdb`)** provides an interactive debugging environment for Python programs. It allows you to pause execution, inspect variables, and step through code line by line.

1. Setting a Breakpoint

Insert `breakpoint()` (Python 3.7+) or `import pdb; pdb.set_trace()` at the exact line you wish to inspect.

```
def calculate_average(grades):  
    total = sum(grades)  
    # Pause execution here  
    breakpoint()  
    count = len(grades)  
    return total / count  
  
calculate_average([85, 90, 78])
```

2. Common `pdb` Commands

When execution pauses, you will see a command prompt `(Pdb)`. Use these commands:

- **n (next)**: Executes the current line and stops at the next line in the current function.
- **s (step)**: Steps *into* the function called on the current line.
- **c (continue)**: Resumes standard execution until the next breakpoint.
- **p <var> (print)**: Evaluates and prints the value of variable `<var>`.
- **l (list)**: Shows the current line and surrounding lines of source code.
- **w (where)**: Prints the stack trace, showing the nested function calls leading to this line.
- **q (quit)**: Instantly aborts execution and exits the debugger.

Section 4: Relational Databases (`sqlite3`)

Python interacts with SQL database backends using standard drivers compliant with **DB-API 2.0**. For lightweight local storage, Python includes `sqlite3`.

1. Parameterized Queries (Preventing SQL Injections)

CAUTION:

*Never format or concatenate strings directly into an SQL statement (e.g., `f"INSERT INTO users VALUES ('{user}')"`). Doing so exposes your database to **SQL Injection Attacks**. Always use parameterized placeholder queries.*

```
import sqlite3

# 1. Establish connection to local database file
conn = sqlite3.connect("database.db")

# 2. Create a cursor object to execute SQL commands
cursor = conn.cursor()

# 3. Create a table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL,
    email TEXT
)
""")

# 4. Insert data using Parameterized SQL (represented by ? placeholders)
new_user = ("vinod_acts", "vinod@acts.com")
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", new_user)

# 5. Commit transaction changes
conn.commit()

# 6. Retrieve data
cursor.execute("SELECT * FROM users WHERE username = ?", ("vinod_acts",))
rows = cursor.fetchall()
for row in rows:
    print(f"ID: {row[0]}, Name: {row[1]}, Email: {row[2]}")

# 7. Close resources safely
cursor.close()
conn.close()
```


Practical Examples (Interactive & Runnable)

Example 1: JSON File Configuration Loader

Demonstrates `pathlib`, `json`, and exception safety.

```
import json
from pathlib import Path

def load_system_config(filepath):
    config_path = Path(filepath)

    # 1. Check file existence
    if not config_path.exists():
        print(f"Warning: '{filepath}' not found. Loading defaults.")
        return {"env": "prod", "cache": True, "limit": 100}

    # 2. Attempt to parse JSON contents
    try:
        with open(config_path, "r") as file:
            return json.load(file)
    except json.JSONDecodeError as err:
        print(f"Corrupted config JSON: {err}. Reverting to safety parameters.")
        return {"env": "error-state", "cache": False, "limit": 10}

# Write a dummy config file to test
Path("settings.json").write_text('{"env": "development", "cache": false, "limit"

# Load configuration
settings = load_system_config("settings.json")
print("Settings:", settings)
```

Example 2: SQLite database Transaction with Rollback

Demonstrates transaction management with commit and rollback.

```

import sqlite3

def init_bank_db():
    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS accounts (acc_num TEXT PRIMARY KE
    cursor.execute("INSERT OR REPLACE INTO accounts VALUES ('ACC01', 500.0)")
    cursor.execute("INSERT OR REPLACE INTO accounts VALUES ('ACC02', 200.0)")
    conn.commit()
    conn.close()

def transfer_funds(sender, receiver, amount):
    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()

    try:
        # Deduct from sender
        cursor.execute("UPDATE accounts SET balance = balance - ? WHERE acc_num

        # Verify sender balance isn't negative (simulate business logic error)
        cursor.execute("SELECT balance FROM accounts WHERE acc_num = ?", (sender
        sender_balance = cursor.fetchone()[0]
        if sender_balance < 0:
            raise ValueError(f"Insufficient funds in sender account: {sender}.")

        # Add to receiver
        cursor.execute("UPDATE accounts SET balance = balance + ? WHERE acc_num

    except Exception as e:
        # Roll back all database operations executed in the try block
        print(f"Transaction failed: {e}. Rolling back changes.")
        conn.rollback()
    else:
        # Commit changes if all operations succeeded
        print("Transaction successful! Committing balances.")
        conn.commit()
    finally:
        conn.close()

# Run Setup and Execution
init_bank_db()
transfer_funds("ACC01", "ACC02", 150.0) # Succeeded
transfer_funds("ACC01", "ACC02", 1000.0) # Fails and rolls back

```

Practice Assignments

Objective

Utilize standard datetime operations, parse JSON configuration trees safely using `pathlib` checks, configure dual handler loggers, establish and query SQLite databases using parameterized commands, and manage rollback database transactions.

Easy Assignments

Assignment 1: System Date Arithmetic Utility

Scenario

You are writing a cleanup utility for system backups. The script must calculate file retention expiration dates and output warning flags when a backup is nearing its expiration date.

Problem Description

Write a function `calculate_backup_dates(start_date_str, retention_days)` that performs date arithmetic:

1. `start_date_str` is a string representing the backup base date. Format: `"YYYY-MM-DD"`.
2. `retention_days` is an integer representing the retention duration in days.
3. **Validation:** Attempt to parse `start_date_str` into a `datetime.date` object using `datetime.strptime()`. If the string doesn't match the format, catch the `ValueError`, print the warning: `"Invalid date format. Expected YYYY-MM-DD."`, and return `None`.
4. **Calculations:**
 - `expiry_date`: Calculate the date exactly `retention_days` **after** the parsed start date.
 - `warning_date`: Calculate the warning date exactly **3 days before** the calculated `expiry_date`.

1. **Formatting:** Format the calculated dates back to string representations in the exact format

`"DD-Mon-YYYY"` (e.g. `"15-May-2026"`, `"08-Sep-2026"`).

2. **Return:** A dictionary containing:

```
{"expiry_date": <expiry_date_str>, "warning_date": <warning_date_str>}
```

Example Walkthrough

```
## 1. Valid Input
dates = calculate_backup_dates("2026-08-28", 14)
print(dates)
## Output: {'expiry_date': '11-Sep-2026', 'warning_date': '08-Sep-2026'}

## 2. Invalid Input
invalid_dates = calculate_backup_dates("28/08/2026", 10)
## Console output: Invalid date format. Expected YYYY-MM-DD.
print(invalid_dates) # Output: None
```

Assignment 2: JSON Configuration Validator

Scenario

An AI web engine boots configurations from a JSON file. If the file is missing, empty, or contains corrupt syntax, the system crashes. You need to write a safe configuration loading module that provides default values in case of errors.

Problem Description

Write a function `load_config_safely(file_path_str)` :

1. Convert `file_path_str` to a path using `pathlib.Path` .
2. **Check Existence:** Check if the file exists and is indeed a file. If the file is missing, print:
`"Error: Config file not found."` and return the default fallback configuration:

```
{"status": "default", "port": 8080}
```

1. **Check Size:** Check if the file size is 0 bytes (empty). If empty, print: `"Error: Config file is empty."` and return the default fallback dictionary.
 2. **JSON Parsing:** Read the file contents. Attempt to deserialize the contents as JSON using `json.loads()` or `json.load()`.
- If the file contains invalid JSON structures (throws `json.JSONDecodeError`), catch the exception, print: `"Error: Invalid JSON syntax."`, and return the default fallback dictionary.
1. If the file parses successfully, return the loaded configuration dictionary.

Example Walkthrough

```
from pathlib import Path

## Setup files for testing
Path("corrupt.json").write_text("invalid data string")
Path("empty.json").write_text("")

## Test Cases
print(load_config_safely("missing.json"))
## Output: Error: Config file not found.
## Returns: {"status": "default", "port": 8080}

print(load_config_safely("empty.json"))
## Output: Error: Config file is empty.
## Returns: {"status": "default", "port": 8080}

print(load_config_safely("corrupt.json"))
## Output: Error: Invalid JSON syntax.
## Returns: {"status": "default", "port": 8080}
```

Medium Assignments

Assignment 3: Corporate Event Logger

Scenario

You are developing a secure authentication firewall. The login gateway must log authorization attempts. File logs must track serious threats with detailed timestamps, while console channels show warnings in real time.

Problem Description

1. Write a function `configure_system_logger(log_file_path) :`
 - Configure a logger named `"Security_Logger"` and set its baseline capture level to `logging.DEBUG`.
 - Clear any existing handlers on the logger to prevent duplicate outputs.
 - Create two log destination handlers:
 - **File Handler (`logging.FileHandler`):** Writes logs to `log_file_path`. Set its logging threshold level to `logging.WARNING`.
 - **Console Handler (`logging.StreamHandler`):** Outputs logs to standard console. Set its logging threshold level to `logging.INFO`.
 - Create and link formatters for both handlers:
 - The File Handler log format must be: `"[% (asctime)s] [% (levelname)s] - %(message)s"`
 - The Console Handler log format must be: `"[CONSOLE] %(levelname)s: %(message)s"`
 - Add both handlers to the `"Security_Logger"` logger and return it.
1. Write a function `process_login_attempt(logger, username, is_success, ip_address) :`
 - If `is_success` is `True`, write an `INFO` message: `"User '<username>' successfully logged in from IP <ip_address>."`
 - If `is_success` is `False`:
 - If the username is `"admin"`, this indicates a critical threat. Write an `ERROR` message: `"CRITICAL: Unauthorized admin access attempt from IP <ip_address>!"`
 - For any other username, write a `WARNING` message: `"Failed login attempt for user '<username>' from IP <ip_address>."`

Expected Output

- Calling `process_login_attempt(logger, "arham", True, "192.168.1.100")` prints to console:

[CONSOLE] INFO: User 'arham' successfully logged in from IP 192.168.1.100. (Not written to file).

- Calling `process_login_attempt(logger, "lisa", False, "10.0.0.5")` prints to console:

[CONSOLE] WARNING: Failed login attempt for user 'lisa' from IP 10.0.0.5. And writes to log file: [`<timestamp>`] [WARNING] - Failed login attempt for user 'lisa' from IP 10.0.0.5.

- Calling `process_login_attempt(logger, "admin", False, "8.8.8.8")` prints to console:

[CONSOLE] ERROR: CRITICAL: Unauthorized admin access attempt from IP 8.8.8.8! And writes to log file: [`<timestamp>`] [ERROR] - CRITICAL: Unauthorized admin access attempt from IP 8.8.8.8!

Assignment 4: Relational Employee Database CRUD Registry

Scenario

You are developing a local employee database registry tool for HR. The application needs CRUD (Create, Read, Update, Delete) methods and must use parameterized parameters to prevent SQL injection.

Problem Description

Create a class named `EmployeeDBRegistry` that connects to a local SQLite database:

1. Constructor (`__init__`):

- Accepts a database file name (string, e.g. `"hr.db"`).
- Establishes a connection to the database.
- Creates a table named `employees` if it does not exist, with the following columns:
- `emp_id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `name` (TEXT NOT NULL)
- `department` (TEXT)

- `salary` (REAL)

1. Methods:

- `add_employee(name, department, salary) :`
 - Inserts a new employee record using parameterized SQL execution (using `?` placeholders).
 - Commits the transaction.
 - Returns the newly created auto-incremented `emp_id` (using `cursor.lastrowid`).
 - `get_employees_by_department(department) :`
 - Queries the database for all records matching `department`.
 - Returns a list of tuples containing all columns of the matching employees.
 - `update_salary(emp_id, new_salary) :`
 - Updates the salary column of the record matching `emp_id`.
 - Commits the change.
 - Returns `True` if a record was modified, and `False` if no employee matching `emp_id` was found in the database.
 - `delete_employee(emp_id) :`
 - Deletes the record matching `emp_id`.
 - Commits the change.
1. Ensure you close cursors and database connections safely.

Example Walkthrough


```

db = EmployeeDBRegistry("hr.db")

## 1. Add employees
id1 = db.add_employee("Alice", "Engineering", 75000.0)
id2 = db.add_employee("Bob", "HR", 50000.0)

## 2. Query department
eng_staff = db.get_employees_by_department("Engineering")
print(eng_staff) # Output: [(1, 'Alice', 'Engineering', 75000.0)]

## 3. Update salary
success = db.update_salary(id1, 80000.0)
print(success) # Output: True

## 4. Attempt update on non-existent ID
print(db.update_salary(999, 100.0)) # Output: False

```

Difficult Assignments

Assignment 5: SQLite Contact Synchronizer with JSON Transaction Log

Scenario

You are writing a database synchronization daemon that reconciles client contact profiles with a central server. The daemon receives contact changes as a JSON formatted transaction log. The changes must be processed atomically: if *any* single transaction in the batch fails, the entire batch must roll back, and a rollback log must be written.

Problem Description

Create a database named `contacts.db` with a table named `contacts` (Schema: `name TEXT PRIMARY KEY, phone TEXT, email TEXT`). Write a function `sync_contacts_batch(db_name, json_patch_str, log_path)`:

1. Parse `json_patch_str` as a JSON list. If parsing fails, raise a `ValueError` with the message `"Invalid patch JSON"`.
2. Connect to the database `db_name` and start a transaction.

3. Process each dictionary entry in the parsed JSON patch list. An entry has an `"action"` key which can be `"insert"`, `"update"`, or `"delete"`.

- `"insert"` :
 - Insert a new contact using `name`, `phone`, and `email`.
 - If the contact `name` already exists in the database (violating the PRIMARY KEY constraint and raising `sqlite3.IntegrityError`), catch it and instead update the existing contact's phone and email (UPSERT behavior).
- `"update"` :
 - Update the `phone` and `email` for the matching `name`.
 - If the `name` does not exist in the database, raise a custom exception `ContactNotFoundError` with the message `"Contact <name> not found for update"`.
- `"delete"` :
 - Delete the record matching `name`.
 - If the `name` does not exist in the database, raise a custom exception `ContactNotFoundError` with the message `"Contact <name> not found for deletion"`.

1. Atomicity & Logging:

- Wrap the loop in a `try` block. If any exception occurs (including `ContactNotFoundError` or standard SQLite errors):
- Roll back the database transaction using `conn.rollback()` to prevent partial changes.
- Open the file at `log_path` and append a log entry:

```
[SYNC FAILED] Batch aborted: <Exception Message>\n
```

- Re-raise the exception to notify the caller.
- If all operations execute successfully:
- Commit the transaction using `conn.commit()`.
- Open the file at `log_path` and append a log entry:

```
[SYNC SUCCESS] Batch processed. <N> changes synchronized.\n
```

 (where `N` is the number of items in the JSON patch).

- Return `True`.

Example Walkthrough

```
## Database has initial entries: [("Alice", "111", "alice@abc.com"), ("Bob", "22

## Valid Patch (UPSERT and Update)
valid_patch = """[
    {"action": "insert", "name": "Alice", "phone": "123", "email": "alice@new.co
    {"action": "update", "name": "Bob", "phone": "999", "email": "bob@new.com"}
]"""
sync_contacts_batch("contacts.db", valid_patch, "sync.log")
## database successfully commits changes. sync.log writes "[SYNC SUCCESS]..."

## Invalid Patch (Raises ContactNotFoundError)
invalid_patch = """[
    {"action": "insert", "name": "Charlie", "phone": "444", "email": "charlie@ab
    {"action": "delete", "name": "David"}
]"""
## David does not exist in the database!
try:
    sync_contacts_batch("contacts.db", invalid_patch, "sync.log")
except ContactNotFoundError as e:
    print(e) # Output: Contact David not found for deletion

## Verify Database Atomicity: Charlie must NOT be inserted into contacts databas
## sync.log writes "[SYNC FAILED]..."
```

Assignment 6: File System Space Monitor & Database Auditing Daemon

Scenario

You are writing a system audit utility that scans disk directories, tracks storage space allocation, and logs records to an SQLite database for monitoring.

Problem Description

Write a function `audit_directory_space(directory_path_str, db_name, log_path)` :

1. Directory Scan:

- Check if `directory_path_str` represents a valid directory using `pathlib.Path`. If it does not exist or is not a directory, raise `FileNotFoundError` with message: "Target directory not found."
- Recursively scan all files inside the directory (use `Path.glob("**/*")` or `Path.rglob("**")`).
- Calculate:
 - `file_count` : The total count of file objects found.
 - `total_bytes` : The sum of all file sizes in bytes.
 - `largest_file_name` : The filename (basename string) of the largest file.
 - `largest_file_bytes` : The size of the largest file in bytes.
- (Note: Ignore directory objects during scanning, only count files). If the directory has no files, set the largest filename to `""` and largest file size to `0`.

1. Logging:

- Write a log entry to the log file at `log_path` (level `INFO`):

```
"Audited directory '<dir_path>': <file_count> files, <total_bytes> bytes."
```

1. Database Audit Entry:

- Connect to database `db_name` and create a table `dir_audit` if it does not exist:
- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `scan_time` (TEXT, ISO format timestamp)
- `dir_path` (TEXT)
- `file_count` (INTEGER)
- `total_bytes` (INTEGER)
- `largest_file_name` (TEXT)
- `largest_file_bytes` (INTEGER)
- Insert a new record containing the audit values using a parameterized query (`?` placeholders).
- For `scan_time`, use the current system time in ISO format: `datetime.now().isoformat()`.
- Commit the transaction and close SQLite resources.

Example Walkthrough

```
## Assuming a directory structure:
## my_data/
##   |- file1.txt (50 bytes)
##   |- docs/
##       |- doc1.pdf (500 bytes)

audit_directory_space("my_data", "monitoring.db", "system.log")
## system.log writes: "Audited directory 'my_data': 2 files, 550 bytes."
## database inserts record: ("2026-08-28T...", "my_data", 2, 550, "doc1.pdf", 50
```

Chapter 8: Data Science Foundations, Wrangling & Visualization

Welcome to Chapter 8! Today we begin our data science module, transitioning from general-purpose programming to numerical data processing and tabular data analysis. We will cover:

1. **NumPy Fundamentals:** Array creation, shapes, sizes, and dimensions.
2. **Indexing & Slicing:** 1D and 2D arrays, and filtering using boolean masks.
3. **Vectorization & Linear Algebra:** Mathematical operations, matrix multiplication, and aggregates.
4. **Introduction to SciPy:** Optimization and curve minimization.
5. **Introduction to Pandas:** Series, DataFrames, indexing with `.loc / .iloc`, and filtering.

NOTE:

Jupyter Notebook & Virtual Environment Recommended: For the data science and plotting modules (Chapters 8 and 9), it is highly recommended to run your code inside an isolated virtual environment and use Jupyter Notebooks.

Step A: Setup & Activate Virtual Environment

* macOS/Linux: `python -m venv .venv && source .venv/bin/activate`

* Windows (CMD): `python -m venv .venv && .venv\Scripts\activate.bat`

* Windows (PS): `python -m venv .venv && .venv\Scripts\Activate.ps1`

Step B: Install Packages & Run Jupyter

1. **Install packages:** `pip install jupyter numpy scipy pandas matplotlib seaborn plotly`

2. **Launch Jupyter:** `jupyter notebook` (or open the `.ipynb` file in VS Code selecting the `.venv` kernel).

Section 1: NumPy Fundamentals & Vectorization

NumPy (Numerical Python) is the foundational package for scientific computing in Python. It provides the `ndarray` (N-dimensional array) object, which is a fast, space-efficient container for homogeneous numerical data.

1. Why NumPy?

- **Contiguous Memory:** NumPy arrays are stored in contiguous blocks of memory, allowing vector processors to perform calculations at hardware speeds.
- **No Loops Required:** Operations on NumPy arrays are vectorized, meaning element-wise calculations are performed natively in compiled C code.

2. Creating Arrays

You can initialize NumPy arrays in several ways:

```
import numpy as np

# 1. From a Python List
arr_from_list = np.array([1, 2, 3, 4])

# 2. Array of zeros or ones
zeros = np.zeros((3, 4)) # Shape: 3 rows, 4 columns
ones = np.ones((2, 3))   # Shape: 2 rows, 3 columns

# 3. Sequential ranges
seq_range = np.arange(0, 10, 2) # Output: [0, 2, 4, 6, 8]

# 4. Linearly spaced intervals
linspace_arr = np.linspace(0, 1, 5) # Output: [0. , 0.25, 0.5 , 0.75, 1. ]

# 5. Identity matrix
identity_matrix = np.eye(3) # 3x3 Identity matrix
```

3. Array Properties

Every array has properties describing its structure:

```
matrix = np.array([[10, 20], [30, 40]])

print("Dimensions:", matrix.ndim) # Output: 2
print("Shape:", matrix.shape)     # Output: (2, 2)
print("Total size:", matrix.size) # Output: 4
print("Data type:", matrix.dtype) # Output: int64 (or int32)
```

Indexing, Slicing & Boolean Masking

1. 2D Array Slicing

Slicing in NumPy follows the syntax `array[row_slice, col_slice]`.

```
# 3x3 matrix
mat = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

# Extract first two rows, last two columns
sub_mat = mat[0:2, 1:3]
print(sub_mat)
# Output:
# [[2 3]
#  [5 6]]

# Extract column at index 1
col_1 = mat[:, 1]
print(col_1) # Output: [2, 5, 8]
```

2. Boolean Masking (Filtering)

You can filter arrays without loop conditions by applying comparison operations directly, which returns a boolean index mask.


```
data = np.array([10, 15, 20, 25, 30])

# Create boolean mask
mask = data > 20
print("Mask:", mask) # Output: [False, False, False, True, True]

# Apply mask
filtered_data = data[mask]
print("Filtered:", filtered_data) # Output: [25, 30]
```

Vectorization & Linear Algebra

1. Element-wise Operations

Arithmetic operations on arrays apply element-by-element:

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y) # Output: [5, 7, 9]
print(x * y) # Output: [4, 10, 18] (Element-wise multiplication)
```

2. Matrix Multiplication

For standard linear algebra matrix multiplication, use the `@` operator or `np.dot()` :

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Dot product
C = A @ B
print(C)
# Output:
# [[19 22]
#  [43 50]]
```

3. Aggregate Operations & Axes

You can aggregate values (sum, mean, max) across the entire matrix or along specific axes.

- `axis=0` : Operations perform **column-wise** (down rows).
- `axis=1` : Operations perform **row-wise** (across columns).

```
numbers = np.array([
    [10, 20],
    [30, 40]
])

print("Total Sum:", np.sum(numbers))          # Output: 100
print("Column-wise Sum (axis=0):", np.sum(numbers, axis=0)) # Output: [40, 60]
print("Row-wise Mean (axis=1):", np.mean(numbers, axis=1))  # Output: [15., 35]
print("Index of Max (flat):", np.argmax(numbers))           # Output: 3 (points to 40)
```

Section 2: SciPy Optimization & Statistical Computing

SciPy builds on the NumPy array object to provide algorithms for optimization, integration, interpolation, and statistics.

Example: Optimizing a Mathematical Function

We use `scipy.optimize.minimize` to find the local minimum of a simple 1D quadratic curve:
 $f(x) = x^2 - 4x + 4$.

```

from scipy.optimize import minimize

# Define the target function
def cost_function(x):
    return x**2 - 4*x + 4

# Provide initial guess
initial_guess = 0.0

# Run optimization
result = minimize(cost_function, initial_guess)

print("Optimization Successful:", result.success)
print("Minimum location x:", result.x[0]) # Output: 2.00000...
print("Minimum function value f(x):", result.fun) # Output: 0.0

```

Section 3: Tabular Analysis Foundations (Pandas)

While NumPy is ideal for homogeneous array calculations, **Pandas** is designed for working with heterogeneous (tabular) data (similar to a SQL table or Excel sheet). For this module, we will use the [Northwind_Orders.csv](#) dataset.

1. Loading Data and Inspecting Properties

To load a CSV file, use `pd.read_csv()` .

```

import pandas as pd

# Load the Northwind Orders dataset
df = pd.read_csv("Northwind_Orders.csv")

# Inspect basic properties
print("Shape (rows, columns):", df.shape)
print("\nColumn names:\n", list(df.columns[:10])) # Show first 10 columns
print("\nFirst 3 rows:\n", df.head(3))

```

2. Slicing with `.loc` vs. `.iloc`

- `.loc` : Label-based indexing (uses row index labels and column names).
- `.iloc` : Position-based indexing (uses integer offsets starting from 0).

```
# Slicing with .loc (using column labels)
print("--- Slicing with .loc ---")
print(df.loc[0:2, ["product_name", "quantity_ordered", "total_item_revenue"]])

# Slicing with .iloc (using integer column positions)
# 2: product_name, 7: quantity_ordered, 9: total_item_revenue
print("\n--- Slicing with .iloc ---")
print(df.iloc[0:3, [2, 7, 9]])
```

3. Basic Aggregations & Column Operations

```
# Compute aggregate statistics
average_revenue = df["total_item_revenue"].mean()
total_quantity = df["quantity_ordered"].sum()

print(f"Average Item Revenue: ${average_revenue:.2f}")
print(f"Total Items Ordered: {total_quantity}")
```

Practical Examples (Interactive & Runnable)

Example 1: Matrix Feature Scaling (Normalizer)

Using NumPy vectorization to scale features of an array to normal standard ranges $[0, 1]$.

```

import numpy as np

def min_max_normalize(data_matrix):
    # Minimum and Maximum values of each column (axis=0)
    col_mins = np.min(data_matrix, axis=0)
    col_maxs = np.max(data_matrix, axis=0)

    # Range of each column
    col_ranges = col_maxs - col_mins

    # Scaled matrix: (X - min) / range
    # Broadcasting takes care of dimensions automatically
    scaled_matrix = (data_matrix - col_mins) / col_ranges
    return scaled_matrix

# Run Example
raw_features = np.array([
    [10.0, 100.0],
    [20.0, 300.0],
    [15.0, 200.0]
])

normalized = min_max_normalize(raw_features)
print("Normalized Features:\n", normalized)

```

Example 2: Pandas Data Filtering and Column Transformations

Loads the Northwind Orders dataset, filters for high-value orders shipped to Germany, and extracts relevant sales details.

```
import pandas as pd

# 1. Load the Northwind dataset
df = pd.read_csv("Northwind_Orders.csv")

# 2. Filter: select high-value items (> $2000.0) shipped to Germany
germany_high_value = df[(df["customer_country"] == "Germany") & (df["total_item_

# 3. Slicing: get order ID, product name, customer name, and item revenue
summary = germany_high_value.loc[:, ["order_id", "product_name", "customer_compa

print(f"Total matching items: {len(summary)}")
print("\nFirst 5 matching records:\n", summary.head(5))
```

Welcome to Chapter 9! Today we will expand our Pandas data analytics toolkit and explore data visualization using both static and interactive plotting libraries. We will cover:

1. **Data Wrangling & Cleansing:** Detecting nulls, filling/dropping missing entries, and string manipulation.
2. **Merging & Joining:** Reconciling different relational DataFrames using joins.
3. **Grouping & Aggregations:** Compiling database-like groupings and multi-aggregations.
4. **Pivot Tables:** Reshaping tables to summarize metrics across index-column intersections.
5. **Static Visualization (Matplotlib):** Plotting line graphs, bar charts, and pie charts with axis metadata.
6. **Enhanced Plots (Seaborn):** Styling box plots, scatter plots, and distribution trends.
7. **Interactive Visualization (Plotly):** Generating dynamic line charts, bubble charts, and grouped bar plots that support zooming, hovering, and toggling series.

NOTE:

Jupyter Notebook & Virtual Environment Recommended: For the data science and plotting modules (Chapters 8 and 9), it is highly recommended to run your code inside an isolated virtual environment and use Jupyter Notebooks.

Step A: Setup & Activate Virtual Environment

* macOS/Linux: `python -m venv .venv && source .venv/bin/activate`

* **Windows (CMD):** `python -m venv .venv && .venv\Scripts\activate.bat`

* **Windows (PS):** `python -m venv .venv && .venv\Scripts\Activate.ps1`

Step B: Install Packages & Run Jupyter

1. **Install packages:** `pip install jupyter numpy scipy pandas matplotlib seaborn plotly`

2. **Launch Jupyter:** `jupyter notebook` (or open the `.ipynb` file in VS Code selecting the `.venv` kernel).

Section 4: Data Wrangling & Cleaning with Pandas

Real-world datasets contain anomalies, missing attributes, or raw text columns. We must sanitize this data before modeling. We use the [Northwind_Orders.csv](#) dataset.

1. Handling Missing Data

- `isna()` / `isnull()` : Returns a boolean mask of elements that are null.
- `fillna(value)` : Replaces null values with a specified constant, column mean, or interpolation.

```
import pandas as pd

# Load the Northwind dataset
df = pd.read_csv("Northwind_Orders.csv")

# 1. Detect missing values across all columns
print("Missing values per column:\n", df.isna().sum())

# 2. Fill missing values in 'ship_region' with a default string
df["ship_region"] = df["ship_region"].fillna("Unknown")
print("\nMissing values in ship_region after fill:", df["ship_region"].isna().su
```

2. String Manipulation on Columns

Pandas provides a `.str` accessor to execute vectorized string operations.

```
# Convert category names to lowercase
df["category_name"] = df["category_name"].str.lower()

# Create a composite product descriptor key: "category: product_name"
df["product_descriptor"] = df["category_name"] + ": " + df["product_name"]
print(df[["product_descriptor"]].head(3))
```

Merging and Joining DataFrames

To combine relational datasets based on shared keys, use `pd.merge()`. Here we merge our main orders dataset with a supplementary table mapping countries to continents.

```
# 1. Create a supplementary DataFrame mapping countries to continents
continent_data = pd.DataFrame({
    "customer_country": ["Germany", "Mexico", "UK", "Sweden", "France", "Spain",
    "continent": ["Europe", "North America", "Europe", "Europe", "Europe", "Euro
}))

# 2. Left Join: Merge continent info into our main orders DataFrame on 'customer
df_merged = pd.merge(df, continent_data, on="customer_country", how="left")
print(df_merged[["order_id", "customer_country", "continent"]].head(5))
```

Grouping & Aggregations

The `.groupby()` method splits a DataFrame into groups and applies aggregate operations (like sum, mean, count).


```
# Group by category name and sum the quantities and revenue
category_summary = df.groupby("category_name")[["quantity_ordered", "total_item_
print("Category Summaries:\n", category_summary)

# Group by employee name and get multiple statistics on their transactions (.agg
employee_stats = df.groupby("employee_name")["total_item_revenue"].agg(["sum", "
print("\nEmployee Sales Statistics:\n", employee_stats.head(5))
```

Pivot Tables

A pivot table summarizes metrics across row-column intersections.

```
# Create pivot table summarizing sales revenue by Country (rows) and Category (c
pivot_df = df.pivot_table(
    index="customer_country",
    columns="category_name",
    values="total_item_revenue",
    aggfunc="sum",
    fill_value=0.0
)
print("Sales Pivot Table:\n", pivot_df.head(5))
```

Section 5: Static & Interactive Data Visualization

1. Basic Line Plots (Matplotlib)

Plot monthly sales revenue trends.

```

import matplotlib.pyplot as plt

# Parse date and extract year-month
df["order_date"] = pd.to_datetime(df["order_date"])
df["year_month"] = df["order_date"].dt.to_period("M")

# Group and calculate monthly revenue
monthly_revenue = df.groupby("year_month")["total_item_revenue"].sum()

# Convert PeriodIndex to strings for plotting
months_str = [str(m) for m in monthly_revenue.index]

plt.figure(figsize=(10, 4))
plt.plot(months_str, monthly_revenue.values, color="green", marker="o", linestyle='solid')
plt.title("Northwind Sales Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue ($)")
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("monthly_sales_trend.png")
plt.close()

```

2. Statistical Box Plots (Seaborn)

Show the distribution of unit prices sold across different product categories.

```

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")

# Create box plot
sns.boxplot(data=df, x="category_name", y="unit_price_sold", palette="Set3")
plt.title("Product Unit Prices Sold by Category")
plt.xlabel("Product Category")
plt.ylabel("Unit Price Sold ($)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig("category_price_distribution.png")
plt.close()

```

Interactive Visualization (Plotly)

Unlike Matplotlib and Seaborn, **Plotly** generates interactive HTML objects, permitting users to toggle legend items, zoom in, and hover to see coordinates.

```

import plotly.express as px

# Create an interactive scatter plot comparing quantity ordered and price sold
fig = px.scatter(
    df.head(500), # Plot first 500 rows for performance
    x="list_unit_price",
    y="quantity_ordered",
    size="total_item_revenue",
    color="category_name",
    hover_name="product_name",
    title="Product Orders: Price vs Quantity (First 500 records)"
)

# Save as a standalone HTML file
fig.write_html("interactive_orders_plot.html")

```

Practical Examples (Interactive & Runnable)

Example 1: Sales Analysis by Category

Aggregates total revenue per category and saves a static horizontal bar chart.

```
import pandas as pd
import matplotlib.pyplot as plt

def analyze_category_sales(csv_path, output_image_path):
    df = pd.read_csv(csv_path)

    # Calculate total revenue per category
    category_sales = df.groupby("category_name")["total_item_revenue"].sum().sort_values(ascending=False)

    plt.figure(figsize=(10, 5))
    category_sales.plot(kind="barh", color="coral")
    plt.title("Total Revenue by Product Category")
    plt.xlabel("Total Revenue ($)")
    plt.ylabel("Product Category")
    plt.tight_layout()
    plt.savefig(output_image_path)
    plt.close()
    print("Category Sales Analysis Completed.")

analyze_category_sales("Northwind_Orders.csv", "category_revenue.png")
```

Example 2: Interactive Customer Sales Bubble Plot (Plotly)

Aggregates sales performance by customer country and plots it interactively.

```

import plotly.express as px
import pandas as pd

def build_country_sales_distribution(csv_path, output_html_path):
    df = pd.read_csv(csv_path)

    # Group by customer country and compute statistics
    country_df = df.groupby("customer_country").agg(
        total_revenue=("total_item_revenue", "sum"),
        total_quantity=("quantity_ordered", "sum"),
        unique_products=("product_id", "nunique")
    ).reset_index()

    # Generate interactive scatter/bubble chart
    fig = px.scatter(
        country_df,
        x="total_quantity",
        y="total_revenue",
        size="unique_products",
        color="customer_country",
        hover_name="customer_country",
        title="Country-level Sales Performance (Bubble Size = Unique Products Pu
    )

    fig.write_html(output_html_path)
    print(f"Interactive bubble plot written to {output_html_path}")

build_country_sales_distribution("Northwind_Orders.csv", "country_sales_bubble.h

```

Practice Assignments

Objective

Apply NumPy operations for data normalization, boolean filtering, matrix transformations, and dot products on the Northwind dataset. Access Pandas DataFrames loaded from `Northwind_Orders.csv` using `.loc` and `.iloc` slicing, and compute shipping cost optimizations using SciPy.

Easy Assignments

Assignment 1: Product Order Vector Processor (NumPy Vectorization)

Scenario

You are writing a billing module for the Northwind e-commerce system. The system processes order items by calculating subtotals, applying discounts, and computing final checkout costs.

Problem Description

Write a function `process_order_vector(quantities, unit_prices, discounts)` :

1. The parameters are lists of size 5 containing item details from an order:
 - `quantities` (list of integers): The count of items ordered.
 - `unit_prices` (list of floats): The list price per unit.
 - `discounts` (list of floats): The discount percentage per item (e.g. `0.05` represents a 5% discount).

1. **Vector Conversion:** Convert all three input lists into 1D NumPy arrays.

2. **Calculations:**

- Calculate the **raw subtotal** for each of the 5 items:

$$\text{Subtotal} = \text{quantities} \times \text{unit_prices}$$
 (Perform this as a vectorized element-wise multiplication; do not write loops).

- Calculate the **discounted price** for each of the 5 items:

$$\text{Discounted Price} = \text{Subtotal} \times (1.0 - \text{discounts})$$

- Calculate the **total order cost** by summing all of the discounted prices.

1. **Return:** A tuple containing:

```
(subtotals_array, discounted_prices_array, total_order_cost)
```

Example Walkthrough

```

import numpy as np

qty = [2, 10, 5, 1, 4]
prices = [15.0, 10.0, 20.0, 100.0, 25.0]
disc = [0.0, 0.10, 0.0, 0.05, 0.20]

subtotals, final_prices, total = process_order_vector(qty, prices, disc)

print(subtotals)      # Output: [ 30. 100. 100. 100. 100.]
print(final_prices)   # Output: [30. 90. 100. 95. 80.]
print(total)          # Output: 395.0

```

Assignment 2: Northwind Sales Filter (Pandas Filtering)

Scenario

An analyst needs to query the historical sales data in `Northwind_Orders.csv` to evaluate individual salesperson performance. You need to write a filter that retrieves order details for a specific employee and destination country.

Problem Description

Write a function `filter_sales_by_employee(csv_path, employee_name, target_country)`:

1. `csv_path` is a string pointing to `Northwind_Orders.csv`.
2. **Load Data:** Read the CSV file into a Pandas DataFrame.
3. **Filtering:** Filter the DataFrame to find all rows where:
 - `"employee_name"` matches `employee_name` (case-sensitive, e.g. `"Steven Buchanan"`).
 - `"ship_country"` matches `target_country` (case-sensitive, e.g. `"France"`).
1. **Slicing:** Extract only the columns: `["order_id", "product_name", "total_item_revenue"]`.
2. **Return:** The filtered subset DataFrame. Keep the original index of the matching rows. If no rows match, return an empty DataFrame with columns `["order_id", "product_name", "total_item_revenue"]`.

Example Walkthrough

```
csv_path = "Northwind_Orders.csv"
sales_df = filter_sales_by_employee(csv_path, "Steven Buchanan", "France")
print(sales_df.head(2))

## Expected Console Output:
##      order_id      product_name  total_item_revenue
## 0      10248      Queso Cabrales             168.0
## 1      10248  Singaporean Hokkien...             98.0
```

Medium Assignments

Assignment 3: Quantity Capping & Reshaping (NumPy Arrays)

Scenario

An inventory forecast model processes transaction quantities from `Northwind_Orders.csv`. Due to stocking limits, any quantity ordering count above a threshold must be identified and capped, and the flat data must be reshaped into a grid for analysis.

Problem Description

Write a function `cap_and_grid_quantities(quantities_list, max_allowed_qty)`:

1. `quantities_list` is a list of 24 integers representing product quantities ordered in transactions.
2. **Boolean Masking:** Convert the list to a 1D NumPy array. Create a boolean mask to locate all elements in the array that are strictly greater than `max_allowed_qty`. Count the number of elements that exceed this threshold.
3. **Capping:** Replace all values in the array that exceed `max_allowed_qty` with `max_allowed_qty`.
4. **Reshaping:** Reshape the capped 1D array into a 2D array of shape `(6, 4)` (6 rows, 4 columns).
5. **Column Averages:** Calculate the average quantity along the columns (mean along axis 0).

6. Return: A tuple containing:

```
(capped_count, reshaped_grid, column_averages_array).
```

Example Walkthrough

```
import numpy as np

raw_qtys = [12, 10, 5, 9, 40, 10, 35, 15, 6, 15, 20, 40, 25, 40, 6, 15, 12, 40]

count, grid, col_avgs = cap_and_grid_quantities(raw_qtys, 25)
print(count)          # Output: 6   (values 40, 35, 40, 40, 40, 30 are capped)
print(grid.shape)     # Output: (6, 4)
print(col_avgs)       # Output: 1D array of 4 average values (axis 0)
```

Assignment 4: Shipping Freight Slicing (Pandas loc vs. iloc)

Scenario

The logistics team audits shipping freight costs in `Northwind_Orders.csv`. You need to write two slicing functions to extract chunks of freight transactions using both row labels and integer coordinates.

Problem Description

Write two separate slicing functions:

1. `extract_freight_by_label(df, start_row_label, end_row_label, col_names) :`
 - Uses label-based indexing (`.loc`) to slice the DataFrame from `start_row_label` to `end_row_label` (inclusive) and extract only the columns listed in `col_names` .
 - Returns the sliced DataFrame.
1. `extract_freight_by_position(df, row_start_idx, row_end_idx, col_indices) :`
 - Uses position-based indexing (`.iloc`) to slice the DataFrame from row index `row start idx` to `row end idx` (exclusive) and columns at integer indices specified in the

`list col_indices .`

- Returns the sliced DataFrame.

Example Walkthrough

```
import pandas as pd

df = pd.read_csv("Northwind_Orders.csv")

## Slicing rows 10 to 12 (inclusive) with columns 'order_id' and 'freight'
loc_res = extract_freight_by_label(df, 10, 12, ["order_id", "freight"])
print(loc_res)

## Slicing rows 10 to 12 (exclusive, i.e. 10 and 11) with columns index 0 (order)
iloc_res = extract_freight_by_position(df, 10, 12, [0, 13])
print(iloc_res)
```

Difficult Assignments

Assignment 5: Coordinates Alignment for Port Shipments (NumPy Linear Algebra)

Scenario

A shipping port logs the (x, y) coordinate offsets of customer cities relative to a central hub. To calibrate satellite distances, the coordinates must be translated relative to a new port dock and rotated to align with the camera angle.

Problem Description

Write a function `align_customer_coordinates(coordinates, angle_degrees, dock_offset) :`

1. `coordinates` is a 2D NumPy array of shape `(N, 2)` representing (x, y) coordinate offsets.
2. `angle_degrees` is the rotation angle in degrees (float). Convert this angle to radians:

$$\theta = \text{angle_degrees} \times \frac{\pi}{180}$$

1. Construct the 2D **Rotation Matrix** R :

```
$$R = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$
```

1. **Determinant Verification:** Calculate the determinant of the rotation matrix R using

```
np.linalg.det(R) .
```

- If the absolute difference between the determinant and 1.0 is greater than $1e-6$ (i.e. $|\text{det} - 1.0| > 10^{-6}$), raise a `ValueError` with the message "Invalid rotation matrix."

1. **Transformation Sequence:**

- **Translation:** Translate the points by adding `dock_offset` (a 1D array of shape `(2,)` representing $[dx, dy]$) to `coordinates` using broadcasting.
- **Rotation:** Rotate the translated points by multiplying them by the rotation matrix.

```
$$\text{Aligned Coordinates} = \text{Translated Coordinates} \times R^T$$
```

1. Return a tuple containing: `(rotation_determinant, aligned_coordinates)`.

Example Walkthrough

```
import numpy as np

coords = np.array([
    [10.0, 0.0],
    [0.0, 10.0],
    [10.0, 10.0]
])
offset = np.array([-5.0, 5.0])
angle = 90.0

det, aligned = align_customer_coordinates(coords, angle, offset)
print(det) # Output: 1.0 (very close to 1.0)
print(np.round(aligned))
## Step 1: Translate -> [[5.0, 5.0], [-5.0, 15.0], [5.0, 15.0]]
## Step 2: Rotate 90 deg -> [[-5.0, 5.0], [-15.0, -5.0], [-15.0, 5.0]]
```

Assignment 6: Shipping Cost Decay Optimizer (SciPy Optimization)

Scenario

A logistics company analyzes Northwind shipping freight costs. The average freight charge per unit decays exponentially as the total quantity of items ordered in a batch increases. The freight charge is modeled by: $\text{freight}(q) = a \cdot e^{-b \cdot q} + 10.0$ where q is the quantity ordered, a is the maximum shipping surcharge, and b is the cost decay coefficient. You need to write a SciPy optimization script to fit parameters a and b to observed transaction data by minimizing the Sum of Squared Errors (SSE).

Problem Description

Write a function `optimize_freight_model(quantity_array, observed_freight_array)` :

- `quantity_array` is a 1D NumPy array representing quantities q .
- `observed_freight_array` is a 1D NumPy array of the same size representing observed freight costs.
- Objective Error Function:** Define a local error function `calculate_sse(params)` :
 - `params` is a list or tuple: `[a, b]` .
 - Calculate the modeled freight for each quantity q in `quantity_array` :

$$\text{modeled}(q) = a \cdot e^{-b \cdot q} + 10.0$$

- Compute and return the Sum of Squared Errors (SSE) between the model and observed data:

$$\text{SSE} = \sum \left(\text{modeled}(q) - \text{observed}(q) \right)^2$$

1. Optimization:

- Use `scipy.optimize.minimize` to find parameters `[a, b]` that minimize `calculate_sse` .
- Set the initial guess to `[100.0, 0.05]` .
- Enforce parameter bounds to ensure physical validity: $a \geq 0.0$ and $b \geq 0.001$.

- Return:** A dictionary containing:

`{"a": opt_a, "b": opt_b, "sse": min_sse}` where all values are rounded to **4 decimal places**.

Example Walkthrough

```
import numpy as np

## Mock experimental data (quantities and matching observed freights)
q_data = np.array([5, 10, 15, 20, 30, 40, 50])
observed_f = np.array([90.5, 75.2, 60.1, 50.8, 35.4, 25.1, 18.0])

fit_results = optimize_freight_model(q_data, observed_f)
print(fit_results)
## Expected Output format:
## {'a': , 'b': , 'sse': }
```

Objective

Apply Pandas operations for missing data cleaning, column concatenation, table merging/joining, grouping aggregates, and pivot table construction. Render custom charts using Matplotlib/Seaborn and interactive figures using Plotly Express on the real `Northwind_Orders.csv` dataset.

Easy Assignments

Assignment 7: Northwind Order Shipped Date Sanitizer

Scenario

The shipping logs in `Northwind_Orders.csv` contain missing shipped dates for orders that are still in transit. Before executing monthly calculations, you need to clean these nulls based on the destination country.

Problem Description

Write a function `sanitize_order_dates(df)` that processes the Northwind orders DataFrame:

1. `df` is the Pandas DataFrame loaded from `Northwind_Orders.csv`.

2. Conditional Shipped Date Fill:

- For rows where the `"shipped_date"` is null (`NaN`):
- If the shipping destination country (`"ship_country"`) is `"USA"` or `"Canada"` , fill the missing `"shipped_date"` with `"In-Transit: Domestic"` .
- If it is any other country, fill it with `"In-Transit: International"` .

1. Region Fill:

- For rows where `"ship_region"` is null (`NaN`), fill it with the string `"No-Region"` .

1. Return: The sanitized DataFrame.

Example Walkthrough

```
import pandas as pd
import numpy as np

## Load dataset
df = pd.read_csv("Northwind_Orders.csv")

## Verify there are missing shipped dates
print("Null dates before:", df["shipped_date"].isna().sum())

clean_df = sanitize_order_dates(df)

## Verify nulls are resolved
print("Null dates after:", clean_df["shipped_date"].isna().sum())
print(clean_df[clean_df["shipped_date"].str.startswith("In-Transit")][["ship_cou
```

Assignment 8: Interactive Freight Cost Trend Visualizer

Scenario

A logistics coordinator requires an interactive line graph showing shipping freight charges handled by different carrier firms over time. The graph must allow toggling shipping lines and hovering to view exact shipment costs.

Problem Description

Write a function `generate_freight_chart(df, output_html_path)` :

1. `df` is the Pandas DataFrame loaded from `Northwind_Orders.csv` .

2. Interactive Charting:

- Use Plotly Express (`plotly.express` as `px`) to generate a line chart (`px.line()`):
- Set the X-axis to `"order_date"` .
- Set the Y-axis to `"freight"` .
- Set the line color category to `"shipper_company_name"` .
- Enable markers on the data points.
- Add a custom title: `"Northwind Order Freight Costs by Shipping Carrier"` .

1. **Save:** Save the interactive chart as a standalone HTML file to `output_html_path` using `fig.write_html()` .
-

Medium Assignments

Assignment 9: Product Supplier Performance Reconciler

Scenario

You have product catalog details and order transactions stored in separate sheets. To evaluate supplier performance, you must join these tables and aggregate total sales quantity and revenue per supplier.

Problem Description

Write a function `reconcile_supplier_performance(df_orders, df_suppliers)` :

1. `df_orders` is a DataFrame with columns: `["order_id", "product_id", "total_item_revenue", "quantity_ordered"]`.
2. `df_suppliers` is a DataFrame with columns: `["product_id", "supplier_company_name", "supplier_country"]`.
3. **Merge:** Perform an **inner join** on `"product_id"` using `pd.merge()`.
4. **Aggregation:** Group the records by `"supplier_company_name"`. Calculate:
 - The sum of `"total_item_revenue"` (rename/store as `"Total_Revenue"`).
 - The sum of `"quantity_ordered"` (rename/store as `"Total_Quantity"`).
1. **Return:** A new DataFrame indexed by `"supplier_company_name"` containing the columns `["Total_Revenue", "Total_Quantity"]`. Sort the resulting DataFrame's index alphabetically.

Example Walkthrough

```
import pandas as pd

## Extract subsets from main dataset to simulate separate tables
df_full = pd.read_csv("Northwind_Orders.csv")
orders_subset = df_full[["order_id", "product_id", "total_item_revenue", "quantity_ordered"]]
suppliers_subset = df_full[["product_id", "supplier_company_name", "supplier_country"]]

summary_df = reconcile_supplier_performance(orders_subset, suppliers_subset)
print(summary_df.head(3))

## Expected Console Output:
##
##      supplier_company_name      Total_Revenue  Total_Quantity
## Aux joyeux ecclésiastiques      33827.65           769
## Bigfoot Breweries              26335.50           940
## Cooperativa de Quesos...        21980.20           800
```

Assignment 10: Interactive Product Sales Bubble Plot

Scenario

To perform pricing analysis, a sales manager wants a bubble plot comparing catalog list prices (X-axis) against order quantities (Y-axis). The bubble sizes must represent total item revenue, colored by the product's category.

Problem Description

Write a function `generate_product_sales_bubble(df, output_html_path)` :

1. `df` is the Pandas DataFrame loaded from `Northwind_Orders.csv` .

2. Plotting:

- Use Plotly Express to generate an interactive scatter plot (`px.scatter()`):
- Set the X-axis to `"list_unit_price"` .
- Set the Y-axis to `"quantity_ordered"` .
- Set the bubble sizes (`size` parameter) to `"total_item_revenue"` .
- Set the colors (`color` parameter) to `"category_name"` .
- Set the hover overlay name label (`hover_name` parameter) to `"product_name"` .
- Add the title: `"Product Sales Analysis: Price, Quantity & Revenue"` .

1. **Save:** Save the interactive chart as a standalone HTML file to `output_html_path` using `fig.write_html()` .

Difficult Assignments

Assignment 11: Employee Monthly Sales Summary & Pivot Table

Scenario

Sales coordinators need to analyze monthly revenue generated by each employee across different product categories. You need to write a module that parses order dates, classifies high-value transactions, and builds a summary pivot table.

Problem Description

Write a function `generate_employee_monthly_pivot(df)` :

1. `df` is the Pandas DataFrame loaded from `Northwind_Orders.csv`.
 2. **Date Standardizing:** Parse the `"order_date"` column into standard Pandas Datetime objects using `pd.to_datetime()`.
 3. **Extract Month:** Extract the full month name (e.g. `"July"`, `"August"`) from the datetime column and store it in a new column named `"order_month"`.
 4. **Value Classification:** Add a new column named `"value_class"`:
 - If `"total_item_revenue"` is strictly greater than `1000.0`, the row's value is `"Premium"`.
 - Otherwise, the value is `"Regular"`.
 - Perform this classification using vectorized checks (such as `np.where()`).
1. **Pivot Table:** Build a pivot table summarizing the transactions:
 - **Index (Rows):** `"employee_name"`
 - **Columns:** `"category_name"`
 - **Values:** Sum of `"total_item_revenue"`
 - Set the pivot table's aggregation function to `"sum"`. Fill any empty pivot intersections (`NaN`) with the value `0.0`.
 1. **Return:** The resulting pivot table DataFrame.
-

Assignment 12: Interactive Category Revenue by Country Dashboard

Scenario

Executive leadership requires a country-level e-commerce dashboard. You need to write a module that filters out discontinued products, groups revenues by category and country, and builds an interactive side-by-side grouped bar plot.

Problem Description

Write a function `generate_category_country_dashboard(df, output_html_path)`:

1. `df` is the Pandas DataFrame loaded from `Northwind_Orders.csv`.

2. Data Wrangling:

- Filter out and drop all rows where `"product_discontinued"` matches `1` (representing discontinued items).
- Group the remaining transactions by both `"customer_country"` and `"category_name"`.
- Calculate the sum of `"total_item_revenue"` for each group.
- Reset the index of the aggregated DataFrame.

1. Grouped Bar Plot:

- Use Plotly Express to construct a grouped bar plot (`px.bar()`):
- Set the X-axis to `"category_name"`.
- Set the Y-axis to `"total_item_revenue"`.
- Set the bar color category to `"customer_country"`.
- Set the bar positioning mode (`barmode` parameter) to `"group"` (side-by-side columns).
- Add the title: `"Northwind Category Revenue by Customer Country (Active Products)"`.

1. **Save:** Save the interactive chart as a standalone HTML file to `output_html_path` using `fig.write_html()`.
-

Chapter 9: Web Technologies — Micro-Frameworks & Web Scraping

Welcome to Chapter 10! Today we explore web development concepts and micro-framework web routing in Python. We will cover:

1. **Web Architectures:** Understanding Client-Server request cycles and MVC (Model-View-Controller) / MTV (Model-Template-View) design patterns.
2. **Comparing Frameworks:** Highlighting differences between lightweight Flask micro-frameworks and full-stack Django platforms.
3. **Flask Routing:** Defining routes, handling path parameters (strings, integers), and parsing HTTP methods (GET, POST).
4. **Jinja2 Templates & Redirects:** Dynamically rendering HTML views, executing form validations, and routing users using redirect hooks.
5. **Web API endpoints:** Returning clean JSON payloads and binding REST endpoints to SQLite databases.

NOTE:

***Prerequisites:** It is highly recommended to run the web server examples and assignments inside an isolated virtual environment.*

Step A: Setup & Activate Virtual Environment

* macOS/Linux: `python -m venv .venv && source .venv/bin/activate`

* Windows (CMD): `python -m venv .venv && .venv\Scripts\activate.bat`

* Windows (PS): `python -m venv .venv && .venv\Scripts\Activate.ps1`

Step B: Install Flask

````bash`

`pip install flask`

`````

Section 1: Web Architecture & Flask Micro-framework Basics

1. Client-Server Flow

```
+-----+          HTTP Request (URL, GET/POST)          +-----+
| Client | -----> | Server |
| (Browser) | <----- | (Python) |
+-----+          HTTP Response (HTML/JSON, 200/404) +-----+
```

2. MVC vs. MTV Architectures

- **MVC (Model-View-Controller):**
 - **Model:** Manages database access and business logic.
 - **View:** Renders visual representation (HTML page).
 - **Controller:** Coordinates between Model and View, handling requests.
- **MTV (Model-Template-View):** Used by **Django**.
 - **Model:** Database layer (same as MVC).
 - **Template:** Renders pages (similar to MVC View).
 - **View:** Acts as Controller (handles requests, loads models, maps templates).

3. Flask vs. Django

| Feature | Flask | Django |
|------------|--|--|
| Philosophy | Micro-framework (modular, simple, choose-your-own-addons). | Full-stack (opinionated, "batteries-included" toolkit). |
| Best For | Microservices, lightweight web APIs, quick prototyping. | Large-scale enterprise applications, complex web applications. |

Section 2: Form Processing & Dynamic Web Databases

1. Simple App Setup & Path Parameters

```
from flask import Flask, jsonify

app = Flask(__name__)

# Basic GET route returning plain text
@app.route("/")
def home():
    return "Welcome to the API!"

# Dynamic path parameter: matches string parameter
@app.route("/greet/")
def greet(student_name):
    return f"
```

Hello, {student_name}!

"

```
# Dynamic integer parameter: converts path parameter to int
@app.route("/square/")
def calculate_square(num):
    result = num ** 2
    # Returns a JSON response with HTTP 200
    return jsonify({"input": num, "square": result}), 200

if __name__ == "__main__":
    # Start local development server
    app.run(debug=True, port=5000)
```

2. Handling POST Requests & Redirects

Use `request.form` to parse incoming HTML form inputs, and `redirect()` to guide navigation.

```

from flask import Flask, request, redirect, url_for, render_template_string

app = Flask(__name__)

# Basic form template
form_html = """

    Username: 

    """

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        # Extract form field named 'user_input'
        username = request.form.get("user_input")
        if username == "admin":
            # Redirect to the dashboard view
            return redirect(url_for("dashboard"))
        return "Unauthorized Access!", 403
    return render_template_string(form_html)

@app.route("/dashboard")
def dashboard():
    return "

```


Welcome Administrator.

"

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Practical Examples (Interactive & Runnable)

Example: SQLite database API micro-service (Flask)

Demonstrates setting up a Flask app that queries database tables and returns JSON results based on request arguments.

```

import sqlite3
from flask import Flask, jsonify, request

app = Flask(__name__)

def init_db():
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT)")
    cursor.execute("INSERT OR IGNORE INTO users VALUES (1, 'Alice')")
    cursor.execute("INSERT OR IGNORE INTO users VALUES (2, 'Bob')")
    conn.commit()
    conn.close()

@app.route("/api/users", methods=["GET"])
def get_users():
    # Read query parameter e.g., /api/users?name=Alice
    filter_name = request.args.get("name")

    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()

    if filter_name:
        cursor.execute("SELECT * FROM users WHERE name = ?", (filter_name,))
    else:
        cursor.execute("SELECT * FROM users")

    rows = cursor.fetchall()
    conn.close()

    # Build list of dicts
    users_list = [{"id": r[0], "name": r[1]} for r in rows]
    return jsonify(users_list), 200

if __name__ == "__main__":
    init_db()
    # Runs in standard background mode
    # app.run(port=5000)

```

Welcome to Chapter 11! Today we conclude our Python web technologies and scraping modules. We will cover:

1. **Dynamic Web Applications (Flask + SQLite):** Designing page templates, processing HTML form submissions, and managing database connections.
2. **Web Scraping Foundations:** Extracting web page HTML using the `requests` library and bypassing basic crawler blocks using custom headers (`User-Agent`).
3. **HTML Parsing with BeautifulSoup:** Navigating the Document Object Model (DOM) and extracting tag attributes and text elements.
4. **Structured Crawling with Scrapy:** Building spiders, setting up pagination followers, and processing items via Database Pipelines.
5. **Wrangling Web Tables:** Utilizing Pandas `pd.read_html()` to parse tables in single function calls.

NOTE:

***Prerequisites:** It is highly recommended to run the database web application and scraping examples inside an isolated virtual environment.*

Step A: Setup & Activate Virtual Environment

* macOS/Linux: `python -m venv .venv && source .venv/bin/activate`

* Windows (CMD): `python -m venv .venv && .venv\Scripts\activate.bat`

* Windows (PS): `python -m venv .venv && .venv\Scripts\Activate.ps1`

Step B: Install Libraries

```bash`

`pip install flask requests beautifulsoup4 scrapy lxml html5lib`

`````

SQLite-Powered Flask Web Applications

A dynamic web application renders views by pulling data from a database and injecting it into HTML templates. We use Jinja2 syntax in Flask templates to perform loops and conditions.

1. Project Directory Structure

For Flask to find templates and static files (like stylesheets), you must organize files into standard subfolders:

```
my_web_app/  
|- app.py (Main Python application)  
|- templates/  
    |- list_books.html (Jinja2 Template file)  
|- static/  
    |- style.css (Static stylesheet)
```

2. Implementation: Flask App with SQLite

Here is a complete Flask application that queries database items and allows users to append items via HTML form posts.

Flask backend (`app.py`):

```
import sqlite3  
from flask import Flask, render_template_string, request, redirect, url_for  
  
app = Flask(__name__)  
DB_FILE = "library.db"  
  
def init_db():  
    conn = sqlite3.connect(DB_FILE)  
    cursor = conn.cursor()  
    cursor.execute("CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, ti  
    conn.commit()  
    conn.close()  
  
# HTML template with Jinja2 loop syntax  
TEMPLATE_HTML = """
```

Book Collection

```
{% for book in books %}

• {{ book[1] }} by {{ book[2] }}

{% endfor %}
```

Add a New Book

Title:

Author:

```
"""
```

```
@app.route("/books")
def list_books():
```

```

conn = sqlite3.connect(DB_FILE)
cursor = conn.cursor()
cursor.execute("SELECT * FROM books")
books = cursor.fetchall()
conn.close()
return render_template_string(TEMPLATE_HTML, books=books)

@app.route("/add-book", methods=["POST"])
def add_book():
    title = request.form.get("title")
    author = request.form.get("author")

    if title and author:
        conn = sqlite3.connect(DB_FILE)
        cursor = conn.cursor()
        cursor.execute("INSERT INTO books (title, author) VALUES (?, ?)", (title,
        conn.commit()
        conn.close()

    return redirect(url_for("list_books"))

if __name__ == "__main__":
    init_db()
    # app.run(debug=True)

```

Section 3: Web Scraping & Crawler Pipelines

Web Scraping with BeautifulSoup

Web Scraping is the technique of programmatically downloading web pages and extracting data points from their HTML structure.

1. HTTP Requests and Custom Headers

Many websites block default automation tools. To bypass these restrictions, add a **User-Agent** string to your HTTP request headers.

```
import requests

url = "https://quotes.toscrape.com/"
custom_headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
}

response = requests.get(url, headers=custom_headers)
print("HTTP Status Code:", response.status_code) # Output: 200
```

2. Navigating the HTML DOM

Use **BeautifulSoup** to parse HTML and search for specific tags using CSS classes, IDs, or element names.

```
from bs4 import BeautifulSoup

# Initialize BeautifulSoup parser
soup = BeautifulSoup(response.text, "html.parser")

# 1. Locate elements by class name
quote_boxes = soup.find_all("div", class_="quote")

for box in quote_boxes:
    # 2. Extract text contents
    text = box.find("span", class_="text").get_text()
    author = box.find("small", class_="author").get_text()
    print(f"Quote: {text} | Author: {author}")
```

Web Crawling with Scrapy

Scrapy is a powerful Python framework designed for crawling websites. While BeautifulSoup parses single pages, Scrapy is optimized for traversing pagination links recursively and pipeline operations.

1. Basic Spider and Pagination

In Scrapy, you define a `Spider` class containing start URLs and parse callbacks.

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = "quotes_spider"
    start_urls = ['https://quotes.toscrape.com/']

    def parse(self, response):
        # Loop through quote boxes
        for quote in response.css('div.quote'):
            yield {
                'text': quote.css('span.text::text').get(),
                'author': quote.css('small.author::text').get(),
            }

        # Locate next page link and crawl recursively
        next_page = response.css('li.next a::attr(href)').get()
        if next_page is not None:
            # Yield request to follow link
            yield response.follow(next_page, callback=self.parse)
```

2. Item Pipelines

Pipelines process scraped items yielded by the spider (e.g., verifying attributes, deduplicating, or committing directly to a database).


```

import sqlite3
from scrapy.exceptions import DropItem

class SQLitePipeline:
    def open_spider(self, spider):
        # Open database connection on startup
        self.conn = sqlite3.connect("quotes.db")
        self.cursor = self.conn.cursor()
        self.cursor.execute("CREATE TABLE IF NOT EXISTS quotes (text TEXT, autho

    def process_item(self, item, spider):
        # Validation checks
        if not item.get("text") or not item.get("author"):
            raise DropItem("Missing text or author.")

        # Parameterized insert
        self.cursor.execute("INSERT INTO quotes VALUES (?, ?)", (item["text"], i
        self.conn.commit()
        return item

    def close_spider(self, spider):
        # Close connection when crawl completes
        self.conn.close()

```

Pandas Web Table Scraper

If you need to extract structured rows and columns from HTML table elements, Pandas offers `pd.read_html()`, which scans the page and returns all found tables as a list of DataFrames.

```
import pandas as pd

# Target URL containing table elements
url = "https://www.w3schools.com/html/html_tables.asp"

# Parse tables
tables = pd.read_html(url)
print("Total tables found:", len(tables))

# Inspect first table
df = tables[0]
print(df.head(2))
```

Practice Assignments

Objective

Implement Flask applications containing dynamic routes, path variable conversion, conditional redirection, in-memory state tracking, database-connected API querying, and JSON payload validation.

Easy Assignments

Assignment 1: Academic Greeting Web App

Scenario

You are developing a student registration microservice. The routing module needs dynamic handlers to parse student names and compute grades passed as URL path variables.

Problem Description

Implement a complete Flask web application containing the following routes:

1. Route `/` (GET):

- Returns the plain text message: `"Welcome to the Software Academy Registration Portal."`

1. Route `/greet/<student_name>` (GET):

- Dynamically parses the `<student_name>` variable from the URL path.
- Returns a formatted HTML heading: `"<h1>Hello, <student_name>! Welcome to Academy.</h1>"`.

1. Route `/calculate/grade/<int:marks_obtained>/<int:total_marks>` (GET):

- **Validation:** If `total_marks` is less than or equal to `0`, return a JSON response: `{"error": "Total marks must be greater than zero."}` with an HTTP status code of `400`.
- **Success:** Calculate the percentage:

$$\text{Percentage} = \frac{\text{marks_obtained}}{\text{total_marks}} \times 100$$

Round the value to **1 decimal place** (e.g. `82.5`).

- Return a JSON response: `{"obtained": <marks_obtained>, "total": <total_marks>, "percentage": <percentage>}` with an HTTP status code of `200`.

Note: Define the Flask app instance variable as `app`. Ensure `app.run` is enclosed within `if __name__ == '__main__':` so it does not block import statements during testing.

Example Walkthrough

- Requesting `GET /greet/Lisa` returns the HTML: `<h1>Hello, Lisa! Welcome to Academy.</h1>`
- Requesting `GET /calculate/grade/45/50` returns the JSON payload:

`{"obtained": 45, "total": 50, "percentage": 90.0}` with HTTP 200.

- Requesting `GET /calculate/grade/45/0` returns the JSON payload:

`{"error": "Total marks must be greater than zero."}` with HTTP 400.

Assignment 2: Dynamic Temperature Converter Web App

Scenario

A weather application microservice requires API endpoints to convert temperatures between Celsius and Fahrenheit scales dynamically using float path parameters.

Problem Description

Implement a Flask application with the following endpoints:

1. Route `/convert/c_to_f/<float:celsius>` **(GET):**

- Takes a floating-point temperature value in Celsius from the path.
- Performs the conversion to Fahrenheit:

$$F = C \times 1.8 + 32$$

- Returns a JSON response: `{"celsius": <celsius>, "fahrenheit": <fahrenheit>}` (with both values rounded to **1 decimal place**) and an HTTP status code of `200`.

1. Route `/convert/f_to_c/<float:fahrenheit>` **(GET):**

- Takes a floating-point temperature value in Fahrenheit from the path.
- Performs the conversion to Celsius:

$$C = \frac{F - 32}{1.8}$$

- Returns a JSON response: `{"fahrenheit": <fahrenheit>, "celsius": <celsius>}` (with both values rounded to **1 decimal place**) and an HTTP status code of `200`.

Example Walkthrough

- Requesting `GET /convert/c_to_f/0.0` returns JSON:

`{"celsius": 0.0, "fahrenheit": 32.0}` with HTTP 200.

- Requesting `GET /convert/f_to_c/100.0` returns JSON:

`{"fahrenheit": 100.0, "celsius": 37.8}` with HTTP 200.

Medium Assignments

Assignment 3: Secure Administrator Gateway

Scenario

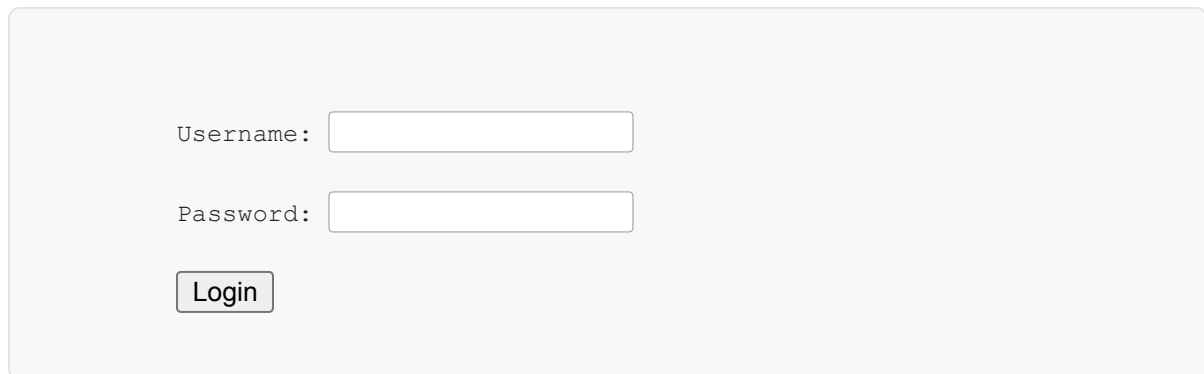
You are developing an administrative gateway. Access to the control panel requires validation. If credentials pass, the router redirects the administrator to the dashboard; if they fail, they are directed to an unauthorized error page.

Problem Description

Implement a Flask application containing the following endpoints:

1. **Route** `/login` (**GET, POST**):

- **GET Request:** Returns a string containing a raw HTML form with two input text boxes (names `"username"` and `"password"`) pointing to `POST /login` :



```

<div>
  Username: <input type="text"/>
  Password: <input type="password"/>
  <input type="button" value="Login"/>
</div>

```

- **POST Request:** Reads the form parameters `username` and `password` from `request.form` .
- If `username` matches `"admin"` **and** `password` matches `"Academy@admin2026"` :
- Redirect the client to the `/dashboard` route (using `redirect(url_for('dashboard'))`).
- If the credentials do not match:
- Redirect the client to the `/login-failed` route.

1. **Route** `/dashboard` (**GET**):

- Returns a formatted HTML page heading: `"<h1>Welcome to the Admin Dashboard!</h1>"` .

1. Route `/login-failed` (GET):

- Returns a JSON error response payload: `{"status": "Unauthorized", "message": "Invalid credentials provided."}` with an HTTP status code of `401` .

Assignment 4: Academy Course Enrollment System

Scenario

A course catalog registrar tracks student enrollments. You need to write a Flask controller that processes course selections via post form fields, stores enrollments in an in-memory roster list, validates duplicate registrations, and permits clearing logs.

Problem Description

Implement a Flask application with the following parameters and routes:

- 1. Roster Storage:** Initialize a global list in your script named `ENROLLED_COURSES = []` .
- 2. Route `/enroll` (GET, POST):**
 - **GET Request:** Returns an HTML string showing the current list of enrolled courses as a comma-separated list: `"Enrolled: <course1>, <course2>..."` . If no courses are enrolled, display `"Enrolled: None"` .
 - Below the list, include an HTML form to submit new enrollments:

Course Name:

- **POST Request:** Reads the form parameter `"course_name"` from the form.

- Strip any leading/trailing spaces from `"course_name"` .
- **Validation:**
- If `"course_name"` is empty, return a plain text error `"Error: Course name cannot be empty."` with HTTP status code `400` .
- If `"course_name"` (case-insensitive) already exists in the `ENROLLED_COURSES` list, return a plain text error `"Error: Already enrolled in <course_name>."` with HTTP status code `400` .
- **Success:** Append the original `"course_name"` to `ENROLLED_COURSES` , and redirect the user back to `GET /enroll` with HTTP status code `303` (See Other).

1. Route `/enroll/clear` (POST):

- Clears all elements from the global `ENROLLED_COURSES` list.
- Redirects the client back to `/enroll` with HTTP status code `303` .

Difficult Assignments

Assignment 5: Patient Ledger API & Query Module

Scenario

A hospital database tracks patient check-ups. You are writing a Flask web API to query patient tables. The API must receive optional query filters, compile dynamic parameterized SQL statements safely, and return structured JSON records.

Problem Description

Implement a Flask application connected to a local SQLite database named `clinic.db` :

1. Database Initialization:

- Before the app receives requests, check if the table `patients` exists. If not, create it:

```
id INTEGER PRIMARY KEY, name TEXT, age INTEGER, ailment TEXT, doctor TEXT .
```

- If the table is empty, insert three dummy records:

- `(1, "John Doe", 45, "Flu", "Dr. Smith")`
- `(2, "Jane Roe", 30, "Migraine", "Dr. Jones")`
- `(3, "Bob Vance", 50, "Flu", "Dr. Smith")`

1. API Endpoint 1: `GET /api/patients` :

- Expect optional search parameters in the query string (`request.args`): `ailment` and `doctor`.
- **Dynamic SQL:** Compile a safe parameterized SQL SELECT statement dynamically:
- If `ailment` is provided, append `WHERE ailment = ?` to the query.
- If `doctor` is provided, append `AND doctor = ?` (or `WHERE doctor = ?` if no ailment was provided).
- Execute the query on `clinic.db` using parameter lists/tuples. Do not format parameters directly into the query string.
- **Response:** Map matching rows to dictionaries:

`{"id": row[0], "name": row[1], "age": row[2], "ailment": row[3], "doctor": row[4]}`. Return this list as a JSON payload using `jsonify()` with an HTTP status code of `200`.

1. API Endpoint 2: `POST /api/patients/add` :

- Extract json parameters `name`, `age`, `ailment`, and `doctor` using `request.get_json()`.
- **Validation:** If any parameter is missing, or if `age` is not a positive integer, return a JSON error payload: `{"status": "Bad Request", "error": "Missing or invalid fields."}` with an HTTP status code of `400`.
- **Execution:** Insert the new record into `clinic.db` using a parameterized SQL query. Commit changes.
- **Response:** Return a success JSON payload: `{"status": "Created", "message": "Patient record added successfully."}` with an HTTP status code of `201`.

1. Ensure all database connections and cursors are closed cleanly inside your route handlers.

Assignment 6: Inventory Management API with JSON Schema Validation

Scenario

A warehouse stock-control ledger is exposed as a web service. The dashboard requires a RESTful JSON API to update quantity metrics of inventory parts. The API must validate incoming payload schemas, check item presence in SQLite databases, commit transaction edits, and handle error scenarios.

Problem Description

Implement a Flask application connected to a local SQLite database named `store.db` :

1. Database Initialization:

- On start, verify if table `items` exists:

```
id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT UNIQUE, quantity INTEGER, price REAL .
```

- Pre-populate the table with two records if empty:

- `("Laptop", 15, 1200.0)`
- `("Mouse", 50, 25.0)`

1. API Endpoint 1: `GET /api/items` :

- Queries and returns all rows in `items` table.
- Maps columns to a list of dicts: `[{"id": row[0], "name": row[1], "quantity": row[2], "price": row[3]}, ...]`
- Returns a JSON response with HTTP status code `200` .

1. API Endpoint 2: `PUT /api/items/update` :

- Expects a JSON request body (`request.get_json()`) containing:
- `"name"` (string)
- `"quantity"` (integer)
- **Schema Validation:**
- Verify both `"name"` and `"quantity"` keys are present in the JSON body.
- Check that `"quantity"` is an integer and is greater than or equal to `0` .

- If the validation fails, return a JSON error response: `{"error": "Invalid request payload. Ensure name and non-negative integer quantity are provided."}` with an HTTP status code of `400`.
 - **Database Check & Update:**
 - Connect to `store.db`. Execute a query to check if an item matching `"name"` (case-sensitive) exists in the table.
 - If the item does not exist, return a JSON error response: `{"error": "Item '<name>' not found in inventory."}` with an HTTP status code of `404`.
 - If it exists, update its `"quantity"` to the new integer value. Commit the transaction.
 - Return a JSON success response: `{"message": "Stock updated successfully.", "name": "<name>", "new_quantity": <quantity>"}` with an HTTP status code of `200`.
1. Close SQL connections and cursors cleanly within the route functions.
-

Objective

Combine web framework layouts with local databases, implement web scrapers using BeautifulSoup and custom request headers, write recursive crawlers, and simulate Scrapy Spider classes and pipeline components.

Easy Assignments

Assignment 7: Academic Quotes Collector (BeautifulSoup Scraper)

Scenario

You are building an educational dashboard. You need to write a scraper that extracts quotes and authors from the sandbox quotes website and saves the formatted results to a local text file.

Problem Description

Write a function `scrape_academic_quotes(url, output_file_path)` :

1. HTTP Request:

- Use the `requests` library to fetch the HTML content of the target quotes website `url` (e.g. `https://quotes.toscrape.com/`).
- Define a custom request headers dictionary containing a browser identity:

```
{"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"}
```

Pass this headers dictionary in your `requests.get()` call.

1. Parsing:

- Parse the page content using BeautifulSoup (`html.parser`).
- Locate all quote container blocks (represented by `div` tags with class `"quote"`).
- From each block, extract:
 - The quote text (inside a `span` tag with class `"text"`).
 - The author name (inside a `small` tag with class `"author"`).

1. Save:

- Write the parsed quotes and authors to a local text file at `output_file_path` in the exact format:

```
"Quote: <quote_text> | Author: <author_name>\n".
```

- 1. **Return:** The total count of quotes parsed and written (integer).

Example Walkthrough

```
total_quotes = scrape_academic_quotes("https://quotes.toscrape.com/", "scraped_q
print(f"Scraped {total_quotes} quotes.")
## Check your local folder for a file named "scraped_quotes.txt" containing 10 l
```

Assignment 8: SQLite-Powered Library Directory (Flask Web App)

Scenario

You are developing a library registry database portal. The backend needs to query book registers from an SQLite database and display them on a web page, and allow users to append new book titles using an HTML form.

Problem Description

Implement a complete Flask web application connected to a local SQLite database named `library.db`:

1. Database Setup:

- In your initialization code, check if the table `books` exists. If not, create it:

```
id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, author TEXT .
```

1. Flask Routes:

- `GET /books` :
 - Establishes an SQLite database connection and queries all records from the `books` table.
 - Formats and returns an HTML list string containing the book records. Each book must be rendered inside a list element:

```
<li><strong><title></strong> by <author></li> .
```

- Include a simple HTML form pointing to `POST /add-book` at the bottom of the page:

Title:

Author:

- `POST /add-book` :
 - Extracts the form fields `"title"` and `"author"` from the request (`request.form.get()`).
 - **Validation:** If either parameter is missing or empty, return a plain text error message `"Error: Title and Author are required!"` with an HTTP status code of `400` .

- If both are valid, execute a parameterized SQL INSERT query to append the book record to `library.db`, commit the changes, and redirect the client to `/books` (using `redirect(url_for('list_books'))` or direct path).
1. Ensure cursors and database connections are closed cleanly inside each route handler.

Note: Define the Flask app instance variable as `app`.

Medium Assignments

Assignment 9: Academic Tables Extractor (Pandas & BeautifulSoup)

Scenario

A research department analyzes national demographics tables. You need to write a utility that parses HTML tables from a target webpage and saves them to a CSV spreadsheet.

Problem Description

Write a function `extract_html_table_to_csv(url, table_id, output_csv_path)`:

1. Fetch & Locate:

- Fetch the raw HTML from `url` using `requests` with a custom `User-Agent` header.
- Use BeautifulSoup to locate the specific `table` element in the parsed DOM matching the given ID attribute (`id=table_id`).
- If no table matches the ID, raise a `ValueError` with the message: `"Table with id '<table_id>' not found."`

1. Pandas Parsing:

- Convert the BeautifulSoup table tag element to a string, and pass it to Pandas: `pd.read_html(str(table_element))`.
- Extract the first DataFrame from the parsed list of tables.

1. Clean & Save:

- Clean the DataFrame: Drop any columns that are entirely null/empty (`NaN`) using `.dropna(how='all', axis=1)` .
- Save the cleaned DataFrame to `output_csv_path` as a CSV file (set `index=False` to ignore row indexes).

1. **Return:** The total number of rows written to the CSV file (integer).

Example Walkthrough

```
## Extract and save population details from Wikipedia sandbox
url = "https://en.wikipedia.org/wiki/List_of_countries_by_population_(United_Nat
try:
    row_count = extract_html_table_to_csv(url, "wikitable", "wiki_population.csv
    print(f"Extracted {row_count} rows.")
except ValueError as e:
    print(e)
```

Assignment 10: Dynamic News Portal with Auto-Updater

Scenario

You are developing a news aggregation page. The web server needs a route `/refresh-news` that scrapes article headlines and links from a news portal, inserts them into an SQLite database, and displays them on a styled homepage.

Problem Description

Implement a Flask application connected to a local SQLite database named `news.db` :

1. Database Setup:

- Create a table named `articles` if it does not exist:

```
id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT UNIQUE, url TEXT .
```

1. Flask Routes:

- `GET /` :
- Queries all records from the `articles` table in `news.db` .
- Returns an HTML string rendering each article headline as a clickable link:

```
[CONSOLE] <a href="<url>"><title></a><br> .
```

- `GET /refresh-news` :
- Fetches raw HTML from Hacker News (`https://news.ycombinator.com/`) using `requests` and a custom `User-Agent` header.
- Parse the HTML with BeautifulSoup to locate the top 10 article title lines. (In Hacker News, these are represented by `span` elements with class `"titleline"` , containing a nested `a` tag).
- Extract the text of the link (the title) and the link destination (`href`).
- Loop through the top 10 articles and execute a parameterized SQLite query to insert them into `news.db` .
- **Constraint:** Use `INSERT OR IGNORE` to prevent database crashes when trying to insert duplicate titles.
- Commit the transaction, close database resources, and redirect the user back to the homepage `/` .

Difficult Assignments

Assignment 11: Recursive Crawler with Depth Limiter

Scenario

Search engine crawlers index web pages by recursively following hyperlinks on a page. You need to write a crawler that traverses links starting from a base URL up to a maximum depth limit, avoiding loops by tracking visited links.

Problem Description

Write a function `recursive_link_crawler(base_url, max_depth, max_links)` :

1. State Tracking:

- Track visited URLs in a set to avoid loops.
- Maintain a dictionary mapping each visited URL to a list of internal hyperlinks discovered on its page:

```
{url: [list_of_internal_links]} .
```

1. Recursion Details:

- Start crawling at `base_url` (Depth 0).
- Follow discovered internal links recursively up to `max_depth` (Depth `max_depth` represents the last depth layer where links are parsed but not crawled further).
- If the total number of unique visited URLs reaches `max_links`, stop crawling immediately.

1. Page Scraper Rules:

- Fetch each page using `requests.get()` with custom `User-Agent` headers. Set a timeout threshold of `3.0` seconds. If a request fails, times out, or returns a non-200 status, log a warning and skip the URL.
- Parse the page using BeautifulSoup and extract the `href` attribute of all `<a>` tags.
- Clean and filter links:
- If a link is relative (e.g. `"/about"`), convert it to absolute using `urllib.parse.urljoin(current_page, link)`.
- Ignore any anchor links (starting with `#`) or mailto links.
- **Domain Filter:** Only crawl and store links that belong to the **same domain (netloc)** as the `base_url`. (For example, if crawling `https://quotes.toscrape.com`, ignore links pointing to `https://github.com`).

1. **Return:** The dictionary of discovered links mapping.

Example Walkthrough


```
## Limit crawl to a maximum depth of 1 and a maximum of 5 unique pages
crawler_map = recursive_link_crawler("https://quotes.toscrape.com/", max_depth=1)

for page, links in crawler_map.items():
    print(f"Page: {page} | Found {len(links)} internal links.")
```

Assignment 12: Scrapy Database Pipeline and Product Crawler

Scenario

You are configuring a data pipeline for a web scraping project. In Scrapy, spiders crawl pages and yield scraped items, which are then passed through pipelines to be validated and saved to databases. You need to simulate this architecture in a single script.

Problem Description

Write a Python script that implements a Scrapy Spider class and a companion Database pipeline:

1. Class `QuotesSpider` (inherits from `scrapy.Spider`):

- Set properties:
- `name = "quotes"`
- `start_urls = ["https://quotes.toscrape.com/"]`
- Implement the `parse(self, response)` method:
- Iterate through quote container blocks on the page.
- For each quote, extract the quote text and the author name.
- Yield a dictionary containing: `{"text": <text_string>, "author": <author_string>}`.
- Find the pagination "Next" page button link (`response.css('li.next a::attr(href)').get()`).
- If it exists, yield a follow-up request to crawl it recursively:

```
yield response.follow(next_page, callback=self.parse)
```

1. Class `SQLitePipeline` :

- Implement the standard Scrapy Pipeline interface:
 - `__init__(self, db_name="quotes.db")` : Accepts a database name.
 - `open_spider(self, spider)` : Establishes a connection to the database and creates a table named `quotes` if it does not exist: `id INTEGER PRIMARY KEY AUTOINCREMENT, text TEXT, author TEXT`.
 - `process_item(self, item, spider)` :
 - Receives the yielded item dict from the spider.
 - **Validation:** Check if `text` or `author` keys are empty. If either is missing or empty, raise Scrapy's built-in `DropItem` exception (import `DropItem` from `scrapy.exceptions`).
 - If valid, execute a parameterized SQLite query to insert the quote text and author into the `quotes` table. Commit the transaction and return the `item`.
 - `close_spider(self, spider)` : Closes the database connection cleanly.
1. Write a short mock test driver at the bottom of the script that instantiates `SQLitePipeline` and calls its methods using mock dictionary items to demonstrate that the database insertion and validation work.
-